

The Hydra-16 and HydraOS

The whole system in one place

The guide, the tutorial, the guides for using it,
the programmer's guide and the hardware reference

2026-10-07

From the repository's documents (reborn/docs): node reborn/tools/docpdf.js

Contents

The guide

[The Hydra-16 and HydraOS: the whole system in one place](#)

First steps

[First steps with the Hydra-16](#)

Using the Hydra

[Using the Hydra-16](#)

[rc](#)

[The tools](#)

[Using HyForth](#)

[Using hylang](#)

[Using BASIC](#)

Programming

[The programmer's guide](#)

[Calling the system](#)

[Memory](#)

[Tasks, notes and time](#)

[Files and the namespace](#)

[Writing a server or a driver](#)

[Modules and programs](#)

[The screen: the Vera X](#)

[The assembly SDK](#)

[The C SDK](#)

The hardware

[The Hydra-16: hardware reference](#)

[Wiring the Vera X to the Hydra-16 through a bus breakout card](#)

The Hydra-16 and HydraOS: the whole system in one place

The Hydra-16 is a multitasking 65C02 computer: a W65C02S whose address decoding gives each of 16 tasks its own RAM, zero page, stack and bank selections, so a task switch is one register write. HydraOS is its operating system, Plan 9's way: one kernel in the BIOS ROM, everything else modules of the paged ROM run in place in tasks of their own, every device a file server, and each task a namespace of its own.

This guide covers every part of it, hardware and software, with a short account of each and links to the documents that cover it in full. Read it top to bottom for the whole picture, or go to a part from the contents. It describes HydraOS 1.0 and what's come since (October 2026); [status.md](#) says where each part stands. To print: [hydra-16.pdf](#), the guide, the tutorial, the guides, the programmer's guide and the hardware reference as one book (`node tools/docpdf.js` makes it again).

Contents

1. [The machine in one page](#)
2. [Getting started](#)
3. [The hardware](#)
4. [The ROMs](#)
5. [The kernel: tasks, interrupts, memory, notes](#)
6. [Calling the system](#)
7. [Files, devices and namespaces](#)
8. [The console: windows, the serial port, the screen](#)
9. [Storage: disks and HydraFS](#)
10. [The shells](#)
11. [The languages](#)
12. [Programs: assembly and C](#)
13. [Sound](#)
14. [Video: the Vera X](#)
15. [The tools](#)
16. [POST, the hardware test and the kernel's messages](#)
17. [On the PC: the build, the emulator, the tests, the PC tools](#)
18. [The source tree](#)
19. [Status and the design](#)
20. [All the documents](#)

1. The machine in one page

CPU	WDC W65C02S, 3.58 MHz as built (7.16 with jumper J8)
Tasks	16, each with its own 32K of RAM (\$0000-\$7FFF, zero page and stack included) and its own RAM and ROM bank selections: a task switch is one write to T
Memory	512K of task RAM; 2 MB of shared RAM and up to 15 task RAM modules of 2 MB (three as built: 48 banks of 8K for each task), through an 8K window at \$8000; 4 MB of paged ROM in 16K banks at \$A000; a 128K BIOS ROM in 8K pages at \$E000
Interrupts	16 prioritised IRQ lines, each with its own vector
Devices	A 65C22 VIA (the tick, SPI, I2C, GPIO), a 65C51 ACIA (the serial console), a YM2151 (stereo FM sound), SD cards on SPI, a DS1747 clock in U7 if one's fitted
Video	The Vera X in slot 0: the Commander X16's VERA (VGA at 640x480, two layers, 128 sprites, 256 colours, a 16-voice PSG, PCM)

Expansion	6 slots (an 8-bit bus), 8 SPI device headers, a GPIO and I2C header
Kernel	In the BIOS ROM: a preemptive scheduler (200 ticks a second), one interrupt path, calls between tasks, memory and banks, shared segments, semaphores, notes, files' channels, namespaces; one jump table, made from a specification
Modules	Everything else: the drivers, init, the shells, the tools and the languages, each a module of the paged ROM run in place in a task of its own
Files	Plan 9's: every device a file server; 16 fds a task; a namespace each, of binds and mounts with union directories; pipes; <code>/proc</code> , <code>/env</code>
Storage	HydraFS on SD cards (<code>/sd/N</code>), RAM disks (<code>/ram</code> , <code>/sram</code>), and the ROM disk (<code>/rom</code>)
Shells	HyForth over rc, the login shell; rc, Plan 9's; hylang and BASIC as shells too
Languages	HyForth (Forth 2012), hylang (danlang, a lisp, with a bytecode machine and native code), BASIC (Microsoft's, by way of EhyBASIC)
Programs	In assembly (ca65 on a PC, or as on the Hydra) or C (cc65): modules of the paged ROM, or programs in files, read into RAM at <code>\$0800</code>
Tools	The core tools (Plan 9's way), <code>edit</code> (a screen editor), <code>ed</code> , <code>db</code> (a debugger), <code>as</code> , <code>play</code> (ZSM songs, scores, WAV files), <code>xmodem</code> ; on the PC an emulator that runs the real ROMs (with the Vera X and the sound), the regression tests, and the PC tool (<code>/pc</code> , and the terminal)

The tasks as the system starts:

```

task 0  kernel    the scheduler, IRQs, calls, memory, notes, channels, namespaces (the BIOS ROM's)
task 1  init      the RAM disks, the namespace, window 0's shell, wstart (a shell for each window made)
task 2+ programs  the shells, the tools, yours: in place from the paged ROM, or in RAM from $0800
task A  vid       #v /dev/vid: the Vera X (with no card it ends as it starts)
task B  gpio      #g /dev/gpio, #i /dev/i2c
task C  snd       #a /dev/snd, sndctl, bell, psg
task D  kdev      the kernel's devices: the root, null, ticks, modules, /proc, pipes, /env, segments
task E  storage   #S /dev/spi, #d /dev/sd, #f HydraFS: /rom, /ram, /sram, /sd/N
task F  cons      #c /dev/cons and the windows, #P /pc

```

More: [the system in one page](#) (the programmer's guide), [the hardware's overview](#).

2. Getting started

In the emulator (no board needed: it runs the same images): from `reborn/`, with Node.js 18 or later and cc65,

```

node build.js          the BIOS ROM and the paged ROM's chips, into bin/
node sim/run.js -i      the Hydra's serial console in your terminal (Ctrl-A x quits, Ctrl-A h helps)

```

The images are in Git too (`bin/`), so `node sim/run.js -i` works without cc65. [The tutorial](#) is the first hour: the shell, files and disks, windows, the languages, sound, and a program of your own.

On the board:

- Program the chips (an EPROM programmer; there's no write path on the board): `bin/bios.bin` into the BIOS ROM's socket (U6, an SST39SF010 or bigger), and `bin/prom0.bin`, `prom1.bin`, `prom2.bin`, `prom3.bin` into U31, U32, U34 and U36, the first four paged ROM sockets in their chip-select order, each at offset 0 ([the paged ROM](#)).
- Jumpers: J4 (RDY) fitted, and one CPU clock: J7, 3.58 MHz ([connectors and jumpers](#)).
- A terminal on the DE-9 (J3), 9600 baud, 8 bits, no parity, 1 stop bit, RTS/CTS, through a straight-through cable ([the ACIA](#)). The PC tool (`sim/tools/hydrapc.js`) is a terminal that also serves a folder of the PC as `/pc`.
- An SD card adapter on J18 (SPI device 0), if you have one: the card is `/sd/0`.

The boot prints POST's lines, the drivers starting, and HyForth's prompt, `/>`.

3. The hardware

The full reference is [hardware.md](#): the board (V1) as its schematic describes it, every chip, signal, connector and jumper, and the cards. In short:

The memory system. `$0000-$7FFF` is the task's own RAM: one 512K chip (U7) with `T` on its top four address lines, so 16 tasks each have 32K, zero page and stack included. `$00` and `$01` are each task's bank registers (writes; reads come from its RAM): `$00` picks the 8K bank at `$8000-$9FFF` (a task RAM module's, 16 banks a task each, or a shared bank, `$F0-$FF`: 16 of the board's 2 MB, the 128K that `U` picks), and `$01` the 16K bank of the paged ROM at `$A000-$DFFF` (4 MB in eight SST39SF040s). `$E000-$FFFF` is the BIOS ROM, its 8K page chosen by `w`. On the V1 board a bank number's bits 2 and 3, and 6 and 7, trade places on the way to the chips: software doesn't notice, but the ROM images are laid out for it. [The memory map](#), [the pseudo-registers](#), [task RAM and the bank registers](#), [the paged RAM window](#), [the paged ROM](#), [the BIOS ROM](#).

I/O, interrupts, clocks and reset. `$FF00-$FFEF` is 15 ports of 16 bytes (the VIA, the ACIA, the YM2151, and two for each slot); `$FFF0-$FFF3` are `T`, `U`, `V` and `w`. Sixteen IRQ lines in priority order (the VIA, the ACIA, slot 0, the YM2151, the other slots), each with its own vector from a vector RAM (its index the line $\wedge 7$). A 14.318 MHz crystal divided down: the CPU clock by a jumper, the YM2151's 3.58 MHz, the ACIA's 1.79 MHz. Reset resets the CPU and the chips, not the pseudo-registers, the bank registers or RAM. [I/O space](#), [interrupts](#), [clocks](#), [reset](#), [power and bus control](#).

The devices. The VIA: port A on a header (GPIO, the I2C bus on PA0/PA1, CA1 and CA2), port B the SPI bus to eight device headers (and eight more for cards), timer 1 the system's tick, timer 2 the serial port's pacing. The ACIA: the serial console, RS-232 on a DE-9 (its clock 2.9% slow, so 9600 is about 9,320). The YM2151 and its YM3012 DAC, on a mixer with each slot's audio and a line input. [On-board devices](#).

Cards, connectors and the board as built. Six slots, 62-pin card edges ([expansion slots](#)); memory daughter cards (a task RAM module each); a bus breakout card for a logic analyzer; and the Vera X, whose carrier card is still to be made ([companion cards](#), [the Vera X](#)): till then it's wired through the breakout card ([vera-wiring.md](#)). The board's quirks are its [V1 errata](#).

The schematics are KiCad 9's, in `../.. /board/`: the main board's sheets (AddressDecode, BankedROM, Buffers, Clocks, Connectors, FFF_Registers, IRQ_P_E, Mixer, SharedMem, Sound, ZPMirrorRAM), the memory daughter card's and the bus breakout card's. `include/hw.inc` is the board as the software sees it, its names made from the reference.

4. The ROMs

The BIOS ROM (`bin/bios.bin`: 16 pages of 8K, the kernel and nothing else). Every page starts with the reset stub at `$E000` (`w` isn't reset), has the COMMON block at `$FD00` (the IRQ entry and exit, the kernel's far call between pages) and its vectors at `$FFFA`. Page 0 is what runs often (the IRQ path, the scheduler, calls between tasks, `kcop`, a task's side of the calls that wait, the console's calls) and the jump table at `$F800`; page 1 the kernel task's side of the task calls, memory, semaphores, notes and the clock; page 2 files and the environment; page 3 namespaces and the loader (SPAWN); page 4 POST and the debugger's steps; the rest is room. Page 0 is the scarce one. [The kernel's pages](#).

The paged ROM (`bin/prom0.bin` ...: a 512K image for each chip it fills; four now, some 113 of its 256 banks). Bank 0 holds the module directory and the ROM disk's partition table; bank 1 the hardware test (the old system's, unchanged); the modules from bank 2, each at `$A000` of its first bank (about fifty: the drivers, init, the shells, the tools, the languages; a module may span two to eight banks); then the ROM disk's HydraFS volume (`/rom`: the programs that run from RAM, the languages' libraries, songs, the SDK's samples and include files, the calls' reference, some 480K). `modules/rom.txt` lists the modules, `romfs/romfs.txt` the ROM disk's files. [Modules](#).

5. The kernel: tasks, interrupts, memory, notes

Tasks. Sixteen, task 0 the kernel's own. SPAWN starts a program by its path through the caller's namespace (a module of the paged ROM runs in place; a program in a file is read into the new task's RAM at `$0800` by the task itself); WAIT takes a child's exit status (a code and a message: Plan 9's); a task's parent, its note group (the console's Ctrl-C reaches a group), and `/proc/N` (its

state, args, cwd, fds, environment, namespace, registers and memory as files; its `ctl` stops, starts and steps it). [Tasks](#).

Scheduling and interrupts. Preemptive, round robin, 200 ticks a second (the VIA's timer 1); a task may hold the CPU for a while (`PREEMPT_OFF`), sleep, or wait for an event. Every interrupt comes through one path in the `COMMON` block to its line's owner; a driver; the longest stretch with interrupts off is held under 200 cycles, so 115200 baud keeps up. [The conventions' interrupts](#).

Memory. A task's 32K: its program and data, a break (`BREAK`) for more; RAM banks (`BANKS_ALLOC`, 8K each through the window); shared segments of the shared RAM, by name (`/dev/seg`); semaphores (16, counting ones and mutexes). [Memory](#). The C SDK's multitasking demos show tasks sharing a segment and taking turns with semaphores, and draw them as they go: `race` (lost updates, then a mutex), `chorus` (the console shared: a mutex, a baton), `philo` (the dining philosophers, and a deadlock), `prodcons` (a ring and counting semaphores) and `round` (four tasks singing a round, each keeping its own time). [The demos](#).

Notes are Plan 9's signals: by name or number (`interrupt`, `kill`, `hangup` ...), to a task or its group, caught by a handler or not.

Calls between tasks: a server answers in its own task; the kernel copies between tasks (`kcopy`); a driver's `irq` entry runs in its own task too. [Reaching other tasks](#).

6. Calling the system

A program calls the system with `jsr` to the call's slot in the jump table (`$F800` up, on BIOS ROM page 0), its arguments in `.A`, `.X`, `.Y` and the call registers `r0-r15` (`$02-$21` of its zero page), its error in carry and `.A`. The calls are written down once, in `spec/api.def`, and the build makes from it the jump table, `hydra.inc`, C's `hydracalls.h`, HyForth's `sys- words`, `hylang's sys-` functions, BASIC's `SYS "NAME"`, and the reference, `/rom/doc/api.md` on the Hydra. [Calling the system](#).

7. Files, devices and namespaces

Files. A task has 16 `fds`, each naming a channel; `OPEN`, `CREATE`, `READ`, `WRITE`, `SEEK`, `CLOSE`, `STAT`, `PIPE`, `DUP`, `FD2PATH` ...: the same calls whatever's behind them. A directory reads as `stat` records.

Devices are file servers: drivers' (`#c` the console, `#d` the disks, `#f` HydraFS, `#s` SPI, `#g` GPIO, `#i` I2C, `#a` sound, `#v` the Vera X, `#p` /`pc`) and the kernel's own (`#/` the root, `#n` null, zero and the kernel's messages, `#t` the ticks and the time, `#m` the modules, `#p` /`proc`, `#e` /`env`, `#|` pipes, `#s` segments, `#r` raw RAM). A device is controlled by writing commands to its `ctl`. Every server is built on `srvlib`. [Files and the namespace, servers and drivers](#).

Namespaces. Each task's tree of names is built by binds and mounts, with union directories: `/bin` is the RAM disks', a card's, the ROM disk's and the ROM's own programs together; the first found first, so no search paths. The default namespace is `/rom/lib/namespace` (`newns`); a child shares its parent's till one of them changes it. [The namespace, NAMESPACES.md](#).

The environment is each task's own (8K), served at `/env`: `rc`'s variables, and anyone's.

8. The console: windows, the serial port, the screen

The console driver (`cons`, task `F`) serves `#c`: **windows**, `rio`'s way on a serial terminal: each a whole console with its own shell, shown one at a time (`Ctrl-]` and a digit shows that one, `Ctrl-]` `c` makes one, `Ctrl-]` `n` the next), a hidden one running on, its output kept and shown again. A read is a line, edited at the console (Backspace, the arrows, Home, End, `Ctrl-U`, the lines before); `consctl` turns raw keys on; `Ctrl-C` (an interrupt) and `Ctrl-\` (a kill) are notes to the shown window's group. **The serial port** runs at 9600 at boot, and to 115200 (`/dev/serctl`), every byte paced by VIA timer 2. **The screen:** with a Vera X, the shown window is on its screen too (`consctl`'s `screen`, `serial`, both). [The tools, the screen](#). Text windows (screens in the console's RAM banks, a whole VT100, window groups, headers and a bar) are being built: [WINDOWS.md](#).

9. Storage: disks and HydraFS

The storage driver (storage, task E) owns the SPI bus and the disks, at /dev/sd: 0-f the SD cards by their SPI device (through a cache of their blocks), x the ROM disk, r the RAM disk (each shell has its own area, /ram) and s the shared one (/sram). HydraFS is on each, the old system's file system, ported: directories, files to 4 GB, a card's partitions ([HYDRAFS.md](#) is its format). The cards are at /sd/N, and a card's bin and lib join /bin and /lib. df, mkfs, fsck and label look after them; the PC's `sim/tools/hydrafs.js` makes card images. [Disks](#), [DISKS.md](#).

10. The shells

HyForth over rc is the login shell (forth -l, as /lib/shell names it; a card's /lib/shell can name another): a line whose first word is a Forth word or a number is Forth, any other an rc command line. **rc** is Plan 9's: lists, quoting, if, for, while, switch, functions, redirections, pipelines, background tasks, \$status, globs, scripts (#!/bin/rc); it's the shell of scripts and system(). `hylang -l` and `basic -l` are shells too, by the same rule (a line of the language's, or rc's). Each window has a shell of its own, its namespace built as it starts, then its profile. [rc](#), [HyForth](#), [hylang](#), [BASIC](#).

11. The languages

HyForth (forth): Forth 2012 (Core, Core Extension, Exception, Facility, File Access, Programming-Tools, Search-Order, String, Double, Locals, Memory-Allocation, Block), passing the Forth 2012 test suite's tests of them; its core in the ROM, the other word sets pre-compiled libraries from /lib/forth; a sys- word for every call; the Hydra's words (tasks, notes, namespaces, banks, devices, sound); the old HyForth's pieces back. [The guide](#), [its design](#), [its status](#).

hylang (hylang): danlang on the Hydra, a lisp with big integers, exact fractions, hashes and closures, and the Hydra's files, tasks, memory and devices a function away; danlang's own regression suite passes on it, and a bytecode machine and native code make it 1.9 times HyForth's time over twenty benchmarks. [The guide](#), [its design](#).

BASIC (basic): Microsoft BASIC 2A by way of EhyBASIC, the Hydra-16's own, as a program: files, sound and `PLAY`, `SYS` (machine code, or any call by name), a RAM bank, scripts and pipelines, and a shell mode. [The guide](#), [its design](#).

12. Programs: assembly and C

Every executable is a HX2 module: a 48-byte header, then its code and data. **A program in a file** is linked for \$0800 and read into its task's RAM as it starts (a card, a RAM disk, /pc); **a module of the paged ROM** runs in place, its data and BSS in its task's RAM, and may span two to eight banks; **a library module** is code other modules call (XCALL). [Modules and programs](#).

- **Assembly:** the SDK, `sdk/asm` (hydra.inc, hyx2.inc, macros.inc, toollib, srvlib, nslib, samples), with ca65 and ld65 on a PC (node build.js prog DIR), or as **on the Hydra itself:** the same language, the SDK's files in /lib/as, the same program, byte for byte. [The assembly SDK](#), [the assembler](#).
- **C:** cc65 with the Hydra's library under the standard one (files and stdio, the environment, system, signal over notes, conio, the sound's snd.h, the Hydra's own calls in hydra.h). [The C SDK](#).
- **Debugging:** db on the Hydra (a program started stopped, stepped, run to breakpoints, with ld65's symbols), and the emulator's call traces, breaks and monitor. [The debugger](#).

The samples are on the ROM disk (/rom/sample; `bind -a /rom/sample/c /bin` runs the C ones by name), the C SDK's multitasking demos among them. [The programmer's guide](#) is the way in.

13. Sound

The YM2151 (8 FM channels) is the sound driver's (`snd`, task C, #a): `/dev/snd` takes register and value pairs, `/dev/sndctl` claims channels, sets the volume, and takes each channel's command as text (`echo note 0 60 >/dev/sndctl`), so every language and rc make sound the same way; `/dev/bell` rings. ****With a Vera X, channels 8-23 are its PSG's voices (the same commands, and wave), and its **PCM** plays samples (`/dev/vid/pcm`, `pcmctl`). `play` plays the X16's ZSM songs (their PSG and PCM parts too), scores in the score language (compiled as they play), a line of it or a chord (`-m`, `-c`; the X16's MML with `-x`), and WAV files. The languages' words: C's `snd.h`, HyForth's `lib sound`, `hylang's` (`use "snd"`), BASIC's `SOUND` and `PLAY`. In the emulator, `run.js -i --sound` plays it in a browser and `--wav FILE` records it. [Sound and scores](#), [SOUND_PARITY.md](#), [SOUND.md](#).

14. Video: the Vera X

The Vera X's driver (`vid`, task A) finds the card as the system starts, sets it up for the console, and serves it at `/dev/vid: ctl` (modes 80x60, 80x30, 40x30, a bitmap under the text, the cursor, claims), `term` (an ANSI terminal, where the console writes), `vram`, `pal`, `sprites`, `font`, `frame` (a frame waited for), `psg`, `pcm` and `pcmctl`. A program can draw by writing those files, or **claim** the chip and write its registers itself, as an X16 program does. In the emulator, `--vera` puts one in slot 0 and `--view` shows its screen in a browser. The languages' graphics words, the keyboard (an input controller on the card) and the carrier card come next. [The screen, the card, VIDEO.md](#).

15. The tools

The programs in `/bin` behave as Plan 9's do (flags first, `fd 0` when given no names, errors as `tool: name: why, $status`): files (`ls`, `cat`, `cp`, `mv`, `rm`, `mkdir`, `rmdir`, `touch`, `du`, `pwd`, `cmp`), text (`echo`, `wc`, `head`, `tail`, `grep`, `sort`, `uniq`, `tee`, `xd`, `more`), the editors (`edit`, `nano's way`, its text in RAM banks; `ed`, the line editor), tasks (`ps`, `top`, `kill`, `slay`, `sleep`, `ns`), the debugger (`db`), the assembler (`as`), the system (`date`, `free`, `mods`, `hwtest`), the disks (`df`, `mkfs`, `fsck`, `label`), and others (`play`, `xmodem`). `/pc` is a folder of the PC, through the PC tool. [The tools](#).

16. POST, the hardware test and the kernel's messages

POST runs as the system starts: the `T` lines (each task's byte kept apart in its zero page, stack, OS area and RAM), the shared RAM, the `w` lines (each BIOS ROM page knows its number), the `U` lines, and the first bank of each shared RAM chip and of each task RAM module installed (its bank, data and address lines):

```
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000
POST ok
```

A bad line is reported by its number (a bit set), and the memory calls leave a bad module or chip unused. ****The hardware test**** is the old system's, unchanged, in paged ROM bank 1: a `T` typed during POST, or `hwtest` at the shell, starts it ([its guide](#)). **The kernel's messages** (the boot's, POST's, a driver's) are `/dev/kmesg`, its last 4K.

17. On the PC: the build, the emulator, the tests, the PC tools

- **The build**, `node build.js` (Node.js and cc65): the BIOS ROM, the paged ROM's chips, the ROM disk, the SDKs (`bin/sdk`), and a report of each part's size and room left; `--clock 2` for a 7.16 MHz board, `--acia wdc` for a WDC W65C51N; `node build.js prog DIR` for a program of your own.

- **The emulator**, `node sim/run.js`: the board cycle by cycle, running the real images. `-i` is the serial console live (Ctrl-A x quits, r the reset button, b a monitor: steps, registers, memory, breaks, watches); `--sd card.img` a card; `--pc-dir DIR` a folder as `/pc`; `--vera` a Vera X, `--view` its screen in a browser; `--sound` the sound in a browser; `--wav FILE` in a file; `--trace-calls`, `--break`, `--watch` for debugging. The top of `sim/run.js` lists them all; [the hardware reference](#) says what's modelled.
- **The tests**, `node sim/test.js`: 86 of them, each booting its own image and judged on its output, its time budgets and its own checks, as many at a time as the PC has cores; `--dl` runs them in the danlang emulator (`sim/dl`), the emulator written again in danlang.
- **The PC tools** (`sim/tools`): `hydrapc.js` (the PC tool: the terminal, and `/pc` over the serial line; `npm install` in `sim/` for its serial port), `hydrafs.js` (card images), `pcfs.js` (`/pc`'s server), `hysong.js` (scores to ZSM songs).

The [README](#) has every command.

18. The source tree

Folder	What's there
<code>spec/</code>	The system calls and the error codes: the one source of the jump table, <code>hydra.inc</code> , the reference
<code>include/</code>	<code>hw.inc</code> (the board), <code>layout.inc</code> (where the kernel's state lives)
<code>kernel/</code>	The kernel: the BIOS ROM's pages
<code>modules/</code>	The paged ROM's modules, a folder each, and <code>rom.txt</code>
<code>forthlib/</code>	HyForth's libraries
<code>programs/</code>	The ROM disk's programs (<code>/rom/bin</code>)
<code>romfs/</code>	The ROM disk's files, and <code>romfs.txt</code>
<code>sdk/</code>	The assembly and C SDKs
<code>tools/</code>	The build's tools: the specification's outputs, the ROM images, the ROM disk, the budgets, hylang's snapshot
<code>sim/</code>	The emulator, the tests' runner, the browser view, the sound, the danlang emulator (<code>dl/</code>), the PC tools (<code>tools/</code>)
<code>tests/</code>	The tests, their modules and programs, and the Forth, hylang and BASIC suites
<code>bin/</code>	The ROM images (in Git), and the SDKs the build copies out
<code>docs/</code>	These documents

The [tree](#) in full. Beside `reborn/`: `../..board/` (the KiCad files) and `../..old/` (the old system, frozen).

19. Status and the design

[status.md](#) is where everything stands, step by step against the plan, with what the spikes and budgets measured and what's next; HyForth's own steps are [forth-status.md](#). The [design](#) is how HydraOS came to be: the plan it was built to ([reimplementation-from-scratch.md](#)) and the design notes ([design/plans](#)). The rules every source follows are [conventions.md](#).

20. All the documents

Document	What it covers
hydra-16.md	This guide: the whole system in one place
hydra-16.pdf	The book to print: this guide, the tutorial, the guides, the programmer's guide and the SDKs', and the hardware reference
tutorial.md	The first hour: switching it on, the shell, files and disks, windows, the languages, sound, a program of your own
hardware.md	The board, the cards and the Vera X, in full

Document	What it covers
vera-wiring.md	The Vera X wired to the board through a bus breakout card: the glue logic, pin by pin, and bringing it up
using/	The guides: rc , the tools , HyForth , hylang , BASIC
programming/	The programmer's guide: calls , memory , tasks and notes , files and namespaces , servers and drivers , modules , video
../sdk/asm/README.md , ../sdk/c/README.md	The SDKs: building and running programs in assembly and in C
conventions.md	The rules every source follows
hyforth.md , hylang.md , basic.md	The languages' designs
status.md , forth-status.md	Where everything stands, and what was measured
design/	The plan HydraOS was built to, and the design notes
../tests/forth/README.md , ../tests/hylang/README.md	The Forth 2012 test suite, and danlang's
../README.md	Building, running and testing, and the tree
/rom/doc/api.md (on the Hydra; the build's obj/gen/api.md)	Every system call: its registers, errors and words

The old system's documents are in [../old/docs](#), frozen with it.

First steps with the Hydra-16

A tutorial for a first hour with HydraOS: switching it on, the shell, files and disks, tasks and windows, the languages, sound, and a program of your own in assembly and in C. It needs no board: the emulator runs the same ROM images the board does, and everything here works the same on one.

You need [Node.js](#) (18 or later) and a clone of the repository; for the programs at the end, [cc65](#) too (the build looks in `$CC65_BIN`, `$CC65_HOME/bin`, then the `PATH`).

1. Switch it on

From `reborn/` in the repository:

```
node build.js
node sim/run.js -i
```

The first builds the BIOS ROM and the paged ROM's chips (`bin/`); the second boots them in the emulator; its terminal the Hydra's serial console, in real time. (On the board, the same images go into the BIOS ROM's socket and the paged ROM's, and a terminal at 9600 baud is the console.) The boot shows POST, the drivers starting, and then the shell's prompt:

```
HydraOS 1.0 for the Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
...
task 1: init
init: up in task 01
...
HyForth (Forth 2012), bye to end
/>
```

The prompt is the current directory and `>`. In the emulator, `Ctrl-A x` quits, `Ctrl-A r` presses the reset button, and `Ctrl-A h` lists the rest. `node sim/run.js -i --vera --view` gives it a Vera X card too: its screen, which shows the console as the terminal does, is at <http://localhost:8016>.

2. The shell

The login shell is HyForth over rc: a line whose first word is a Forth word or a number is Forth; any other line is an rc command line, run by rc. So both of these work at the same prompt:

```
/> 2 3 + .
5
/> : sq dup * ;
/> 7 sq .
49
/> echo hello | wc
      1      1      6
```

The console edits the line as you type: Backspace, the arrows, Home and End, `Ctrl-U`, and Up and Down for the lines before. `Ctrl-C` stops what's running.

3. Files

Everything is a file in one tree of names. `ls /` shows its top:

```

/> ls /
bin/
dev/
env/
lib/
mnt/
pc/
proc/
ram/
rom/
sd/
sram/
tmp/
/> cd /rom
/rom> ls
README
bench/
bin/
doc/
lib/
sample/
songs/
/rom> cat README
The Hydra-16's ROM disk
...

```

`/rom` is the ROM disk, read only, there with no card. `/ram` is your own area of the RAM disk (each shell has one), `/sram` the shared RAM disk, `/sd/0` the card in SD device 0, `/dev` the devices, `/proc` the tasks. `cd` alone goes back to `/`.

Make a file with `edit`, the screen editor: `edit /ram/todo`, then type its lines (Enter after each), Ctrl-O and Enter to save it, Ctrl-X to leave. The two lines at the bottom name the other keys (^K cut, ^U paste, ^W find ...), and Ctrl-G shows them all ([using/tools.md](#)). `ed` is the line editor, ed's way.

```

/> edit /ram/todo
/> cat /ram/todo
buy a card
fix the clock

```

Redirection and pipes are rc's: `echo hi >/ram/f, cat /ram/f | wc -c, ls /rom/bin | sort -r`. The tools are [using/tools.md](#).

4. A card

In the emulator, a card is an image file. `sim/tools/hydrafs.js` makes one and puts files on it:

```

node sim/tools/hydrafs.js mkfs card.img 8 MYCARD
node sim/run.js -i --sd card.img

```

The card is `/sd/0`; what's written there is in `card.img`, which `hydrafs.js ls card.img` lists:

```

/> echo a note >/sd/0/note.txt
/> ls -l /sd/0
--rw-rw-rw- f0      7 2000-01-01 00:00 note.txt

```

A card's `bin` and `lib` join `/bin` and `/lib`, so programs and libraries on it run by name. `df` shows the disks, `mkfs`, `fsck` and `label` work on them. The clock starts at 2000-01-01 with no DS1747; `date` shows it, and `echo 2026-10-06 12:00:00 >/dev/time` sets it.

5. Tasks and windows

Each program runs in a task of its own (16 in all). `ps` lists them; `&` runs one in the background:

```
> sleep 100 &
> ps
task state parent cpu group name
0 ready - 10.1 0 kernel
1 wait - 0.1 1 init
2 wait 1 0.3 2 forth
...
5 sleep 4 0.0 4 sleep
...
> kill 5
```

`/proc/5` has a task's state as files: `status`, `args`, `fd` (its open files), `regs`, `mem`, and `ctl`, which takes `stop`, `start`, `kill`.

The console has windows, as `rio` has on Plan 9: each is a whole console with its own shell. `Ctrl-] c` makes one (and shows it), `Ctrl-] a` and a digit shows that window, `Ctrl-] n` the next. A window that isn't shown runs on; its output is kept, and shown again when you come back. `echo $window` says which one you're in.

Tasks working together. The C SDK's multitasking demos start four or five copies of themselves, each in a task of its own, and draw what they do as they do it: memory they share (a shared segment), and semaphores to take turns and wait for each other. Put them in `/bin` first, then run them one at a time (each needs the tasks):

```
> bind -a /rom/sample/c /bin
> race
lost updates on a shared counter, then none with a mutex
> chorus
four tasks print on one console: tangled, then whole lines, then in turn
> philo
the dining philosophers (Ctrl-C ends it); philo -d deadlocks, and says so
> prodcons
producers and consumers through a ring: counting semaphores
> round
a four-voice round, each voice a task keeping its own time
```

The C SDK's [guide](#) says what each shows, and their sources are in `sdk/c/samples`.

6. The languages

HyForth is the shell already: definitions, `.s`, words, files with `include`, and libraries with `require` (`require tools.fl`). [using/hyforth.md](#) is its guide.

hylang is the lisp, danlang on the Hydra:

```

/> hylang
hylang (danlang on the Hydra-16)
Type 'exit' to Exit

hylang> (+ 1/3 1/6)
=> 1/2
hylang> (map (fn {n} {* n n}) {1 2 3})
=> {1 4 9}
hylang> (ls "/rom")
=> {"README" "bench" "bin" "doc" "lib" "sample" "songs"}
hylang> exit

```

[using/hylang.md](#) is its guide. BASIC is Microsoft's, as the PET and its cousins had it: `basic`, then program lines and `RUN`, or `basic prog.bas` for a file ([using/basic.md](#)). And `rc` itself, for an `rc` session: `rc` ([using/rc.md](#)).

7. Sound

`scom` plays a short song on the YM2151 (in the emulator; hear it with `node sim/run.js -i --sound`, then the Sound button at <http://localhost:8016>): `play /rom/songs/test.zsm` plays a ZSM file, the format the Commander X16's tools (and Furnace) export. `play -m 0 't180 o4 l8 c d e f g'` plays a line of the score language (its notes, lengths and octaves: [using/tools.md](#), "Scores"), and `echo note 0 60 >/dev/sndctl` a note (`echo off 0 >/dev/sndctl` ends it). With a Vera X, channels 8-23 are its PSG's voices (`echo note 8 69 >/dev/sndctl`), and `play` plays a WAV file on its PCM.

8. A program of your own

The SDKs are in `reborn/sdk`: `asm` for assembly and `c` for C. Copy a sample to a folder of your own and build it:

```

mkdir hi
cp sdk/asm/samples/hi/hi.s hi/
node build.js prog hi

```

That makes `hi/hi.hyx`, a program for the Hydra. To run it, give the emulator the folder as `/pc` (what the PC tool, `sim/tools/hydrapc.js`, does for a board over the serial line):

```

node sim/run.js -i --pc-dir hi

```

```

/> /pc/hi.hyx one two
Hello, one!
Hello, two!
I'm task 5, in /, in window 0.
/> cp /pc/hi.hyx /ram/bin/hi
/> hi three
Hello, three!
I'm task 5, in /, in window 0.

```

`/ram/bin` is the first member of `/bin`'s union, so a program there runs by its name. The Hydra assembles one itself too: `as /lib/as/hi.s /ram/bin/hi` makes the same program (the SDK's include files and three of its samples are in `/lib/as`; [using/tools.md](#)), and `edit` is there for a source of your own. A C program is the same: a folder of `.c` files (`cp sdk/c/samples/hello/hello.c myc/`, then `node build.js prog myc`):

```
/> /pc/hello.hyx  
hello from C
```

[../sdk/asm/README.md](#) and [../sdk/c/README.md](#) are the SDKs' guides, and [programming/README.md](#) the programmer's guide: the system calls, memory, tasks and notes, files and namespaces, and writing a server or a driver.

Where next

- [hydra-16.md](#): the guide to the whole system, hardware and software, with links to the rest.
- [using/README.md](#): the guides for rc, the tools, HyForth, hylang and BASIC.
- [/rom/doc/api.md](#) on the Hydra: every system call, its registers and errors.
- [programming/README.md](#): the programmer's guide, for programs of your own.
- [hardware.md](#): the board, its cards and the Vera X, in full.
- [status.md](#): where the system stands, and what each part measured.

Using the Hydra-16

The guides for using HydraOS (reborn/), the Hydra-16's operating system. The first hour is [the tutorial](#); the whole system, hardware and software, is [the guide](#); and these are the guides for each part:

Guide	What it covers
The tutorial	Switching it on (in the emulator or on the board), the shell, files and disks, windows, the languages, a program of your own
rc	The shell underneath everything: commands, quoting, variables, pipelines, redirection, control flow, scripts
The tools	Every program in <code>/bin</code> : files, text, the editors, tasks, the debugger, the assembler, disks, and the rest
HyForth	The Forth, and the login shell: its words, libraries, the shell rule, files, devices, sound
hylang	The lisp (danlang on the Hydra): its REPL and scripts, the system library, the Hydra's built-ins, devices
BASIC	Microsoft's BASIC (EhyBASIC): programs and scripts, files, sound and <code>PLAY</code> , <code>SYS</code> and the system's calls, memory, the shell

For writing programs: [../programming/README.md](#), the programmer's guide, and the SDKs' own guides, [../sdk/asm/README.md](#) (assembly) and [../sdk/c/README.md](#) (C). The system calls' reference is `/rom/doc/api.md` on the Hydra (made from `spec/api.def` at the build: `obj/gen/api.md`).

The system in a paragraph

The Hydra-16 runs 16 tasks, each with its own 32K of RAM and its own RAM banks. The kernel is in the BIOS ROM; everything else, the drivers and the programs, are modules in the paged ROM that run in place. Almost everything is a file, Plan 9's way: a device is a directory of files (the console's at `/dev`, the disks' at `/dev/sd`, the tasks' at `/proc`), controlled by writing commands to its `ctl`. Each task has a namespace, the tree of names it sees, built by binds and mounts, so `/bin` is a union of the RAM disk's, the cards', the ROM disk's and the ROM's programs. The console is windows, rio's way on a serial terminal: each window is a whole console with its own shell, shown one at a time (Ctrl-] and a digit). The login shell is HyForth over rc (`forth -1`): a line is Forth if its first word is a Forth word or a number, else an rc command line.

rc

rc is the Hydra's shell underneath everything: Plan 9's rc, small, in assembly, run in place from the paged ROM. HyForth's and hylang's shells give it every line that isn't theirs (rc -c), C's system() runs it, and scripts start with #!/bin/rc. It's also a shell of its own: type rc at the HyForth prompt for an rc session (exit or Ctrl-D ends it), or name /bin/rc -l in a card's /lib/shell to have it in every window.

This guide is a tour with examples (each run on the Hydra). Plan 9's rc paper ("Rc — The Plan 9 Shell", Tom Duff) is the full story; what's here is what the Hydra's has.

Contents: [Starting it](#) · [Commands](#) · [Words and quoting](#) · [Variables](#) · [Redirection](#) · [Pipelines and background](#) · [Control flow](#) · [Functions](#) · [Scripts](#) · [Built-ins](#) · [At HyForth's prompt](#)

Starting it

Command	What runs
rc	An rc session: a prompt (%), each line run, till exit or Ctrl-D
rc -l	The same as a login shell: its namespace made from /rom/lib/namespace (and a card's), then /rom/lib/profile
rc -c 'line'	The line run, and rc ends with its status
rc file args	The file's commands (\$0 the file, \$* the args)

At the prompt, the console edits the line for it: Backspace, Delete, Left and Right, Home and End (Ctrl-A, Ctrl-E), Ctrl-U, and Up and Down through the last 8 lines. Ctrl-C ends what's running (its note goes to the window's note group), and rc goes on. A command that goes on (an open {) gets the second prompt, a tab.

Commands

A command is a program and its arguments, run in a task of its own and waited for. /bin is searched for a bare name (and .: \$path is (. /bin)); a name with a / is used as it is. A command's status is \$status: its exit message, or its code; empty or 0 is success.

Form	Meaning
a; b	a, then b (a new line is the same)
a & b	a in the background (\$apid its task), then b
a && b, a b	b only if a succeeds, or only if it fails
! a	a, its status turned over
{ a; b }	a group: one command
# words	a comment, to the line's end

```
% ls /nosuch || echo it failed
ls: /nosuch: not found
it failed
% ! ~ a b && echo not the same
not the same
```

Words and quoting

Words are separated by spaces and tabs. '...' quotes: nothing inside is special, and ' ' is a quote in it. Globbing makes names from * (anything), ? (one character) and [a-z] (one of a set; [~a-z] one not in it), sorted, by reading the directories; a pattern that matches nothing stays as it is.

^ joins words: two single words make one; a word and a list make a list, the word joined to each; two lists of the same length join item by item. A caret is put in for you between a word and a \$ right against it (pre\$x).

```
% x=(apple banana cherry)
% echo $x^.txt
apple.txt banana.txt cherry.txt
% echo pre^$x
preapple prebanana precherry
% echo 'single quotes: $x stays'
single quotes: $x stays
```

Variables

Every variable is a list of words: `x=word`, `x=(a b c)`, `x=()` (empty: as good as unset).

Form	Value
<code>\$x</code>	its words
<code> \$#x</code>	how many
<code>\$x(n)</code> , <code>\$x(n-m)</code>	the <i>n</i> th word (from 1), or those from <i>n</i> to <i>m</i>
<code>\$"x</code>	its words joined by spaces, one word
<code>`{command}</code>	the command's output, split into words at spaces, tabs and new lines

```
% x=(apple banana cherry)
% echo $x(2) $ #x
banana 3
% n=`echo hello | wc -c`
% echo $n
6
```

Variables are the environment's: a program `rc` starts gets them (`/env/x` in its task, `getenv` in C, `$x` in `hylang`), a list's words each with a zero byte after it. `x=1 cmd` sets `x` for that command alone.

Some are `rc`'s own: `$status`, `$apid` (the last background task), `$task` (`rc`'s task), `$path`, `$prompt` (its first word the prompt, its second the second prompt), `$*` and `$0` (a script's), `$window` (the shell's window, a login shell's).

Redirection

Form	Meaning
<code>< file</code>	fd 0 from the file
<code>> file, >> file</code>	fd 1 to the file, made or emptied; or added to
<code>>[2] file</code>	fd 2 to the file (any fd: <code>>[3]</code> , <code><[4]</code>)
<code>>[2=1]</code>	fd 2 made a copy of fd 1
<code>>[2=]</code>	fd 2 closed

```
% ls /nosuch >[2=1] | wc -l
1
```

Pipelines and background

`a | b` runs `a` and `b` at once, `a`'s fd 1 into `b`'s fd 0 (a pipe: `/dev/pipe`, 512 bytes held); `a | [2] b` is `a`'s fd 2. Each stage is a program; a stage that isn't one (a group, a function) runs as `rc -c` on its text. The pipeline's status is its last stage's.

`a &` runs `a` without waiting; `$apid` is its task, `wait` waits for the background tasks (`wait $apid` for one), and `ps` shows them.

```
% ls /rom/bin | sort -r | head -3
sort
scom
mkfs
```

Control flow

Form	Meaning
<code>if(cmd) a</code>	<code>a</code> if <code>cmd</code> succeeds
<code>if not b</code>	right after an <code>if</code> : <code>b</code> if it didn't run its command
<code>for(x in words) a</code>	<code>a</code> for each word, <code>\$x</code> it (<code>for(x) a: \$*'s</code>)
<code>while(cmd) a</code>	<code>a</code> while <code>cmd</code> succeeds
<code>switch(word){ case pattern ... }</code>	the commands after the first case whose pattern matches
<code>~ subject pattern ...</code>	success if the subject matches one of the patterns (glob patterns, matched as words, not names)

```
% for(f in $x) echo fruit: $f
fruit: apple
fruit: banana
fruit: cherry
% if(~ $x(1) apple) echo first is apple
first is apple
% switch($x(3)){
    case b*
        echo starts with b
    case c*
        echo starts with c
    }
starts with c
```

Functions

`fn name { commands }` makes one; its arguments are `$*` and `$1 ...`; `fn name` alone removes it. Functions are in the environment too (`fn#name`), so a child `rc` has them. `what is name` says what a name is: a function's text, a variable's value, or a program's path.

```
% fn greet { echo hello, $1 }
% greet world
hello, world
% what is greet
fn greet { echo hello, $1 }
```

Scripts

A file of rc commands runs with `rc file args`, or by its own name if its first line is `#!/bin/rc` (rc runs a file that isn't a program by the interpreter its `#!` line names: Plan 9's way). `/rom/bin/scom` is one:

```
#!/bin/rc
# scom [-l | n]: the old riff ... a song: once; its loop n more times; or -l, till it's stopped
if(~ $1 -l) play -l /rom/songs/scom.zsm
if not play /rom/songs/scom.zsm $*
```

`.` `file` runs a file's commands in this rc (its variables and functions stay); `exit [status]` ends a script, or rc. `/rom/lib/profile` is what a login shell runs first; a card's `/lib/profile` comes after it.

Ctrl-C reaches a script's rc (and `rc -c`'s) as well as what it's running: rc waits for that to end, as at the prompt (a program may take the note itself, finish up and say so), and then ends, with its status, rather than going on to the next command.

Built-ins

These change rc's own task, so they can't be programs:

Built-in	What it does
<code>cd [dir]</code>	the current directory (none: <code>\$home</code> , else <code>/</code>)
<code>bind [-a -b] [-c] new old</code>	new at old in rc's namespace: in place of what's there, or <code>-a</code> after it / <code>-b</code> before it in a union; <code>-c</code> creates go there
<code>mount [-a -b] [-c] dev old [spec]</code>	a device's tree at old (mount <code>'#f'</code> <code>/mnt/c 1</code> : card 1's file system)
<code>unmount [new] old</code>	old's member new gone, or all of old
<code>newns</code>	the default namespace made again
<code>exit [status], wait [task], eval words, . file, builtin cmd, whatis name, shift [n]</code>	as Plan 9's

`#` starts a comment, so a device's name is quoted: `ls '#c'`, `bind '#s' /dev/seg`.

At HyForth's prompt

The login shell is HyForth (`forth -l`). A line whose first word isn't a Forth word or a number is an rc line, run whole by an rc of its own (`rc -c`) and waited for. Pipes, redirection, globbing and quoting are rc's there, but each line is a new rc: a variable or function set on one line is gone on the next (the environment's `$status` is kept for the next line). For an rc session, type `rc;%` before a line makes it rc's whatever its first word (`% free` past a Forth word `free`). `cd`, `bind`, `mount`, `unmount` and `newns` are Forth words there, taking rc's arguments, so they change the shell's own directory and namespace. [hyforth.md](#) has the rest.

The tools

The programs in `/bin`. Most are modules in the paged ROM, which run in place (`/dev/mod/NAME`, bound into `/bin` by `#m/bin`); a few are files on the ROM disk, read into RAM to run (`/rom/bin: mkfs, fsck, label, grep, sort, db, edit, scom`). `/bin` is a union, so a program of your own in `/ram/bin`, `/sram/bin` or a card's `/bin` comes before the ROM's of the same name.

They behave as Plan 9's do:

- Flags come first (`ls -l, -abc` together), then names. A tool that reads files reads fd 0 when it's given none, so each works in a pipeline (`cat f | wc`).
- A name that fails is said on fd 2 as `tool: name: why (ls: /none: not found)`, the rest go on, and the tool ends with code 1. A bad flag or too few names is `usage: ...`, and the status `usage`.
- `$status` after a tool is its exit message, or its code; empty or 0 is success.

Every tool here runs the same way at rc's prompt and at HyForth's (whose shell gives a line whose first word isn't a Forth word to rc). At HyForth's prompt, a tool whose name is also a Forth word needs a % before it (% `free` when `memory.fl's free` is loaded).

Contents: [Files](#) · [Text](#) · [The screen editor](#) · [Tasks](#) · [The debugger](#) · [The assembler](#) · [The system](#) · [Disks](#) · [Others](#)

Files

Tool	What it does
<code>ls [-ld] [name ...]</code>	A directory's entries, a line each (a / after a directory's); a file's own name. <code>-l</code> : the mode, the device letter and instance, the length, the time, the name: <code>--rw-rw-rw- fr 13 2000-01-01 00:00 a</code> . <code>-d</code> : a directory itself, not its entries. None: the current directory
<code>cat [file ...]</code>	Each file to fd 1 (none: fd 0)
<code>cp [-r] from to, cp [-r] from ... dir</code>	Each file copied to a file made (one there already is emptied first), or into a directory by its own name. <code>-r</code> : a directory's whole tree. A file isn't copied onto itself
<code>mv from to, mv from ... dir</code>	Moved, or renamed: in its own directory a rename (a file there by the new name goes first); to another, a file is copied then removed, and a directory can't be moved
<code>rm [-rf] name ...</code>	Each file removed, or empty directory. <code>-r</code> : a directory's tree too, the deepest first; <code>-f</code> : quiet about what can't be
<code>mkdir [-p] dir ...</code>	Each directory made. <code>-p</code> : the directories it's in too, as needed, and no complaint about one there
<code>rmdir dir ...</code>	Each empty directory removed
<code>touch name ...</code>	Each file made empty if it isn't there, or its time changed if it is
<code>du [-a] [name ...]</code>	The kilobytes each tree takes (its files' lengths rounded up to whole KB), a line a directory, the deepest first; <code>-a</code> : a line a file too
<code>pwd</code>	The current directory
<code>cmp file1 file2</code>	Compared byte by byte: the first that differs said (<code>file1 file2 differ: byte 5</code>), or nothing, and the status

```
> mkdir /ram/notes
> echo first >/ram/notes/a; echo second >>/ram/notes/a
> cp /ram/notes/a /ram/notes/b; ls -l /ram/notes
--rw-rw-rw- fr      13 2000-01-01 00:00 a
--rw-rw-rw- fr      13 2000-01-01 00:00 b
> rm -r /ram/notes
```

Text

Tool	What it does
echo [-n] [words]	The words, a space between each, and a new line (-n: none)
wc [-lwc] [file ...]	Lines, words and bytes (-l, -w, -c: only those), a line a file, and a total for several
head [-N] [file ...]	Each file's first N lines (10)
tail [-N] [file]	The last N lines (10); the file's last 16K are what it looks at
grep [-chilnsv] [-e] pattern [file ...]	The lines that match a regular expression, Plan 9's (., [a-z], [^a], ^, \$, *, +, ?, , ()): -c the count, -l the files' names, -n with line numbers, -i case ignored, -v the lines that don't, -s no messages, -h no file names; status no matches when none did
sort [-bfnru] [file ...]	The lines in order: by bytes; -n by the number each starts with; -f case folded; -b leading blanks ignored; -r reversed; -u one of each run of equal lines. Stable; some 20K of lines at most
uniq [-c] [file]	Each run of the same line once; -c with its count
tee [-a] [file ...]	Fd 0 to fd 1 and to each file (-a: added to its end)
xd [file ...]	Bytes in hex, 16 a line after their offset, then as text: 0000010 68 65 6c 6c 6f 0a hello.
more [file ...]	A screen at a time (22 lines, then --more--): Enter for the next, q to stop. Its keys come from the console even when fd 0 is the file
ed [file]	The line editor, ed's way: p, a, i, c, d, w [name], q (twice if the text's changed), Q, h, with line numbers (1, 5p, \$). Lines are typed after a, i or c until a line of just .; Ctrl-C comes back to its * prompt

```

/> ls /rom/bin | sort -r | head -3
sort
scom
mkfs

/> grep -n Hydra /rom/README
1:The Hydra-16's ROM disk
5:machine. It's a HydraFS volume, as a card's is, in the paged ROM's banks after

```

The screen editor

edit [file ...] is the screen editor, nano's way: what you type goes in at the cursor, and the Ctrl keys and the Meta keys (Esc then a key, or Alt with it: M-) are commands. The two lines at the bottom name the commonest, and ^G shows them all. Up to 6 files are open at once, each in a buffer of its own; their text is in your task's RAM banks, so a file can be as big as they are (a few hundred K: kdev.s's 140K reads in some 6 seconds). A file whose lines end CR LF is written with CR LF again.

Keys	What they do
^O, ^S	Save (^O asks for the name: Enter keeps it)
^X	Close the file (if it's changed, saved or not: y, n); with the last closed, edit ends
^R; M-, M-.	Open another file; go to the file before, or after
^W; M-W, M-Q	Find (letters either case; round from the other end if need be); find again, find backwards
M-R	Replace: each match shown, y yes, n no, a all the rest
M-G	Go to a line (\$: the last)
^K, M-G, ^U	Cut the line (^K again: the next joins it in the cut buffer), copy it, paste (into any file); with the mark set (M-A), the block from the mark to the cursor
M-U, M-E	Undo, redo (a line typed, a cut, a paste: one step each)
the arrows, Home, End, PgUp, PgDn	Move (and ^B ^F ^P ^N, ^A ^E, ^Y ^V); M-\ and M-/ the text's start and end
Backspace; Del, ^D	Rub out; delete
M-I, M-X, ^L	Auto-indent on and off; the help lines off and on; the screen drawn again

Ctrl-C does nothing in `edit` (it's the console's interrupt), and `^\\` and `^]` never reach a program, so nano's keys there are Meta keys here. The screen is the terminal's size (`$COLUMNS` and `$LINES`, else 80 by 24). `ed` is the line editor (Text, above), for scripts and a terminal without a screen.

Tasks

Tool	What it does
<code>ps [-a]</code>	The tasks in use: number, state, parent, CPU time (seconds, to a tenth), note group, name; <code>-a</code> : their arguments too. A stopped task's state is <code>stopped</code>
<code>top</code>	The same each second, with the CPU each took in it, the screen drawn again; Ctrl-C ends it
<code>kill [-i] task ...</code>	Each task (its number) ended with the kill note, which nothing catches; <code>-i</code> : interrupted instead (Ctrl-C's note), which a program may catch
<code>slay [-i] name ...</code>	Each task running a program of that name, as <code>kill</code>
<code>sleep seconds</code>	Nothing for that long; Ctrl-C ends it
<code>ns [task]</code>	A task's namespace (none: this one's) as the binds and mounts that make it

The tasks' own files are under `/proc/N`: `status`, `args`, `cwd`, `env`, `ns`, `fd` (its open files: `0 rw #c 291 #c/cons`), `regs`, `mem` and `ram` (its memory), `note` (write `interrupt`, `kill`, `hangup` or a number to send it one), and `ctl` (`kill`, `interrupt`, `note N`, `stop` and `start`: a stopped task doesn't run till it's started again; `step`, `next`, `break` and `nobreak`: the debugger's, below). Another task's memory and registers are anyone's but the kernel task's and a driver's.

```

/> sleep 100 &
/> ps
task state parent cpu group name
...
5 sleep 4 0.0 4 sleep
...
/> echo stop >/proc/5/ctl; cat /proc/5/status
sleep stopped 4 0 4
/> echo kill >/proc/5/ctl

```

The debugger

`db program [arg ...]` starts a program stopped at its first instruction, in a task and note group of its own (so Ctrl-C reaches `db`, which stops it); `db -p task` stops a task that's running, where it is. Then it takes commands, a line each, at its `db>` prompt. An address is hex (`0830`, `$0830`) or a symbol, with `+` or `-` a hex offset (`main+14`); a count is decimal.

Command	What it does
<code>r</code>	Its registers, and the instruction at its PC
<code>s [n]</code>	<code>n</code> instructions (1), a step at a time, into a subroutine at a JSR
<code>n [n]</code>	The same, but a JSR's subroutine runs whole (a JSR into the kernel's jump table always does)
<code>c</code>	On, till a breakpoint, a BRK, its end, or Ctrl-C
<code>u addr</code>	Steps (as <code>n</code>) till its PC is <code>addr</code>
<code>b [addr]</code>	A breakpoint at <code>addr</code> (in RAM: a BRK written there while it runs), or the list
<code>x [addr]</code>	The breakpoint at <code>addr</code> gone, or all of them
<code>d [addr] [n]</code>	<code>n</code> instructions (12) from <code>addr</code> (its PC; then on from the last), disassembled
<code>m [addr] [n]</code>	<code>n</code> bytes (64) from <code>addr</code> , in hex and as text
<code>w addr byte ...</code>	Bytes (hex) written at <code>addr</code>

Command	What it does
l file	Symbols from an ld65 label file (ld65 -Ln: the build makes one for each program, obj/.../NAME.lbl)
q	Quit: a program db started is killed; a task it stopped runs on

```

/> db /rom/sample/hi Ann Bob
task 4
PC=0830 A=30 X=FF Y=48 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830 A5 02 LDA $02
db> l /pc/hi.lbl
8 symbols
db> b main+14
db> c
breakpoint 1
PC=0844 A=41 X=FF Y=48 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0844 A9 52 LDA #$52 main+14
db> n 4
0846 85 02 STA $02 main+16
0848 A9 08 LDA #$08 main+18
084A 85 03 STA $03 main+1A
PC=084C A=08 X=FF Y=48 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
084C 20 53 F9 JSR $F953 main+1C
db> x
db> c
Hello, Ann!
Hello, Bob!
I'm task 4, in /, in window 0.
task 4 ended: code 0

```

A step can't be taken in the kernel: a task stopped in a call ((in a call: its state 6), as Ctrl-C finds a program in SLEEP) is run on with c. Breakpoints are in its memory only while it runs, so d and m show its own bytes; one in a ROM can't be set (u reaches an address there, a step at a time). A BRK of the program's own stops it too; c from there gives it its note (sys: brk), as it would have had. A subroutine that reads the bytes after its JSR (its arguments) can't be stepped over with n; step into it with s. db works through /proc/N/ctl and mem: echo step >/proc/N/ctl does the same by hand, on a task that's stopped.

The assembler

as [-bl] file.s [out] assembles a program for the 65C02 on the Hydra itself, from the same source ca65 takes on a PC: a RAM program, out (file without its .s, if there's no out), that runs as any program does. -l writes its labels too, out.lbl (ld65's -Ln form, which db's l reads); -b writes the bytes alone, no header, from .org's address (\$0800 if there's none): a ROM's image, say.

The SDK's files are in /lib/as: hydra.inc, hyx2.inc, macros.inc, toollib.inc and toollib.s, srvlib.inc and srvlib.s, nslib.s, and three of its samples, hi.s, tick.s and upper.s. .include and .incbin find a file as it's named, then beside the file that names it, then in /lib/as. A program made from the same source by ca65 and ld65 (sdk/asm/hyx2.cfg) is the same, byte for byte.

What it takes is ca65's, as the SDK's sources use it:

Kind	What as takes
Instructions	The W65C02S's, each mode written as ca65 writes it; a: or z: before an address makes it absolute or zero page. An address is on the zero page if it's known by then and under \$100 (a label further on is absolute)

Kind	What as takes
Labels	name:: cheap locals, @name:, between one normal label and the next; unnamed ones, :, reached as :+ :++ ... and :- :-- ...; constants, name = expr and name := expr
Expressions	32 bits, with ca65's operators and their order (* / .mod & ^ << >>, + - , the comparisons, && .and .or .xor .not, unary - ~ < > ^ !), * (here), \$hex, %binary, 'c', and .lobyte .hibyte .bankbyte .loword .hiword .strlen .defined .blank .match
Data	.byte (and strings), .word, .addr, .dword, .res, .asciiz, .incbin "file" [, start [, count]]
Files	.include
Conditions	.if, .ifdef, .ifndef, .ifblank, .ifnblank, .elseif, .else, .endif
Macros	.macro name paramsendmacro, .exitmacro; an argument in { } may have commas in it
Segments	.segment "NAME", .zeropage, .code, .rodata, .data, .bss, .pushseg, .popseg
Checks	.assert expr, error warning, "text", .error, .warning
Taken and left	.import, .export, .global (and their zp kinds), .setcpu, .feature, .macpack, .debuginfo, .list, .case ...

Not there: .proc and .scope, .repeat, .struct, .sprintf and .ident (but in a branch that's not taken), and objects to link: one source file and what it includes make one program. Its segments are laid out as hyx2.cfg lays them out: ZEROPAGE from \$22 (the program's \$5E bytes), then from \$0800 HEADER, CODE, RODATA, DATA and BSS, to \$8000 at most, with ld65's names for them (__DATA_LOAD__, __BSS_RUN__, __BSS_SIZE__, __RAM_LAST__ ...) and HYX2_RAM defined, as hyx2.inc needs.

An error is said as as: file:line: what, and as ends with status 1; a warning is said, and the program made. It reads its source three times (the segments' sizes, then each symbol's value, then the bytes), each file read from the disk once and kept in the task's RAM banks; a pass that finds errors is the last, so another pass's errors show once they're mended. hi.s, with hydra.inc (some 47K of source), takes about 4 seconds; upper.s, with toollib.s too (83K), about 9.

```

/> as -l /lib/as/hi.s /ram/hi
/> /ram/hi Ann
Hello, Ann!
I'm task 4, in /, in window 0.
/> db /ram/hi Ann
...
db> l /ram/hi.lbl
8 symbols

```

The system

Tool	What it does
date [-n]	The clock (2026-10-03 15:04:05), or -n seconds since 2000-01-01. Set it with echo 2026-10-03 15:04:05 >/dev/time, which sets the DS1747 too if there's one
free	RAM: each task's own banks (16 for each RAM module, 8K each), and the shared RAM: what segments have, what's free
mods	The paged ROM's modules: bank(s), type (program, driver, library; boot a driver started at boot), name
hwtest	The system starts again into the hardware test (as a T typed during POST does); the reset button ends it

The kernel's messages (the boot's, POST's, a driver's) are /dev/kmesg, its last 4K.

Disks

The disks are under /dev/sd, a directory each: 0-f the SD cards (by their SPI device), x the ROM disk, r the RAM disk (each shell's own area of it is its /ram), s the shared RAM disk (/sram). Each has data (the disk's bytes) and ct1, which reads as the disk and its file system:

```

/> cat /dev/sd/r/ctl
ram 256 KB 512 blocks
hydrads label=RAM
free 252 KB of 255 KB
banks $00-$1f

```

Tool	What it does
df	Each disk started: its kind, its file system's size and what's free, its label
mkfs [-fp] disk [label ...]	A new, empty HydraFS on the disk (0-f, r, s): -f a full format; -p in a partition of its own after a card's others
fsck [-f] disk	The disk's HydraFS checked (-f: and its faults fixed): what was lost, unmarked, or used twice
label disk [text ...]	The disk's label set, or said

Through the ctl files directly: start SIZE [FROM-TO] and stop on a RAM disk (echo start 128K 1-1 >>'#d/r/ctl': from RAM module 1's banks; >>, as a stopped disk's directory isn't listed), init on a card (after it's changed), format, label, check. The cards' file systems are at /sd/N; a card's bin and lib, if it has them, join /bin and /lib.

Others

Tool	What it does
play [-l] song [n]	A ZSM song (the X16's format; Furnace exports it) on the YM2151 (and with a Vera X its PSG and PCM parts): once, its loop n more times, or -l till Ctrl-C; or a score (a name ending in .mml: below); or a WAV file on the Vera X's PCM (8 or 16 bits, mono or stereo, to 48,828 Hz: from a card, 8 bits mono to about 11 kHz). /rom/songs has a few; scom plays one
play -o score.mml song.zsm	The score compiled into a ZSM file instead
play [-lx] -m ch mml ...	A line of MML (a score's channel's: below) on channel ch, the channel's own instrument if the line names none: play -m 0 t180 o4 l8 c d e f g
play [-lx] -c ch notes ...	A chord: each note on the next channel from ch, the commands before each with it (play -c 0 o4 l2 i0 c e g); -x, either in the Commander X16's MML (play -x -m 0 T120 O4 L8 CDEFG)
xmodem -r file, xmodem -s [-k] file	A file received or sent with XMODEM over the serial line, with any terminal program on the PC: -r receives, -s sends (-k: 1K blocks)
forth, hylang, basic, rc	The languages and the shell: hyforth.md , hylang.md , basic.md , rc.md

The sound device is /dev/snd (register and value pairs), /dev/sndctl (claim N [P], release N [P], volume N, reset; and a channel's commands as text: patch CH P, note CH N, off CH, level CH V, pan CH left|right|both, bend CH B, drum CH N, freq CH HZ, glide CH N, wave CH pulse|saw|triangle|noise [WIDTH], sens CH PMS AMS, reg R V; the YM2151's lfo RATE PMD AMD WAVE and noise N|off; so echo note 0 60 >/dev/sndctl plays middle C), /dev/bell and /dev/psg. Channels 0-7 are the YM2151's, 8-23 (with a Vera X) its PSG's voices: the same commands, and wave; claim's P is their mask, bit n channel 8 + n. The VIA's port A is /dev/gpio (pins 0-7, port, ctl, ca1) and /dev/i2c the I2C bus on two of its pins; /dev/spi the SPI devices; /dev/seg names shared segments; /dev/vid is the Vera X (its screen, VRAM, the PSG, and pcm and pcmctl, its PCM: [../programming/video.md](#)); /pc a folder on the PC (through the PC tool, [sim/tools/hydrapc.js](#), which is the terminal too).

Scores

A score is music as text, the language of the PC's score compiler ([sim/tools/hysong.js](#), whose header has it whole), and play plays one as it is, compiling it as it goes into the same register writes at the same ticks as hysong.js would put in a ZSM file (play -o writes that file):

```

; A score: ; to the line's end is a comment
#tempo 120                ; quarter notes a minute (120); #rate N: its ticks a second (200)
@piano { gm 0 }           ; an instrument: one of the driver's 163 patches, or a voice's operators
A @piano o4 l8 c d e f g4 r4 c2 ; channel 0 (A-H: 0-7); a channel's lines are joined in order
B @piano o3 l2 c [g e]2 c   ; [ ... ]N: repeated N times

```

Notes c to b (+ or # sharp, - flat), a length (1 a whole note ... 64; 3 6 12 24 48 triplets), dots, ^ ties; r a rest; x N a General MIDI drum; o > < the octave; l the default length; q the part of a note held (eighths); v the volume (0-127); p 1|r|c|0 the speakers; k transpose; D detune (64ths); M and L the LFO; N the noise; y reg, val a register; _ a slide to the next note, & legato; I N the driver's patch N as the instrument. A score is read whole into memory (some 24K at most); a mistake in it is said with its channel (play: x.mml: channel 2: no such drum). A line (-m) or a chord (-c) is the same language, and t N sets its tempo, before its first note. -x takes the X16's MML (FMPLAY's): upper-case notes, T the tempo, V 0-63 (doubled), P 1-3 (left, right, both), S 0-7 (the gap after a note; S0 legato), K (the next note keyed on), I a patch, O, L, R, <, > as above; each line starts afresh (T120 O4 L4), not where the last left off. The languages' words run it: C's snd_mml and snd_chord, HyForth's and hylang's snd-mm1 and snd-chord.

Using HyForth

HyForth is the Hydra-16's Forth: a Forth 2012 system whose core is in the paged ROM, with the rest of the standard's word sets as libraries it loads into its dictionary as they're wanted. It's also a shell: `forth -l` runs a line as Forth when its first word is a Forth word or a number, and as an rc command line otherwise. This guide is for using it; [../hyforth.md](#) is its design (and why each piece is as it is), and [../forth-status.md](#) where it stands. The first HyForth, the old system's, had a guide of its own ([docs/using/hyforth.md](#) at the repository's top); this one is the new system's.

Contents: [Starting it](#) · [The basics](#) · [Libraries](#) · [The shell](#) · [Files](#) · [The Hydra's words](#) · [The terminal and keys](#) · [Sound](#) · [Devices](#) · [Tools](#) · [Memory, locals and blocks](#) · [Errors and Ctrl-C](#) · [Memory](#)

Starting it

Typed at the shell's prompt	What runs
<code>forth</code>	Forth at the console: each line typed is run, then <code>ok</code>
<code>forth -l</code>	Forth as a login shell: its namespace made, <code>startup.fs</code> and <code>profile.fs</code> run, then its prompt (<code>/></code>)
<code>forth file.fs a b</code>	A script: the file run, then <code>forth</code> ends (with code 1 after an error). A file whose first line is <code>#!/bin/forth</code> runs by its name too (<code>./file.fs a b</code>)

`bye` ends a plain `forth`, and `exit` the shell (exits `( n -- )` with a code). HyForth is the login shell: the ROM's `/lib/shell` names `/bin/forth -l`, which `init` starts in window 0 and in each window made (`Ctrl-]` `c`). A line in `/lib/shell` on a card (`/sd/0/lib/shell`) or the shared RAM disk names another (`/bin/rc -l`, `/bin/hylang -l`): it comes first in `/lib`'s union.

A script's arguments are `argc ( -- n )` and `arg ( n -- c-addr u )` (in `hydra.fl`): `forth file.fs a b` has 3, the file's name `0` `arg`. At the console, and under `forth -l`, there are none.

The basics

Words are run left to right; numbers go on the data stack, and words take their arguments from it. Names are in lower case and found in either case (`DUP` is `dup`).

```
/> 2 3 + .
5
/> 1 2 3 .s
<3> 1 2 3
/> . . .
3 2 1
```

Numbers are 16 bits, read in base (decimal at the start). A prefix gives another: `$ff` hex, `#10` decimal, `%101` binary, `'a'` a character's code. `hex` and `decimal` set base; `.` prints signed, `u.` unsigned.

```
/> $ff . #10 . %101 . 'a' .
255 10 5 97
/> -1 u.
65535
```

Definitions are Forth 2012's: `:` `name ... ;`, `constant`, `variable`, `value` (and `to`), `create` and `allot`, `defer` and `is`, `marker`. The control structures (`if` `else` `then`, `begin` `until`, `begin` `while` `repeat`, `do` `loop`, `?do`, `case` of `endof` `endcase`) work in definitions.

```

/> : sq ( n -- n*n ) dup * ;
/> 7 sq .
49
/> : count-to ( n -- ) 0 ?do i . loop ;
/> 3 count-to
0 1 2
/> see sq
: sq dup * ;

```

Compile-only words. The words Forth 2012 leaves undefined outside a definition (the return stack's: >r, r>, i, unloop ...; the control structures'; ;, literal, postpone, ." ...) give an error there, -14, and forth goes on. .(text) is ."s form for the prompt.

```

/> 1 >r
>r: compile only

```

Libraries

The core is Forth 2012's Core word set. As it starts, forth loads /lib/forth/startup.fs, which loads Core Extension, Exception, File Access and Programming-Tools; forth -l adds the shell's. The rest are loaded when they're wanted: lib name at the prompt, or require name.fl in a program.

```

/> libs
forth coreext exception file tools shell
/> lib hydra
/> lib random
/> -lib random
/> libs
forth coreext exception file tools shell hydra (random)

```

libs lists the core, then each library; (random) is one -lib took out of the search (its code stays, so what was compiled with it still runs), and lib random puts it back where it was. lib name loads name.fl, else name.fs, from the current directory or /lib/forth. A marker made before a library takes it out with the rest.

Library	Words
coreext.fl	Core Extension: ?do, case, value, defer, :noname, pick, roll, .(, c", s\" ... (at the start)
exception.fl	catch, throw (at the start)
file.fl	File Access: open-file, read-file, read-line, write-file, include, require ... (at the start)
tools.fl	Programming-Tools: .s, ?, dump, see, words, [if], synonym ...; libs, lib, -lib (at the start)
shell.fl	The shell (forth -l's)
facility.fl	Facility: key?, ms, time&date, page, at-xy, structures; the terminal's words and keys
string.fl	String: compare, search, /string, -trailing, sliteral, substitute, replaces ...
search.fl	Search-Order: wordlist, set-order, also, only, previous, definitions ...; library name ... end-library, a source library's words in a word list of its own (hydra.fs's)
double.fl	Double-Number: d+, d-, d., d.r, d<, d=, dmax, m*/, 2constant, 2value, 2rot ...
memory.fl	Memory-Allocation : allocate, free, resize
locals.fl	Locals : {: ... :}, (local), locals
block.fl	Block : block, buffer, update, flush, load, list, thru ...
hydra.fl	The Hydra's words : the sys- words, sh, run, banks, directories, notes, argc, arg, sys, ctl
hydra.fs	The system's constants and error codes (O_RDWR, E_NOENT ...), in a word list of their own, hydra

Library	Words
disasm.fl	disasm; with it, see of an assembly word shows its instructions
bits.fl	tbit, sbit, cbit
random.fl	random (u -- u') (0 to u-1), rand, rand32, rseed
sound.fl	Sound: the YM2151's words, note-of, tune
gpio.fs i2c.fs spi.fs cons.fs proc.fs clock.fs disk.fs pc.fs	Devices, in Forth source

Your own libraries are .fs files: put them where lib looks, or on a card's /lib/forth (the cards' /lib is bound before the ROM's, so a card's startup.fs or profile.fs takes the place of the ROM's too).

The shell

Under forth -l, a line whose first word is a Forth word or a number is Forth; any other line goes to rc, whole (rc -c), so pipes, redirections, quotes and \$name are rc's. % at a line's start sends it to rc whatever its first word is.

```
/> ls /rom/lib/forth
bits.fl
clock.fs
...
/> echo hello | wc -c
6
/> foo
rc: foo: not found
```

Words of the shell. An rc line runs in a task of its own, which can't change forth's directory or namespace, so these are Forth words that read their arguments as rc does: cd [dir] (none: \$home), bind [-a|-b] [-c] new old, mount, unmount, newsn.

The prompt is a format: s" %v%d> " prompt (the start's). %v the card (0:), %d the directory on it, %p the whole path, %t the task, %w the window, %% a %. While a definition is being compiled over more than one line the prompt is the second one, prompt2's (a tab at the start).

```
/> s" %t:%p%% " prompt
4:/% cd /rom/lib
4:/rom/lib% s" %v%d> " prompt
/rom/lib>
```

Statuses. status (-- n) is the last rc line's code, and \$status rc's, the program's message. A line ending in & isn't waited for: its task is \$apid, and wait (task --) waits for it.

```
/> nosuchprogram
rc: nosuchprogram: not found
/> status .
1
/> echo $status
not found
```

The environment: getenv (c-addr1 u1 -- c-addr2 u2), setenv (c-addr1 u1 c-addr2 u2 --) (the name first), unsetenv (c-addr u --).

```

/> s" greeting" s" hello" setenv
/> echo $greeting
hello

```

Programs as values.

Word	Stack	Does
sh	(c-addr u -- n)	An rc command line run, waited for: its code
run	(c-addr u -- n)	A program run (its name and arguments, no rc), waited for
sh-out	(c-addr u -- c-addr2 u2)	An rc line's output, as a string (status its code)
output-of	(xt -- c-addr u)	What a word writes, as a string
	(xt "line" --)	A word's output the rest of the line's input: ' words wc -l
piped	(xt c-addr u --)	The same, the command line a string
spawn	(c-addr u -- task)	A program not waited for
send	"n line"	The line typed in window n, as if at its keyboard

```

/> s" echo hi there" sh-out type
hi there
/> ' words | wc -l
115

```

The strings sh-out and output-of give are in the dictionary's free space: use them before anything's added to it (move one somewhere to keep it).

Files

File Access's words work on the system's files: a fileid is an fd, and an ior is 0 or -512 less the system's error code, whose text ior>text gives (in hydra.fl).

```

/> s" /none" r/o open-file . .
-544 0
/> s" /none" r/o open-file nip ior>text type
not found
/> create buf 64 allot
/> s" /rom/README" r/o open-file throw value fd
/> buf 40 fd read-file throw buf swap type
The Hydra-16's ROM disk
=====
/> fd close-file throw

```

include file and require file load source; a name with no / that isn't in the current directory is /lib/forth's. In hydra.fl, Gforth's words for directories: open-dir (c-addr u -- dirid ior), read-dir (c-addr u1 dirid -- u2 flag ior) (a name at a time), close-dir, get-dir (c-addr u1 -- c-addr u2) (the current directory), set-dir (c-addr u -- ior), mkdir (c-addr u mode -- ior).ctl (c-addr1 u1 c-addr2 u2 --) writes the text c-addr2 u2 to the file c-addr1 u1 in one write, as a device's ctl wants (s" /dev/sndctl" s" volume 100" ctl).

The Hydra's words

lib hydra (hydra.fl) has a word for each system call, sys- and its name in lower case (sys-getpid, sys-sleep, sys-banks-alloc ...; /rom/doc/api.md lists them, with each one's stack), and the system's constants in Forth are require hydra.fs (O_RDWR, E_NOENT ..., in their own word list).

Words	Does
note (task n --), note-group (group n --)	A note sent to a task, or a note group (Plan 9's)
on-note (xt --)	The notes that come (but Ctrl-C and kill) given to xt (n -- flag): true, forth goes on; false, as Ctrl-C
pause	The other tasks' turn
bank! (bank --), bank@, bank-window (-- addr), seg-bank!	A RAM bank of the task's at \$8000 (sys-banks-alloc gives them); not in a definition compiled into a bank (THROW -21: see Memory)
code-banks (flag --)	False: the colon definitions from now on compiled into the dictionary; true (as forth starts): into the task's banks
sys (addr a x y -- a x y p)	Machine code called with its registers, its flags after
sh, run, ctl, argc, arg, ior>text, the directories'	As above

```

/> : h ( n -- flag ) ." note " . true ;
/> ' h on-note sys-getpid 16 note 7 .
note 16 7

```

The terminal and keys

The console is an ANSI terminal. lib facility has, beside page and at-xy (x y --): clear-line, clear-below, cursor-up (n --) (and -down, -right, -left), cursor-save, cursor-restore, cursor-off, cursor-on, form (-- rows cols); colours as C's conio numbers them, color (n --) and bgcolor (n --), with black ... white and bright; plain, bold, dim, underline, blink, reverse; beep. Keys: key and key?, and ekey, ekey?, ekey>char, ekey>fkey with k-up, k-down ... k-f12 for the special keys.

```

: hello page red color bold ." Hello" plain 0 2 at-xy ;

```

Sound

lib sound drives the YM2151 (#a), and with a Vera X its PSG (channels 8-23), with C's and hylang's names, the channel first: snd-reset, snd-volume (v --) (0-200), snd-claim (mask --) (bit n channel n), snd-release, snd-claim-psg (mask --) and snd-release-psg (the PSG's: bit n channel 8 + n), snd-wave (ch w width --) (a PSG channel's waveform, 0 pulse, 1 saw, 2 triangle, 3 noise, and its width, 0-63: 63 a square), snd-patch (ch n --), snd-note (ch note --) (a MIDI number: 60 middle C), snd-off (ch --), snd-level (ch v --) (a channel's volume, 0-127; snd-vol its old name), snd-pan (ch pan --) (1 left, 2 right, 3 both), snd-bend (ch n --), snd-drum (ch n --) (General MIDI's drum n), snd-freq (ch hz --) (a note at a frequency), snd-glide (ch n --) (to a note without a new attack), snd-lfo (rate pmd amd wave --), snd-sens (ch pms ams --) (vibrato 0-7, tremolo 0-3), snd-noise (n --) (channel 7's, 0-31; negative: off), snd-reg (reg value --), snd-regs (c-addr --) (the chip's 256 registers as written). note-of (c-addr u -- n) turns a note's name into its number, and tune (c-addr u ch tempo --) plays a string of notes and their beats (- a rest) at a tempo, beats a minute; Ctrl-C ends it. Songs (ZSM files) are the play program's: snd-play (c-addr u times -- status) runs it and waits (0 times: its loop till Ctrl-C); and lines of MML, its too (play -m, -c: [tools.md](#), "Scores"): snd-mml (ch c-addr u -- status), snd-chord (ch c-addr u -- status).

```

/> s" C#4" note-of .
61
/> s" C4 1 E4 1 - 1 G4 2" 0 240 tune

```

Devices

The device libraries are Forth source over the devices' files, with hylang's names; each uses only the words forth starts with, so each is also an example of driving its device by hand. A failure THROWS the system's error; a text one gives is in its buffer till its next.

Library	Words
gpio	gpio (pin -- level),gpio! (pin level --),gpio-in (pin --),gpio-out,gpio-port (-- byte),gpio-port!,gpio-ddr! (byte --),gpio-ca1! (rise? --),gpio-ca2! (n --),gpio-wait (-- count) (CA1's next edge),gpio-state (-- c-addr u)
i2c	i2c-read,i2c-write (addr reg c-addr u --),i2c-speed (khz --),i2c-reg-size (n --),i2c-devices,i2c? (addr -- flag)
spi	spi (dev c-addr u --) (a transaction: what came back in the bytes' place),spi-read (dev c-addr u --),spi-mode (dev mode --)
cons	window (-- n),windows (-- c-addr u),new-window,show-window (n --)
proc	task-args,task-cwd,task-env,task-ns,task-regs (task -- c-addr u),task-mem (task addr c-addr u --),task-ram (task bank offset c-addr u --)
clock	set-date (c-addr u --) (s" 2026-10-04 12:00:00" set-date),rtc (-- c-addr u) (running, stopped, none)
disk	disk-ctl (disk -- c-addr u) (a disk by its letter: [char] x the ROM disk, [char] 0 card 0),disk-start,disk-stop (disk --),cards (-- mask)
pc	pc? (-- flag): the PC tool answers (/pc's files are files for everything else)

```
> lib gpio
> 2 gpio .
1
> 4 1 gpio!
> lib i2c
> i2c-devices
50
> 1 i2c-reg-size $50 0 s" hello" i2c-write $50 0 pad 5 i2c-read pad 5 type
hello
> lib proc
> sys-getpid task-cwd type
/rom/lib
```

(Pin 2 high; a memory chip at I2C address \$50, its registers a byte's address.)

Tools

words lists the first word list in the search order, newest first: each word's code address in hex, three flags, and its name. l a literal (a constant), i immediate, a or f assembly (the core's or a library's) or Forth (yours).

```
> 5 constant five
> words
41EC l-f five          41D5 --f h          409F --a sys-rtc
...
A271 --a swap          A268 --a drop          A256 --a dup
```

see name shows a definition; with lib disasm, an assembly word's instructions, and disasm (addr n --) any code's. dump (addr u --) shows memory, .s the stack, ? (addr --) a cell.

```

/> lib disasm
/> see 2drop
code 2drop
  A2E3 E8      inx
  A2E4 E8      inx
  A2E5 60      rts
end-code

```

Memory, locals and blocks

lib memory: `allocate ( u -- a-addr ior ), free ( a-addr -- ior ), resize ( a-addr u -- a-addr2 ior )`. The heap is the top of the dictionary's space, so what's allocated is space the dictionary hasn't; freeing gives it back. At the shell's prompt, Forth's `free` then shadows the `free` program: `% free` runs the program.

lib locals: a definition's named values, Forth 2012's way. Between `{ :` and `;} :` arguments, taken from the stack (the last is its top), then after `|` values that start undefined, then after `--` a comment. A local's name pushes its value, `to name` sets it; it hides a word or number of that name till the definition's end.

```

/> lib locals
/> : lt7 { : a b : } b a ; 7 8 lt7 . .
7 8
/> : lt12 { : a | b c : } 20 to b a 21 to a 22 to c a c b ; 19 lt12 .s
<4> 19 21 22 20

```

lib block: Forth's blocks, each 1024 bytes of a file, `blocks.fb` in the current directory (made when it's first wanted) or the one `s" name" open-blocks` names. `n block` gives a block's buffer (read in), `update` marks it changed, `flush` writes the changed ones; `n load` interprets a block (`\` skips to its 64-character line's end), `a b thru` blocks `a` to `b`, `n list` shows one. The blocks read and written are kept in the task's RAM banks too (128 of them), so a block wanted again comes from there, not the file; `flush`, and `forth's end`, write the changed ones to it.

```

/ram> lib block
/ram> 1 block 1024 bl fill s" 2 3 + . blk @ . \ the rest 7 ." 1 block swap move update flush
/ram> 1 load
5 1

```

Errors and Ctrl-C

An error says what it was (with the file and line, in a file being included) and empties both stacks; the prompt comes back. A word that isn't there is `name ?` in a plain forth (under forth `-l` the line goes to `rc`, which says it isn't a program); the standard's errors have their text (`stack underflow`, `division by zero`, `compile only ...`), and the system's are its own (`not found`). `catch` and `throw` are Forth 2012's.

Ctrl-C stops the word running (`THROW -28`, `interrupt`), whatever it's doing: a loop, a wait for a key, a program `sh` or `run` started (which it ends too).

Memory

The dictionary is forth's RAM after its own variables, up to `$7F00`: about 18K free at forth `-l`'s prompt (unused). A colon definition takes only its header and 6 bytes of it: its code goes to one of the task's RAM banks, which forth takes as they're wanted, so 100 definitions of a couple of hundred bytes take about 1.2K of the dictionary, not 22K. A call to a word in another bank costs about 90 cycles more (in the same bank, nothing). Libraries load into the dictionary; `-lib` takes a library out of the search but keeps its code, so it doesn't free anything, while a `marker` does (`marker -work ... -work` takes back everything made

since). `allocate`'s heap (`lib memory`) takes its space from the top, so `unused` counts what's below it. PAD and the other buffers are forth's own. The task's RAM banks (16 of 8K for each memory module) are at \$8000 through `bank!`: take them with `sys-banks-alloc` so your program and libraries don't use the same ones. forth keeps the index of its words (by which it finds a name without reading every one) in the task's last bank, which it takes as it starts, so a program that writes to a bank it didn't take can spoil forth's search for words, or your definitions' code. A definition whose code is in a bank can't select another with `bank!` or `seg-bank!` (its code would be gone from under it): it THROWS -21. Compile such a word with code banks off, and it goes in the dictionary as before; it can be called from any definition, if it selects the bank it found again before it ends:

```
false code-banks
: fill-bank ( bank -- ) bank@ swap bank! bank-window 8192 0 fill bank! ;
true code-banks
```

Using hylang

hylang is danlang on the Hydra-16: a lisp with big integers, exact fractions, hashes and closures, and the Hydra's files, tasks, memory and devices a function away. The language is danlang's, exactly: the specification is danlang's `reference.md` (`C:\source\danlang`, <https://github.com/SNSTRUTHERS/danlang>), and danlang's own regression suite runs on the Hydra unchanged. This guide is for using it; `../hylang.md` is its design (where it differs from danlang, the Hydra's layers in full, and how it's built).

Contents: [Starting it](#) · [The basics](#) · [Numbers](#) · [Lists, strings and hashes](#) · [Control](#) · [Files and programs](#) · [The Hydra](#) · [Devices](#) · [Every system call](#) · [Errors and Ctrl-C](#)

Starting it

Typed at the shell's prompt	What runs
hylang	The REPL: each expression typed is evaluated, its value shown after <code>=></code> ; <code>exit</code> ends it
hylang file.hl a b	A script: the file run, then hylang ends (args is {"file.hl" "a" "b"}); a file whose first line is <code>#!/bin/hylang</code> runs by its name too
hylang -l	hylang as a login shell, over rc: a line that starts with <code>(</code> , <code>{</code> or <code>[</code> (or a character right against one: <code>?()</code>) is hylang's, any other an rc command line. Name <code>/bin/hylang -l</code> in a card's <code>/lib/shell</code> to have it in every window

```
> hylang
hylang (danlang on the Hydra-16)
Type 'exit' to Exit

hylang> (+ 1 2)
=> 3
```

At the REPL the console edits the line (Up and Down for the last ones), and an expression that isn't closed asks for another line. (`exit n`) ends a script with a status.

The basics

An S-expression `(f a b)` calls `f` with the values of `a` and `b`; a Q-expression `{a b}` is a list, as it's written (data, or code to run later); `[a b]` is a list of the values. Names are case-insensitive.

```
hylang> (def {x} 5)
=> NIL
hylang> (* x x)
=> 25
hylang> (fun {sq n} {* n n})
=> NIL
hylang> (sq 12)
=> 144
hylang> (map (fn {n} {* n n}) {1 2 3})
=> {1 4 9}
```

`def` binds globally, `set` in the current scope, `set!` changes the nearest binding; `fn` makes a function (a closure); `fun` defines one by name. A function given fewer arguments than its formals is partially applied: with `(fun {add a b} {+ a b})`, `(map (add 10) {1 2 3})` is `{11 12 13}`. Seven prefixes are shorthand for a call: `?(c a b)` is `(if c a b)`, and `=` `set`, `:` `def`, `#` `hash-create`, `@` `fn`, `.` `unpack`, `~` `format`.

Numbers

Integers have no fixed size (on the Hydra, up to 255 bytes: some 614 digits), division is exact, and a decimal written with a point is a fixed decimal, exact too:

```
hylang> (pow 2 100)
=> 1267650600228229401496703205376
hylang> (/ 1 3)
=> 1/3
hylang> (+ 1/3 1/6)
=> 1/2
hylang> (+ 0.1 0.2)
=> 0.3
```

Bases are read with # (#xFF, #b101, #16rFF) and written with (to-str n "x"); hex, bin, the bit functions (bit-and, shl ...), lo, hi and word are for bytes and words. Complex numbers, random, truncate, to-fixed, div and mod are there too.

Lists, strings and hashes

```
hylang> (filter even? (range 1 10))
=> {2 4 6 8}
hylang> (len "hydra")
=> 5
hylang> (def {h} #({:name "hydra"} {:tasks 16}))
=> NIL
hylang> (h :name)
=> "hydra"
hylang> (format "{} has {} tasks" (h :name) (h :tasks))
=> "hydra has 16 tasks"
```

Lists: head, tail, join, nth, take, drop, reverse, sort, map, filter, foldl, find, zip ... Strings are bytes: substring, index-of, str-split, str-join, str-replace, str-upper ...; + of a string joins what follows as print shows it ((+ "n=" 5)). A hash is called with a key to look it up, and a function in it is a method ((obj :add 3), &0 the hash). Buffers are bytes changed in place ((buffer 16)).

Control

if, cond, case, and, or, while, each, dotimes, do, let:

```

hylang> (if (> 5 3) "bigger" "smaller")
=> "bigger"
hylang> (each {f (ls "/rom/lib")} (print f))
forth
hylang
namespace
profile
shell
=> NIL
hylang> (dotimes {i 3} (print "line" i))
line 0
line 1
line 2
=> NIL
hylang> (fun {fact n} {if (zero? n) 1 (* n (fact (- n 1)))})
=> NIL
hylang> (fact 30)
=> 265252859812191058636308480000000

```

A branch of `if` written as a Q-expression is that data, not run: write `(if c (a) (b))`, not `{a}`. A call in tail position takes its caller's place, so a tail-recursive loop runs in constant space; other calls nest 2,500 deep at most.

Files and programs

```

hylang> (write-file "/ram/x.txt" "hello file")
=> NIL
hylang> (read-file "/ram/x.txt")
=> "hello file"
hylang> ((stat "/ram/x.txt") :length)
=> 10
hylang> (ls "/rom")
=> {"README" "bench" "bin" "doc" "lib" "sample" "songs"}
hylang> (sh-out "ls /rom/bin | wc -l")
=> "      6\n"
hylang> (run "ls" "/rom/lib")
forth/
hylang/
namespace
profile
shell
=> 0

```

Files: `read-file`, `read-lines`, `write-file`, `append-file`, `ls`, `dir`, `stat`, `exists?`, `dir?`, `mkdir`, `remove`, `rename`, `copy-file`, `glob`, `cd`, `cwd`; streams with `open`, `read-line`, `read-byte`, `read-bytes`, `write-bytes`, `seek`, `close`. Programs: `(run prog args...)` (its exit code), `(sh line)` (rc's), `(sh-out line)` (its output), `(spawn prog args...)` and `(wait task)`. The environment is rc's variables: `$name`, `(env name)`, `setenv`, `unsetenv`. The clock: `(time)` (seconds since 2000-01-01), `(date)`, `(ticks)`, `(sleep 1/10)`.

`(load "file")` runs a file; `(use "name")` loads a library once, from `/lib/hylang` (the ROM's, and a card's or the RAM disk's before it).

The Hydra

Built in, for what isn't a file:

Area	Functions
Tasks	(ps) (a hash each), (task-info t), (pid), (yield), (hold body...) (no task switch meanwhile)
Notes	(note task :interrupt), (note-group g n), (on-note f) (each note given to f)
The namespace	(bind new old [:before :after] [:create]), (mount "#f" old [spec]), (unmount old [new]), (ns), (news)
Memory	(peek a), (poke a b), (banks), (bank-alloc n), (bank-read bank ofs n), (bank-write bank ofs bytes); shared segments: (seg-create banks), (seg-read ...), (seg-write ...); (free)
The system	(sysinfo), (mods), (errstr :noent)
Keys	(key) (the next key, raw: a character, or :up, :f1 ...), (key?)

Devices

Each device has a library over its files, loaded by `use`; each is also an example of driving the device by hand:

(use ...)	For
"cons"	The windows: (window), (windows), (new-window), (show-window n), (raw-on), (beep)
"gpio"	Port A's pins: (gpio pin), (gpio! pin level), (gpio-port), (gpio-wait) (CA1's next edge)
"i2c"	(i2c-devices), (i2c-read addr n [reg]), (i2c-write addr bytes [reg])
"spi"	(spi dev bytes) (a transaction), (spi-mode dev 3)
"snd"	The YM2151's channels 0-7, and with a Vera X its PSG's 8-23: (snd-claim 0), (snd-patch 0 n), (snd-note 0 60), (snd-wave 8 :saw) (a PSG channel's waveform), (tune {{60 1} {64 1} {67 2}}) (MIDI notes, beats; (note-of "C4") a note's number), (snd-mm1 0 "t180 o4 l8 c d e") and (snd-chord 0 "o4 c e g") (the score language: play's -m and -c), (play path)
"disk"	(disks), (df), (cards), (disk-start d), (disk-stop d)
"proc"	Another task's (task-args t), (task-cwd t), (task-env t), (task-regs t), (task-mem t addr n)
"clock"	(set-date "2026-10-04 12:00:00"), (rtc)
"pc"	(pc?): whether the PC tool answers (/pc's files are files)
"screen"	The terminal: cls, at, color, bold

```
hylang> (use "cons")
=> NIL
hylang> (window)
=> 0
```

Every system call

Each call a program makes is a function, `sys-` and its name: (`sys-getpid`), (`sys-open "/rom/README" 0`). Its arguments are the registers the call takes, and its value what it gives (one, or a list; `NIL` for none); a failure is the system's error: `/rom/doc/api.md` lists each call's (the `hylang` column). (`sys :name args...`) makes any of them by name.

Errors and Ctrl-C

An error is a value: it stops what it reaches, and the REPL shows it (`=> Error: ...`). (`try x handler`) catches one, `&err` its message and `&code` its code; a system failure's code is an atom (`:noent`, `:exist ...`).

```
hylang> (try (/ 1 0) {+ "caught: " &err})  
=> "caught: Division by zero."  
hylang> (error-code (read-file "/nosuch"))  
=> :noent
```

Ctrl-C is the error `interrupted (:intr)` at the next call or loop step, which `try` can catch; `(on-note f)` gives it to a function instead (T to go on).

Using BASIC

BASIC is the Hydra-16's BASIC: EhyBASIC, which is Microsoft BASIC 2A for the 6502 (the BASIC of the Commodore PET and its cousins), with the Hydra's files, sound, system calls and memory added. It's also a shell: `basic -l` runs a line as BASIC when it is BASIC's, and as an rc command line otherwise. This guide is for using it; [../basic.md](#) is its design (what was converted, and why each addition is as it is).

Contents: [Starting it](#) · [The basics](#) · [Statements and functions](#) · [Programs in files](#) · [Files](#) · [Sound](#) · [Machine code and system calls](#) · [Memory](#) · [The shell](#) · [Errors and Ctrl-C](#)

Starting it

Typed at rc's prompt	What runs
<code>basic</code>	BASIC at the console: the banner, then OK after each command
<code>basic prog.bas a b</code>	A script: the program loaded and run, then BASIC ends (code 1 after an error). A file whose first line is <code>#!/bin/basic</code> runs by its name too (<code>./prog.bas</code>)
<code>basic <prog.bas</code>	The file's lines as if typed, quietly (no banner, no OK); its end ends BASIC
<code>basic -l</code>	BASIC as a login shell: its namespace made, its profile run, then its prompt (<code>/> </code>)

bye ends BASIC (and exit the shell). To make BASIC every window's shell, put a line `/bin/basic -l` in `/lib/shell` on a card (`/sd/0/lib/shell`) or the shared RAM disk.

The basics

A line typed with a number first is part of the program; any other is run at once. Keywords and names are taken in either case (print is PRINT), and LIST shows them in capitals.

```
EHYBASIC FOR THE HYDRA-16 (MICROSOFT BASIC 2A)
38550 BYTES FREE

OK
10 FOR I=1 TO 3: GOSUB 100: NEXT: ? "done"
20 END
100 ? I; I*I;: RETURN
run
  1  1  2  4  3  9 done

OK
list

10 FOR I=1 TO 3: GOSUB 100: NEXT: PRINT "done"
20 END
100 PRINT I; I*I;: RETURN
OK
```

? is PRINT. EhyBASIC's short forms are still taken, and LIST shows the full names: JSR (GOSUB), RTN (RETURN), RSTR (RESTORE), CLR (CLEAR), LT\$ RT\$ MD\$ ST\$ CH\$ (LEFT\$ RIGHT\$ MID\$ STR\$ CHR\$), & | ! (AND OR NOT). As in every Microsoft BASIC, a keyword is found anywhere in a line, even inside a name (SCORE is SC OR E), and only a variable name's first two characters count. Numbers are 9-digit floating point; A% is an integer, A\$ a string (255 characters at most).

Statements and functions

Statements	
LET (or none) PRINT ? INPUT GET READ DATA RESTORE	Variables, the console, data in the program
GOTO GOSUB RETURN ON ... GOTO/GOSUB IF ... THEN FOR ... TO ... STEP NEXT END STOP CONT	Control
DIM DEF FN CLEAR NEW RUN [line "name"] LIST [from-to] REM	The program and its variables
LOAD "name" SAVE "name" [,B]	Programs in files
OPEN n,"name" [, "R" "W" "A"] CLOSE [n] PRINT #n, INPUT #n, GET #n,	Files
SOUND ch,note[,patch[,vol]] SOUND ch SOUND "words" PLAY [ch,]"mml" PLAY "song.zsm" BEEP SLEEP s	Sound and time
SYS addr "NAME" [,a[,x[,y]]] RREG a,x,y,p POKE addr,b WAIT addr,mask[,eor] HIMEM n	Machine code, system calls, memory
BYE	BASIC's end

Functions	
ABS INT SGN SQR RND LOG EXP SIN COS TAN ATN	Numbers (RND(1) the next random number, RND(-n) seeds it)
LEN LEFT\$ RIGHT\$ MID\$ STR\$ VAL ASC CHR\$	Strings
FRE(0) POS(0) PEEK(addr) USR(x)	Memory free, the column, a byte, machine code
EOF(n) ENV\$("name") ARG\$(n)	A file's end; the environment's variable (ENV\$("home")); a script's argument (ARG\$(1) the first, ARG\$(0) the script's name, "" past the last)

Programs in files

SAVE "name" writes the program as text, its listing, so cat shows it and an editor can change it; LOAD "name" reads it back (a line without a number; a #! line say, is passed over). SAVE "name",B writes it tokenized, faster to load but readable only by this BASIC; LOAD tells the two apart. RUN "name" loads and runs. Names are paths as anywhere else (/sd/0/games/hello.bas, or relative to the current directory).

```
#!/bin/basic
10 PRINT "hello from a script, ";ARG$(1)
```

Files

```
10 OPEN 1,"scores.txt","W"
20 FOR I=1 TO 3: PRINT #1, I;",";I*I: NEXT
30 CLOSE 1
40 OPEN 1,"scores.txt"
50 IF EOF(1) THEN 80
60 INPUT #1, A, B: PRINT A; B
70 GOTO 50
80 CLOSE 1
```

Four channels, 1 to 4, each a file or a device (/dev/cons, #n/kmesg ...): read ("R", as with no mode), written ("W": made, or emptied first) or added to ("A"). INPUT #n reads a line's comma-separated values (its end is ?END OF FILE ERROR: check EOF(n) first), GET #n a byte at a time ("" at the end). RUN, NEW and CLEAR close them all, and so does entering a program line.

Sound

`SOUND ch, note, patch, vol` plays a MIDI note (60 is middle C) on one of the YM2151's channels (0-7), with an instrument (0-127: General MIDI's; 128-162: drums and percussion) and a volume (0-127) if they're given; `SOUND ch` lets the note go. With a Vera X, channels 8-23 are its PSG's voices: there a patch below 4 is a waveform (0 pulse, 1 sawtooth, 2 triangle, 3 noise), and with no card they're silent. `SLEEP s` waits `s` seconds (`SLEEP .25`). `SOUND "..."` sends the sound driver a line of text: its words ("claim 255", "volume 150", "reset") and each channel's commands ("pan 0 left", "bend 0 -32", "drum 9 38", "freq 0 440", "level 0 90", "wave 8 saw" ...: tools.md). `PLAY "t180 o4 18 c d e f g"` plays a line of MML (the score language: tools.md, "Scores") on channel 0, `PLAY 2, "..."` on channel 2 (0-7: the YM2151's), and `PLAY "song.zsm"` a song or a score; each waits till it's played. `BEEP` rings the bell.

```
10 FOR N=60 TO 72: SOUND 0,N,0,100: SLEEP .2: NEXT: SOUND 0
```

Machine code and system calls

`SYS "NAME"` makes one of the system's calls by its name (`/rom/doc/api.md` lists them), its registers `.A .X .Y` from `SYS`'s numbers and `r0-r15` from bytes 2-33 (POKE them just before the `SYS`); `RREG A,X,Y,P` reads the registers after it (bit 0 of `P` set: it failed, `A` the error).

```
SYS "GETPID": RREG T: PRINT "my task is"; T
SYS "TICKS": RREG L,H: PRINT H*256+L
```

`SYS address` calls machine code. Keep room for it at the top of BASIC's memory with `HIMEM: HIMEM 40704` keeps `$9F00-$9FFF` (40704-40959), where it can be POKEd. `USR(x)` jumps through bytes 1285-1286.

Memory

BASIC has the task's RAM and a RAM bank of its own after it, one piece to `$9FFF`: 38,550 bytes free for the program, its variables, arrays and strings (`FRE(0)`). Leave the bank register (address 0) alone while BASIC runs: the window at `$8000` is BASIC's own memory.

The shell

`basic -l` is a shell: a line is BASIC's if it's a program line, `?`, a statement's keyword first (`print`, `run`, `list`), or an assignment (`x=5`, `a$(1)="hi"`); any other is an `rc` command line, run by `rc` and waited for.

```
/> print 1+1
2
/> ls /rom/lib/basic
profile.bas
/> cd /ram
/ram> echo $status
0
```

A name in both is BASIC's (`sleep`, `wait`, `if`, `for`): put `%` before a line to make it `rc`'s whatever it is (that works at a plain BASIC's prompt too). `cd`, `bind`, `mount`, `unmount` and `newns` are the shell's own, as an `rc` line can't change BASIC's directory or namespace; `exit` ends it with the last command's code; a line ending in `&` runs in the background (`$apid`). The prompt is the current directory. `basic -l` runs `/lib/basic/profile.bas` as typed lines before its first prompt: a card's or the RAM disk's in the ROM's place, through the `/lib` union.

Errors and Ctrl-C

An error stops the program with Microsoft's message and its line (`?DIVISION BY ZERO ERROR IN 30`); the system's show the same way (`?NOT FOUND ERROR`). Ctrl-C stops a program (`BREAK IN 20`, and `CONT` goes on), or the `INPUT` it waits in; at the prompt it gives a new line.

The programmer's guide

How to write programs for the rebuilt Hydra-16 (reborn/): what the system gives a program, how it's called, and how the parts fit. It's for someone who knows 65C02 assembly or C and wants to write for this system; the SDKs' guides ([../sdk/asm/README.md](#), [../sdk/c/README.md](#)) cover building and running, and [/rom/doc/api.md](#) on the Hydra (the build's [obj/gen/api.md](#), made from [spec/api.def](#)) is every call's reference. The rules every part of the system keeps are [../conventions.md](#); this guide is the gentler way in. The whole system in one place is [../hydra-16.md](#), and the board in full [../hardware.md](#).

Chapter	What it covers
The system in one page	Below: the parts, and where a program fits
calls.md	Calling the system: the jump table, registers, errors, the SDK's macros
memory.md	A task's 64K, its break and pages, its banks, shared segments, semaphores
tasks.md	Starting and ending programs, waiting, notes (signals), time, /proc
files.md	Files and directories, the namespace, devices, the environment
servers.md	Writing a server or a driver: <code>srvlib</code> , requests, waiting, interrupts
modules.md	The HYX2 format, programs in RAM and in the ROM, modules of several banks, libraries and <code>XCALL</code>
video.md	The screen, the Vera X: <code>/dev/vid</code> 's files, the console's terminal, claiming the chip, VRAM, the registers

The system in one page

The hardware a program sees. A 65C02 at 3.58 MHz (7.16 with a jumper). Sixteen tasks, each with its own `$0000-$7FFF` (zero page, stack and 32K of RAM) and its own bank registers: `$00` picks one of its RAM banks for `$8000-$9FFF`, `$01` one of the paged ROM's banks for `$A000-$DFFF`. `$E000-$FFFF` is the BIOS ROM, where the kernel and the jump table live; the I/O area is `$FF00-$FFEF`, and belongs to the drivers.

The kernel is in the BIOS ROM, a task of its own (task 0): the scheduler (round robin, preemptive, 200 ticks a second), the IRQ path, calls between tasks, memory, notes, files' channels and namespaces. Its state is in task 0's RAM, out of other tasks' reach.

Modules are everything else, in the paged ROM, each running in place in a task of its own: the drivers (`cons` the console and `/pc`, storage the SPI bus and every disk, `kdev` the kernel's devices, `snd` the YM2151 and the Vera X's PSG, `gpio` the VIA's port A and I2C, `vid` the Vera X: its screen, its PSG and its PCM), `init`, the shells and the tools. A program in a file (on a card, a RAM disk, `/pc`) is read into a task's RAM at `$0800` and runs there.

Files, Plan 9's way: every device is a file server; a tree of files; a program opens, reads and writes them with the same calls whatever's behind them, and controls a device by writing commands to its `ctl` file. Each task has a namespace, the tree it sees, built from binds and mounts (`/rom/lib/namespace`), shared with its children until one changes it.

The console is windows on the serial terminal, and on the Vera X's screen if there's one: each a whole console with its own `cons`, line editor and shell. The login shell is HyForth over `rc`; `rc` is the shell of scripts and `system()`.

task 0	kernel	the scheduler, IRQs, calls, memory, notes, channels, namespaces
task 1	init	the namespace, the RAM disks, window 0's shell, <code>wstart</code>
task 2+	programs	the shells, the tools, yours: in place from the paged ROM, or in RAM from <code>\$0800</code>
task A	vid	<code>#v /dev/vid</code> : the Vera X (with no card it ends as it starts)
task B	gpio	<code>#g /dev/gpio</code> , <code>#i /dev/i2c</code>
task C	snd	<code>#a /dev/snd</code> , <code>sndctl</code> , <code>bell</code> , <code>psg</code>
task D	kdev	<code>#/ #n #t #m #p # #e #s #r</code> : the root, null, ticks, modules, /proc, pipes, /env, segments
task E	storage	<code>#S /dev/spi</code> , <code>#d /dev/sd</code> , <code>#f HydraFS: /rom, /ram, /sram, /sd/N</code>
task F	cons	<code>#c /dev/cons ...</code> , <code>#P /pc</code>

A first program

`sdk/asm/samples/hi/hi.s` greets its arguments and says where it runs: its arguments come at `r0`, it prints with `PUTS` and `PUTC` (fd 1), asks the kernel its task (`GETPID`) and its directory (`GETCWD`), and reads its environment (`ENV_GET`); returning from `main` ends it with code 0. [calls.md](#) walks through it. Build it with `node build.js prog DIR` (the folder of its `.s` files), and run it from a card, a RAM disk or /pc: [../tutorial.md](#), step 8.

Calling the system

The jump table

A program calls the system with `jsr` to the call's slot in the jump table, on BIOS ROM page 0 from `$F800: jsr OPEN`. The slots' addresses are in `hydra.inc` (the assembly SDK) and `hydracalls.h` (C), both made from `spec/api.def` by the build; a slot never moves once it's published (a call is only ever added to the end of its group, and a withdrawn one answers `E_NOSYS`), so a program built today runs on later ROMs.

The calls come in groups: **system** (`SYSINFO`, `ERRSTR`, `KMSG`, `REBOOT`, `XCALL`), **task** (`SPAWN`, `EXITS`, `WAIT`, `SLEEP`, `NOTE`, the environment's `ENV_*`, semaphores `SEM_*` ...), **memory** (`BREAK`, `PAGES_*`, `BANKS_*`, `SEG_*`), **file** (`OPEN`, `READ`, `WRITE`, `STAT`, `PIPE`, `FD2PATH` ...), **name** (`BIND`, `MOUNT`, `UNMOUNT`), **cons** (`PUTC`, `PUTS`, `GETC`: fd 1 and fd 0), **time** (`TIME`, `TIME_SET`, `RTC`) and **server** (for drivers: `SRV_*`, `CLIENT_*`, `IRQ_OWN` ...). `/rom/doc/api.md` has each one's registers, errors and whether it waits.

Registers

Where	What
<code>.A</code> , <code>.X</code> , <code>.Y</code>	Small arguments and results; a 16-bit value is <code>.A</code> (low) and <code>.X</code> (high)
<code>r0-r15</code> (<code>\$02-\$21</code>)	The call registers: pointers and 16-bit values, two bytes each, low first
C	0: success; 1: failure, with the error code in <code>.A</code>. Always, for every call
<code>\$22-\$7F</code>	The program's own zero page: no call touches it

A call may change `.A`, `.X`, `.Y`, `r0-r15` and the flags; keep what you need in `$22-$7F` or your RAM. The error codes (`E_NOENT` "not found", `E_INTR` "interrupted" ...) are `spec/errors.def`'s; `ERRSTR` gives one's text.

The SDK's macros

`macros.inc` (assembly):

Macro	What it does
<code>CALL name</code>	A system call (the same as <code>jsr name</code> ; it reads as one)
<code>CHECK label</code>	On to <code>label</code> if the call before it failed (<code>C = 1</code>)
<code>LDR reg, value</code>	A call register = a 16-bit value (an address)
<code>MOVR to, from</code>	One register (or zero-page word) = another
<code>PRINT label, PRINT "text"</code>	A string to fd 1

None defines a label (but unnamed ones), so the code around them keeps its cheap locals.

A program, walked through

`sdk/asm/samples/hi/hi.s`:

```

.include "hydra.inc"                ; The calls and constants (made from spec/api.def)
.include "hyx2.inc"                 ; The header: HYX2_PROGRAM
.include "macros.inc"               ; LDR, MOVR, PRINT, CALL, CHECK

        HYX2_PROGRAM "hi", main      ; Its name, and where it starts

.zeropage
arg:     .res      2                  ; An argument (its zero page: $22-$7F)

.bss
cwd:     .res      PATH_MAX + 1      ; Its current directory

```

`main` gets its arguments at `r0`: strings one after another, each with a zero byte, and an empty one after the last (the program's name isn't one of them). It prints `Hello, NAME!` for each:

```

@name:
        PRINT      "Hello, "          ; "Hello, NAME!"
        MOVR       r0, arg
        CALL       PUTS
        PRINT      s_bang

```

then its task and directory:

```

        CALL       GETPID              ; .A = this task (0-15)
        ...
        LDR        r0, cwd
        CALL       GETCWD              ; Its current directory, into cwd
        PRINT      cwd

```

and a variable of its environment, which may not be there:

```

        LDR        r0, s_window        ; Its window, if its environment has one
        LDR        r1, window
        LDR        r2, 7
        stz        r3
        stz        r3 + 1
        lda        #$FF                ; (This task's)
        CALL       ENV_GET
        CHECK      @end                ; (Not there: C = 1, .A = E_NOENT)

```

`rts` from `main` ends it with code 0; `EXITS` ends it anywhere, with a code (`.A`) and a message (`r0`, or 0): its status, which the parent gets from `WAIT` (and `rc` as `$status`).

From C

The C SDK (`sdk/c`) is cc65 with the Hydra's library under the standard one: `stdio` over `fds`, `open/read/write`, `stat`, `opendir`, `getenv`, `system` (`rc -c`), `signal` over notes, `conio` over the console's raw mode. A failed call returns -1 and sets `errno` (and `_oserror` the system's own code). Any call is `hy_call (HY_NAME, ®s)`; `hydra.h` wraps the Hydra's own: `hy_spawn`, `hy_wait`, `hy_note`, `hy_bind`, `hy_banks_alloc`, `hy_sem_*`, `hy_fd2path` ... [../..sdk/c/README.md](#) is its guide.

From HyForth, hylang and BASIC

Each call a program makes is a word in HyForth (`sys-open`, in `hydra.fl`: its registers as stack items) and a function in hylang (`((sys-open "x" 0))`). `/rom/doc/api.md` shows both forms for each call. BASIC's `SYS "NAME"` makes any of them by its name (`.A`, `.X` and `.Y` from its numbers, `r0-r15` POKEd before it; `RREG` reads them after: [../using/basic.md](#)).

Memory

A task's 64K

Where	What
\$00-\$01	Its bank registers: the RAM bank at \$8000, and the paged ROM bank at \$A000
\$02-\$21	r0-r15, the call registers
\$22-\$7F	The program's own zero page
\$80-\$FF	The system's (the task's state; one byte a program writes: TASK_EVENT, a server's event count)
\$0100-\$01FF	The stack
\$0200-\$03FF	The system's: the task's fds, name, arguments (TASK_ARGS, \$0350), current directory ...
\$0400-\$7FFF	The program's: a RAM program from \$0800 (its header, code, data, then BSS); then its break; pages from the top down
\$8000-\$9FFF	The RAM bank \$00 picks: one of the task's own, or a shared one
\$A000-\$DFFF	The paged ROM bank \$01 picks: a module running in place has its own there
\$E000-\$FFFF	The BIOS ROM (page 0: the jump table at \$F800); the I/O area \$FF00-\$FFEF

Every task has its own copy of \$0000-\$7FFF, so a stray write lands in its own RAM, not another's.

The break and pages

BREAK moves the end of the program's data (its break: r0 the new end, or 0 to ask), as Unix's brk. Above it, PAGES_ALLOC gives a run of 256-byte pages from the top down (.A pages; r0 the first's address), and PAGES_FREE gives them back. The two never overlap: the break can't rise over a page given out. C's malloc and HyForth's allot sit on these; fine-grained allocation is the language's.

Banks

A task has 16 banks of 8K for each RAM module installed (the board's default is two or three modules: 256K or 384K a task). BANKS says how many and which modules are good (r0, a bit each; module m's banks are \$m0-\$mF). BANKS_ALLOC takes a run from the lowest free (.A banks; .A the first), BANKS_ALLOC_IN from a range (.X the lowest, .Y the highest: one module's), and BANKS_FREE gives them back. The bitmap is only so libraries in one program don't collide: a program selects a bank by writing \$00, as on the X16, and reads and writes it at \$8000-\$9FFF.

```
lda    #4
jsr    BANKS_ALLOC                ; Four banks: .A = the first
bcs    @nomem
sta    $00                        ; The first at $8000
lda    #$5A
sta    $8000
```

In C: hy_banks_alloc (n) and hy_bank (b), the window at HY_BANK_WINDOW.

Shared segments

A segment is a run of shared banks that several tasks attach to: SEG_CREATE makes one of .A banks (attached to the caller; .A the segment), SEG_ATTACH and SEG_DETACH count a task in and out (the last out frees it, and a task's end detaches it), and SEG_MAP says how to reach bank .X of it: .A the value for the U register (\$FFF1) and .X the value for \$00.

```

lda    seg
ldx    #0                      ; Its first bank
jsr    SEG_MAP
sta    $FFF1                   ; U: the shared macro-page
stx    $00                     ; Then the bank: at $8000

```

To find a segment by name rather than by passing its number, name it in `/dev/seg (#s)`: write name `NAME N` to `/dev/seg/ctl`, and any task reads `/dev/seg/NAME` for its number (3 2: segment 3, of 2 banks), then attaches. The name holds a reference of its own, so the segment stays while it's named; `free NAME` lets go. `SEG_CREATE_IN` makes one from a range of the shared bank IDs (\$80-\$FF are segments'); `SEGINF` and `free` say what's used.

In C: `hy_seg_create`, `hy_seg_attach`, `hy_seg_detach`, and `hy_seg_map` (a bank of it at `HY_BANK_WINDOW`: U, then \$00). The race, philo and prodcons samples share one between their tasks ([../sdk/c/README.md](#)).

Semaphores

For tasks that share something (a segment, a device, a file) or wait for each other, the kernel keeps 16 semaphores, every task's by number:

Call	What it does
SEM_NEW	A semaphore: .A its count, .X 0 or SEM_MUTEX (a mutex: a count of 1, given back only by the task that took it); .A its number
SEM_ACQUIRE	Take one of its count, waiting (using no CPU) till there's one; a note ends the wait (E_INTR)
SEM_TRY	The same without the wait: E_AGAIN if there's none
SEM_RELEASE	Give one back; the tasks waiting wake, and the first to run takes it
SEM_FREE	Free it; its waiters wake with E_INVALID

A task's end frees the semaphores it made and gives back the mutexes it holds. A mutex's holder that asks again gets `E_BUSY`, not a wait for ever.

```

lda    #0
ldx    #SEM_MUTEX
jsr    SEM_NEW                  ; A mutex
sta    lock
...
lda    lock
jsr    SEM_ACQUIRE             ; Ours, till we give it back
bcs    @interrupted
...                              ; (The shared thing, alone)
lda    lock
jsr    SEM_RELEASE

```

In C: `hy_sem_new`, `hy_sem_acquire`, `hy_sem_try`, `hy_sem_release`, `hy_sem_free`. The C SDK's multitasking demos use them each way: a mutex around shared memory (race) and around the console (chorus), a semaphore for each task passed round as a baton (chorus), forks as mutexes and a deadlock (philo), counting semaphores for a ring's free and filled slots (prodcons), and barriers, each task giving "ready" one and taking one of "go" (all of them).

Another task's memory

Only through `/proc: /proc/N/mem` is task N's 64K as it sees it, and `/proc/N/ram` its banks (bank b's byte o at `b * $2000 + o`), read and written as files, any task's but the kernel task's and a driver's. `init` alone may read `#r` (`#r/task`: every task's RAM; `#r/shared`: the shared banks), the system's raw view.

Tasks, notes and time

Starting a program

SPAWN starts a program in a task of its own: `r0` its path (through the namespace: `/bin/ls`, or `#m/ls`, a module in the paged ROM), `r1` its arguments (zero-terminated strings, an empty one after the last; 176 bytes at most), `.A` flags; `.A` gives the new task. A module of the paged ROM runs in place; any other program is read into the new task's RAM at its load address (`$0800`) by the task itself as it starts.

Flag	Meaning
SPAWN_NEWGROUP	A note group of its own (else the caller's: Ctrl-C reaches both)
SPAWN_NEWNS	An empty namespace (else it shares the caller's)
SPAWN_FDMAP	Its fds from the map at <code>r2</code> : a count, then for each of its fds from 0 the caller's fd that becomes it (<code>\$FF</code> : closed)
SPAWN_NOENV	An empty environment (else a copy of the caller's)
SPAWN_STOPPED	Stopped at its entry point, for a debugger (TF_STOPPED in its TI_FLAGS once it's there)

The child gets the caller's fds 0, 1 and 2 (sharing their channels and offsets), its current directory, and a copy of its environment. Its parent is the caller (GETPPID says so).

Ending, and waiting

EXITS ends the calling task with a status: `.A` a code (0 success), `r0` a message (31 characters) or 0. Returning from a program's entry point is EXITS with 0. Everything the task had (its fds, pages, banks, segments' references, semaphores, IRQ lines) is given back.

WAIT waits for a child to end (`.A` the task, or `$FF` any) and takes its status: `.A` the task, `.X` its code, its message in `r0`'s buffer. A child that ended first keeps its status till it's waited for; a parent that ends first leaves its children to init. `rc` shows a status as `$status`: the message, or the code.

Notes

Notes are Plan 9's signals: numbered (1 Ctrl-C's interrupt, 2 kill, 3 hangup, 4 alarm, 5 brk a BRK instruction; 16-31 a program's own), sent by NOTE (`.A` a task, or NOTE_GROUP | g a note group; `.X` the note) or by writing to `/proc/N/note` or `ctl`. Ctrl-C at a window sends interrupt to the group that has the window.

A task takes its notes in its own code, never inside the kernel: at a switch into it, or at the end of a system call that waits (which then gives E_INTR). With no handler, the default ends the task (128 + the Unix number: 130 for Ctrl-C, 137 for a kill). NOTIFY (`r0` a routine) sets a handler: it's called with `.A` the note, and returns `C = 0` to go on where the task was, or `C = 1` for the default. A kill is never the handler's.

`sdk/asm/samples/tick/tick.s`:

```

main:
    LDR        r0, handler                ; Its note handler
    jsr        NOTIFY
@second:
    lda        #<TICK_HZ                  ; A second (TICK_HZ ticks): a note ends it sooner
    ldx        #>TICK_HZ
    jsr        SLEEP
    lda        noted
    bne        @end
    ...

; The note handler: .A = the note.  C = 0: the task goes on where it was (here, SLEEP ends early); C = 1 would be the
; default
handler:
    sta        noted
    clc
    rts

```

A program that runs code in more than one bank (a module of several banks, or one that makes XCALLs) keeps its handler in its RAM (its DATA segment): the kernel calls it with whichever bank is at \$A000 when the note comes. In C, `signal (SIGINT, f)` is a handler for Ctrl-C's note.

Time

TICKS is the tick count (TICK_HZ, 200 a second; 16 bits, so it wraps after about 5.5 minutes). SLEEP waits .A/.X ticks, SLEEP_UNTIL till a tick count (for steady timing), YIELD lets the others run. PREEMPT_OFF and PREEMPT_ON hold the CPU (no task switch; interrupts go on) for a few ticks' worth of timing-critical work. TIME is the clock, seconds since 2000-01-01 in r0:r1, set from the DS1747 at boot if there's one (TIME_SET sets it; /dev/time is it as text).

Tasks working together

A task shares nothing with another but what it's given: fds (pipes, files, devices), shared segments, semaphores, notes, and its exit status. The C SDK's multitasking demos ([../sdk/c/README.md](#)) put them to work: each starts copies of itself (SPAWN), passes them a segment's and semaphores' numbers as arguments, meets them at a barrier, draws them as they go, and waits for their ends (WAIT). round keeps four tasks in time with SLEEP_UNTIL alone, from a start they share.

Seeing tasks

TASKINFO says what the kernel knows of a task (its state, flags, parent, CPU time, name, group: TI_*); TASKREAD reads its arguments, current directory, environment, registers, or an open fd (TR_*). /proc/N has them as files (status, args, cwd, env, regs, fd, ns, mem, ram), with note and ctl (kill, interrupt, note N, stop, start, step, next, break, nobreak) to act on it. ps and top read them.

Debugging

A debugger is a program over /proc (db is one: [../using/tools.md](#)). It starts its program with SPAWN_STOPPED, or stops a task with ctl's stop; reads its registers with TASKREAD's TR_FRAME and its memory through mem; and steps it with ctl's step (one instruction) or next (a JSR's subroutine run whole), waiting for TF_STOPPED again (TASKINFO). Behind ctl is TASKSTEP, a driver's call (kdev's). The kernel runs the instruction out of line: a copy of it in the task's own zero page with a BRK after it, which stops the task again with its PC where the instruction went (a branch has two BRKs, taken and not). JMP, JSR, RTS and RTI it does itself,

on the task's frame; a JSR into the jump table always runs whole. So a step is the instruction as the task would have run it, in its own banks, and takes no breakpoint in a ROM. A task in a call (its PC in the kernel, or waiting) can't be stepped: E_BUSY till it's run on and stopped in its own code.

Breakpoints are the debugger's: a BRK written into the program's RAM through mem. After ctrl's break, any BRK stops the task, its PC back on the BRK, rather than giving it the note sys: brk; nobreak gives a BRK back to the program. A subroutine that reads the bytes after its JSR (its arguments, as some libraries' do) can't be stepped over: its return address is in the kernel's copy.

Files and the namespace

Files

A task has 16 fds (0 stdin, 1 stdout, 2 stderr). Each names a channel, an open file in the kernel's table; DUP and a child's inheritance share a channel, and so its offset.

Call	What it does
OPEN	<code>r0</code> a path, <code>.A</code> a mode (<code>O_READ</code> , <code>O_WRITE</code> , <code>O_RDWR</code> , with <code>O_TRUNC</code> , <code>O_NONBLOCK</code>): <code>.A</code> an fd
CREATE	A file made (or emptied), and opened; <code>.X = DM_DIR</code> makes a directory
READ, WRITE	<code>.A</code> an fd, <code>r0</code> a buffer, <code>r1</code> a count: <code>.A/ .X</code> the count done (READ: 0 at the file's end)
SEEK	An fd's offset (<code>r0</code> , <code>r1</code> : 32 bits, signed), from the start, the offset or the end (<code>.X</code>)
CLOSE, DUP, DUP2	As Unix's
STAT, FSTAT	A file's stat record (64 bytes, <code>SR_*</code> : name, qid, mode, length, time, device)
WSTAT, FWSTAT	A file changed from a stat record: renamed, its mode, its length (fields left all <code>\$FF</code> stay)
REMOVE	A file, or an empty directory
PIPE	Two fds, a pipe's ends: <code>.A</code> to read, <code>.X</code> to write (512 bytes held between them)
FD2PATH	The name an fd was opened by, whole and clean
CHDIR, GETCWD	The current directory

A directory reads as stat records, one after another (`SR_SIZE` each): no text to take apart. READ and WRITE wait as their file's server needs (a pipe that's empty, a console with no line yet); a note ends the wait with `E_INTR`, and an fd opened `O_NONBLOCK` answers `E_AGAIN` instead. `PUTC`, `PUTS` and `GETC` are a byte or a string to fd 1, and a byte from fd 0.

Names

A name is made whole and clean before it's looked up: a relative one after the current directory, then `.`, `..` and empty elements gone. A name that starts with `#` is a device's own (`#c/cons`: the console's; `#c2/cons`: window 2's), in no namespace; any other is found in the task's namespace.

The namespace

The namespace is a table of the task's binds and mounts, Plan 9's: a name's mount point is the longest one it starts with, and the union there, its members in order, is where it's looked for. OPEN, STAT and REMOVE try the members in turn till one has the name; CREATE goes to the member bound with MCREATE (or the first). A directory opened at a union reads as every member's records.

Call	What it does
BIND	<code>r0</code> new at <code>r1</code> old: MREPL in place of what's there, MBEFORE or MAFTER in its union, MCREATE for creates
MOUNT	A device's tree (<code>.X</code> its letter, <code>r0</code> its spec: which of its trees) at old, the same flags
UNMOUNT	All of old, or its member new
NSINFO	An entry of a task's namespace (<code>/proc/N/ns</code> reads them as binds and mounts)

A child shares its parent's namespace till one of them changes it (then it's copied); SPAWN_NEWNS gives it an empty one, and `nslib.s` (the SDK's) builds the default from `/rom/lib/namespace` as `rc`'s `newns` does. The default namespace (`/rom/lib/namespace`):

```

bind '#/' /
bind -a '#c' /dev          # cons consctl ser serctl
bind -a '#n' /dev          # null zero kmesg
bind '#d' /dev/sd
bind '#m' /dev/mod
bind '#s' /dev/seg
mount '#e' /env            # the environment
mount '#p' /proc
mount '#f' /sd             # the cards: /sd/0 ... /sd/f
mount '#f' /rom x          # the ROM disk
mount -c '#f' /ram r/$task # this shell's own area of the RAM disk
bind -c /ram/bin /bin
bind -a /rom/bin /bin      # programs on the ROM disk
bind -a '#m/bin' /bin      # the ROM's program modules
...

```

Devices

Each device is a file server, a driver's (or the kernel's own, `kdev`'s): a directory of files, controlled by writing commands to its `ctl`, and read as text where that's what it is.

Letter	Device	Files
#c	The console (a window each: #cN)	cons, consctl (rawon, rawoff, group), ser, serctl (the rate), wctl, wnew, kbdi
#d	The disks	N/data, N/ctl: 0-f cards, x the ROM disk, r and s the RAM disks
#f	HydraFS	A disk's file system (the spec: its disk)
#S	SPI devices	N/data (a transaction), N/ctl (mode 0, mode 3)
#g, #i	GPIO, I2C	0-7, port, ctl, ca1; ctl and a file each device
#a	Sound	snd, sndctl, bell, psg (the Vera X's PSG)
#v	The Vera X	ctl, term, vram, pal, sprites, font, frame, psg, pcm, pcmctl
#t, #n	Time, null	time, ticks; null, zero, kmesg
#m	Modules	A file each (its image), bin
#p	Tasks	N/status, args, cwd, env, ns, fd, regs, mem, ram, note, ctl
#e, #	The environment, pipes	/env/NAME; pipe
#s, #r	Segments' names, raw RAM	ctl and a file each name; task, shared (init's)
#P	/pc	A folder on the PC, through the PC tool

The environment

Each task has its own environment (8K), a copy of its parent's: `ENV_GET`, `ENV_PUT`, `ENV_DEL` and `ENV_NAME` read and change any task's by number (\$FF: this one's), and `#e` serves the caller's as files at `/env`. A value is any bytes: `rc` keeps its variables there, a list's words each with a zero byte after it, and its functions as `fn#NAME`. C's `getenv` and `setenv`, HyForth's `getenv`, and hylang's `$name` are these.

Writing a server or a driver

A server is a task that serves a device letter: its clients' OPEN, READ, WRITE, STAT ... of #x/... come to it as requests. A driver is a server that's a module of the paged ROM, started in a task of its own (the highest free: task F down), at boot if it's flagged so. Every server is built on **srvlib** (sdk/asm/srvlib.inc at the top of its source, srvlib.s at the end), so a new one is mostly tables: its files, and a routine for each that makes or takes its contents.

A server, walked through

sdk/asm/samples/counter/counter.s serves #k: count, a text file (the count), and ctl (add N, reset).

```
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"
.include "srvlib.inc"                                ; (srvlib.s at the end)

HYX2_DRIVER "counter", init, srv_serve, 0, 0, HF_BOOT
```

HYX2_DRIVER's entries: init (run once as it starts), the serve entry (srvlib's srv_serve, which takes each request and does the rest), the irq entry and the stop entry (none here), and its flags (HF_BOOT: started at boot, which waits for its init). init registers the letter:

```
init:
    stz        count
    stz        count + 1
    lda        #'k'
    jmp        SRV_REGISTER                        ; (C = 1, .A = an error: it ends)
```

The tree is a table: each entry its name, its parent (an entry's number), its kind, its handler, its mode and an aux byte:

```
srv_tree:
    SRV_ENTRY  s_root,  $FF, SK_DIR,  0,          SM_READ,          0      ; 0
    SRV_ENTRY  s_count, 0,    SK_TEXT, gen_count, SM_READ,          0      ; 1
    SRV_ENTRY  s_ctl,   0,    SK_CTL,  ctl_cmds,  SM_READ | SM_WRITE, 1      ; 2 (reads as count)
    .word      0
ctl_cmds:
    ; Its commands: the word, the handler
    .word      s_add, c_add
    .word      s_reset, c_reset
    .word      0
```

A text file's handler makes its text, each time it's read (srvlib slices it at the read's offset, so the server keeps nothing between reads); a ctl command's gets its words after it, parsed (srv_argn of them, srv_argp their places, srv_arg their numbers: decimal, or \$hex):

```

gen_count:
    lda        count
    ldx        count + 1
    jsr        srv_tputdec
    lda        #LF
    jsr        srv_tputc
    clc
    rts

c_add:
    lda        z:srv_argn                ; (No number: E_INVALID)
    beq        @invalid
    clc
    lda        count
    adc        srv_arg
    sta        count
    ...

```

```

% cat '#k/count'
0
% echo add 5 > '#k/ctl'; cat '#k/count'
5

```

The kinds of entry

Kind	What it is
SK_DIR	A directory: the entries whose parent it is, read as stat records
SK_TEXT	A text file: its handler makes the text (srv_tputs, srv_tputc, srv_tputdec, srv_tputhex), 255 bytes at most
SK_CTL	A ctl file: a table of commands; a bad one is E_INVALID and changes nothing. Its aux: the entry of a text file it reads as
SK_DATA	Anything else: the handler gets each request (R_OPEN, R_READ, R_WRITE, R_CLUNK ...) and moves the data itself (CLIENT_READ, CLIENT_WRITE), setting the count done
SK_DYN	A directory whose children a handler makes as it's read (the tasks in /proc, the modules in #m); its aux the entry each child is
SK_RAW	A device whose every request goes to one handler, its fids its own (a file system: #f)

SRV_TREES gives a server several letters (a tree each), as kdev has; a request's RQ_DEV says which.

Requests and waiting

A request is a block the client fills in its own memory (RQ_*: the request, its fid, mode, offset, count, buffer, its task and note group); SRV_TAKE copies it into the server's TASK_INBOX, and SRV_REPLY sends back the count done and a new fid. srvlib does both. Data moves by kcopy: CLIENT_READ and CLIENT_WRITE between the server's memory and the client's buffer, as each task sees its own.

A server never waits. When it can't answer yet (a pipe that's empty, a console with no line), it answers E_AGAIN, and the kernel has the client wait till the server's event count changes from what it was as the server took the request, then sends the request again. So a server adds 1 to its event count (inc TASK_EVENT) whenever something its clients may be waiting for has happened: a byte in, room to send. A note ends a client's wait: the server gets R_FLUSH (SRV_FLUSH, if it keeps anything for a client), the client E_INTR. A server that must give up on something that doesn't come times it itself: the kernel has no timed wait for a client.

Interrupts

A driver owns an IRQ line with `IRQ_OWN` (.A the line); its irq entry runs in its own task with the line in .A, IRQs off, and returns .A = 0, or `IRQ_RESCHED` for a task switch. It has about 85 cycles: it wakes clients by adding 1 to its event count (never `WAKE`), and sends a note to a group with `NOTE_QUEUE`. No stretch of code may hold IRQs off for more than 200 cycles (a character at 115200 is 320). A module of more than one bank owns no line.

Putting it in the ROM

A driver is a folder in `modules/` (`modules/NAME/*.s`), listed in `modules/rom.txt`; `node build.js` builds it with the rest. A test of it is a test module (`tests/mod`) that runs as init, with the driver in the ROM (`tests/tests.js`: a test's `modules`). `/proc` refuses a driver's memory and registers, and `NOTE` its notes: a driver's task is the system's.

Modules and programs

HYX2

Every executable is a HYX2 module: a 48-byte header (`sdk/asm/hyx2.inc`), then its code and data. The header says what it is (`HT_PROGRAM`, `HT_DRIVER`, `HT_LIBRARY`), where it loads (`$A000` in place, `$0800` in RAM), its data and BSS, its entries (`main`, or `init`, `serve`, `irq` and `stop`), its device letter, the banks it spans, and its name.

Macro	For
<code>HYX2_PROGRAM "name", main</code>	A program: <code>main</code> gets its arguments at <code>r0</code> ; returning is <code>EXITS 0</code>
<code>HYX2_DRIVER "name", init, serve, irq, stop, flags</code>	A driver (servers.md)
<code>HYX2_LIBRARY "name", "segment", "memory"</code>	A library module: code alone, called by other modules

In RAM, or in place

A program in a file (on a card, a RAM disk, `/pc`) is a RAM program: assembled with `-D HYX2_RAM` and linked by `sdk/asm/hyx2.cfg` (its header, code and data from `$0800`, its BSS after them). `SPAWN` gives the new task the file as its `fd 15`, and the task reads it in itself as it starts, clears its BSS, and runs it. `node build.js prog DIR` builds one from a folder of `.s` (or `.c`) files.

A module of the paged ROM runs in place: the task that runs it has the module's bank at `$A000` (its `$01`), and only its data is copied into the task's RAM (from `$0400`), its BSS cleared, as it starts. It's built by `modules/module.cfg` and listed in `modules/rom.txt`; `#m/NAME` is its file, and `#m/bin` lists the programs, bound at `/bin`. Running in place costs no load: `SPAWN` of a module takes about 50,000 cycles (14 ms).

Modules of several banks

A module bigger than a bank (16K) spans two to eight, one after another: `HYX2_DRIVER ..., 2` (or `HYX2_PROGRAM "name", main, 2`), its second bank's code in the segment `CODE2` (linked by `modules/module2.cfg`), the third's in `CODE3` ... It calls between its banks through trampolines in its RAM: `FAR2` (from the first into the second), `FAR1` (back), and `FARN` bank, routine from any to any, the caller's bank set again after. Such a module keeps its note handler in its RAM, as the kernel calls a handler with whichever bank is at `$A000`, and owns no `IRQ` line. `rc` and `play` (two banks each), `HydraFS` (the storage driver's second) and `hylang` (eight) are built so.

Libraries and XCALL

A library module is code other modules call, in their tasks, with their RAM: it has no data, BSS or entries of its own. A program finds a library's bank in the module directory (`MODINFO`: entry by entry, its `ME_NAME` and `ME_BANK`), and calls it with `XCALL`, the `X16`'s `jsr far: r15` the routine's address, `r14` its bank; `.A`, `.X`, `.Y`, the flags and `r0-r13` pass through both ways, and the caller's bank comes back after it.

```
lda    lib_bank                ; (MODINFO's ME_BANK, found as it started)
sta    r14
LDR     r15, LIB_ADD           ; The library's routine ($A030: its jump table)
lda     #3
ldx     #4
jsr     XCALL                  ; .A = 7
```

A library keeps a jump table at its start, after its header, so its routines' addresses don't move as it changes: `tests/mod/t_lib/t_lib.s` is one, and `t_xcall` calls it.

Debugging

The emulator (`sim/run.js`) traces a program's system calls by name (`--trace-calls`), stops at a label (`--break K_OPEN, --break rc:main`), watches an address (`--watch ADDR[:T]`), and with `-i` has a monitor (Ctrl-A b: step, registers, memory in any task's view, breaks and watches). The top of `sim/run.js` lists its options. On the Hydra, `/proc/N` is a task's state (regs, mem, fd), and its `ctl` stops, starts and steps it; `db`, the debugger, works through them: a program started stopped, stepped, run to breakpoints, with `ld65`'s symbols (the build's `.1b1` files, or as `-l's`) ([../using/tools.md](#)).

The screen: the Vera X

The Hydra-16's video card is the **Vera X**: the X16's VERA (an iCE40UP5K FPGA with 128K of video RAM, VGA out at 640x480, two layers of text, tiles or bitmaps, 128 sprites, a 256-colour palette, a 16-voice PSG and PCM) in slot 0. Its 32 registers are slot 0's I/O ports 2 and 3, \$FF20-\$FF3F, and its interrupt is IRQ line 2. The card is Joe Burks's VERA X, which runs the X16 community's gateway (v47 on: X16Community/vera-module), so the X16's documentation is this chip's: *The Commander X16 Programmer's Reference*, chapter 9 (the registers, VRAM, layers, sprites, the PSG and PCM) and chapter 10 (FX). The plan behind it is [../design/plans/VIDEO.md](#). Wiring the card to the board through a bus breakout card, till the carrier card exists: [../vera-wiring.md](#).

A driver, **vid** (modules/vid), owns the chip: it finds the card as the system starts, sets it up for the console and serves it as files. A program uses the screen three ways, from the easiest:

1. **As the console.** The console's windows are shown on the screen as well as the serial terminal, so anything a program prints, colours and cursor moves included (ANSI sequences: conio, HyForth's terminal words, hylang's screen.h1), shows there. Nothing to do.
2. **Through /dev/vid's files:** VRAM, the palette, the sprites and the font, read and written as files; the screen's modes as commands to ctl; a frame waited for by reading frame.
3. **Directly:** a program *claims* the chip and writes its registers itself, as an X16 program does, for as long as it holds the claim.

The files

#v, bound at /dev/vid by /rom/lib/namespace (with no card there's no #v, and /dev/vid is empty):

File	Read	Write
ctl	The state, a line each: vera 47.0.2 (the gateway's version), mode 80x60, cursor blink, border 0, bitmap off, claimed (and the claimer's task, and all)	Commands: mode 80x60, mode 80x30, mode 40x30; cursor blink, cursor on, cursor off; border N; bitmap 320 D, bitmap 640 D, bitmap off; claim, claim all, release; reset
term	The screen's characters, a line a row (its columns, then an LF)	Bytes shown as an ANSI terminal shows them (the console writes here)
vram	VRAM: the offset is the address, \$00000-\$1FFFF	VRAM, through the chip's data port
pal	The palette (VRAM \$1FA00: 256 entries of 2 bytes, \$GB then \$0R)	The palette
sprites	The sprites' attributes (VRAM \$1FC00: 128 of 8 bytes)	The attributes (sprite 0 is the console's cursor)
font	The console's font (VRAM \$1F000: 256 characters of 8 bytes, a byte a row)	A font: cat /lib/font/cp437 >/dev/vid/font
frame	Waits for the next frame (59.5 a second), then gives the frames counted, in decimal	
psg	The PSG's 64 registers (VRAM \$1F9C0: 16 voices of 4) as written here	Register/value pairs: the sound driver's (its channels 8-23); kept while the chip's claimed, and written as the claim ends
pcm		Samples into the PCM FIFO (below: "PCM")
pcmctl	The PCM's state: rate 22126, bits 8, mono, volume 15, claimed (and the task that has pcm)	rate HZ, bits 8, bits 16, mono, stereo, volume N (0-15), reset (the FIFO emptied), drain (waits till it's empty)

So a picture is a file copy away: a 320x240 picture of 8 bits a pixel, its bytes in a file, then

```
cat pic.bin >/dev/vid/vram
cat pic.pal >/dev/vid/pal
echo bitmap 320 8 >/dev/vid/ctl
```

shows it on layer 0, under the console's text (which is layer 1: its cells' background colour 0 lets layer 0 show through; `bitmap 320` makes the text 40x30, as the picture is shown 2x). `echo bitmap off >/dev/vid/ctl` takes it away. (`cp` won't write a device's file: it makes the file it copies to, and a device's files are there already.)

A game paces itself by `frame`: a read waits for the next VSYNC. In C, `fread` a line from it; in HyForth, `read-line`.

The console's terminal

The screen shows the console's windows as the serial terminal does: the window shown, with the keys (Ctrl-] and a digit shows another, repainted on both). `consctl` chooses where: `screen`, `serial` or both (every window's; it starts both), and reads with a line `terminal both`. With `screen` alone, output isn't paced by the serial line.

The terminal is 80x60 (`mode 80x30` and `mode 40x30` make the characters bigger), in 16 colours: the ANSI ones, 0-15 as conio numbers them. It takes CR, LF, BS, TAB, FF; ESC 7 and ESC 8, ESC D (index), ESC E (next line), ESC M (reverse index), ESC c; CSI A B C D E F G d H f (moves), J and K (0, 1, 2), m (0, 1 bold, shown bright, 22, 7 reverse, 27, 30-37, 39, 40-47, 49, 90-97, 100-107), r (the scrolling region, as a VT100's: CSI 2;23r, then an LF at row 23 scrolls rows 2-23 alone and ESC M at row 2 scrolls them down; CSI r the whole screen again), s and u, ?25h and ?25l. Others are taken and dropped. The whole screen scrolls by moving layer 1's VSCROLL (the map is a ring of 64 rows), so a scroll costs a row; a region scrolls by copying its rows in VRAM (some 1,500 cycles a row). The cursor is sprite 0, an underline, blinking (`cursor on` steadies it).

The screen has no keyboard yet: keys still come from the serial terminal. A PS/2 keyboard, through an input controller on IRQ line 3, is planned (VIDEO.md).

PCM

The VERA plays samples from a 4K FIFO at a rate of its own: `/dev/vid/pcm` is the FIFO, `/dev/vid/pcmctl` its settings. A program sets the format and rate, then writes the samples:

```
echo rate 11025 >/dev/vid/pcmctl; echo bits 8 >/dev/vid/pcmctl; echo mono >/dev/vid/pcmctl
cat drums.raw >/dev/vid/pcm; echo drain >/dev/vid/pcmctl
```

- **The samples are the VERA's:** signed, 8 or 16 bits (16 little-endian), stereo left first. (A WAV file's 8-bit samples are unsigned: `play` makes them signed.)
- **The rate** is the VERA's nearest: 48,828.125 Hz / 128 steps, 381 Hz each, up to 48,828 (`rate 22050` plays at 22,126, `rate 8000` at 8,011); `rate 0` stops it. `pcmctl` reads the rate it has.
- **A write** goes into the FIFO when it's below a quarter full, as much as it has room for, and the rest waits for the next frame (59.5 a second); a non-blocking fd gets E_AGAIN. So the FIFO holds 3K to 4K while a program keeps it fed: at 11 kHz, 8-bit mono, a third of a second.
- **pcm is one task's at a time:** another's open is E_BUSY till its last fd of it closes, and its `pcmctl` commands too (anyone's, while nobody has `pcm`).
- `drain` waits till the FIFO's empty (a program's end shouldn't cut its last sounds off); `reset` empties it at once.
- **How fast:** the driver puts some 22 cycles into each byte, and a card reads at about 12K a second, so from a card 8 bits mono to about 11 kHz is comfortable; from the RAM disk (`/ram`) to about 22 kHz. 16 bits stereo at 44.1 kHz (176K a second) is past a 3.58 MHz 65C02.
- A claim of the chip stops it: `pcm`'s writes wait, and the release sets its format, volume and rate again (its FIFO emptied).

`play` plays WAV files (8 or 16 bits, mono or stereo; PCM, not compressed or floating point) and a ZSM song's PCM instruments (their data read into RAM when it fits, about 20K, else read from the file as it plays).

Claiming the chip

The VERA has one set of address registers, so two tasks can't both write it. A program that wants the chip writes `claim` to `ctl` and keeps the file open: till it writes `release`, or closes its last file of `#v` (its end does), the chip is its own. It may write any register; the driver leaves the chip alone, and another task's commands and reads of the chip's files get `E_BUSY` (a second `claim` too). The console's output meanwhile waits in the driver (its last 1K), and is shown when the claim ends.

The VRAM a claimer may use without saying so is `$00000-$1AFFF` (108K: a 320x240 bitmap of 8 bits is 75K); the console's map is at `$1B000-$1EFFF`, its font at `$1F000-$1F7FF`, the cursor's image at `$1F800`. With `claim all` all of VRAM is the program's, and the console's map and font are made again at the release. The release sets the chip up for the console: the palette, the sprites (all off but the cursor), the layers, the scales, the interrupts. The PSG is the sound driver's (`/dev/snd`'s channels 8-23, through `/dev/vid/psg`): during a claim its writes are kept, not made, and the release writes them, so a song's voices pick up where they are; a claimer that wants the PSG for itself claims those channels from `/dev/sndctl` too (`claim 0 65535`).

```
LDR      r0, s_ctl                      ; "/dev/vid/ctl"
lda      #O_WRITE
jsr      OPEN
sta      ctl
LDR      r0, s_claim                    ; "claim"
LDR      r1, 5
lda      ctl
jsr      WRITE
; ... the chip is this program's: VERA_ADDR_L ... (hw.inc's names)
```

Interrupts. The driver still owns IRQ line 2 during a claim: its entry counts frames (so `frame` works), and clears the VSYNC, LINE and SPRCOL interrupts a claimer turns on (so they don't hold the line), and turns AFLOW off (the PCM FIFO's, a level that would hold the line: `pcm`'s writers wait for frames instead). A program that wants raster effects polls `VERA_ISR` or `SCANLINE`, or waits on `frame`; one that uses `frame` keeps VSYNC on in `VERA_IEN` (it's on as the claim starts). As on the X16, keep interrupt code on data port 1 and a program's on port 0, and never leave CTRL's DCSEL other than 0 long.

The registers are `include/hw.inc`'s, the X16's names at the Hydra's base: `VERA_BASE` `$FF20`, `VERA_ADDR_L`, `VERA_ADDR_M`, `VERA_ADDR_H`, `VERA_DATA0`, `VERA_DATA1`, `VERA_CTRL`, `VERA_IEN`, `VERA_ISR`, `VERA_IRQ_LINE_L`, `VERA_DC_VIDEO` ... `VERA_SPI_CTRL`, and `VERA_PSG_BASE`, `VERA_PALETTE_BASE`, `VERA_SPRITES_BASE`. X16 code ports with the base changed.

In the emulator

`node sim/run.js --vera` puts a Vera X in slot 0 (`sim/lib/vera.js`: the registers, VRAM, the layers, sprites and their collisions, the scan's timing and interrupts, the PCM FIFO, the PSG's registers; not FX). `--screen` prints the text layer after the report, `--frame-png FILE` saves the screen, and with `-i`, `--view` shows it live in a browser (<http://localhost:8016>) while the terminal stays the serial console; Ctrl-A v prints it, Ctrl-A p saves it. Its sound, the PSG's and the PCM's, is made with the YM2151's (`sim/lib/audio.js`) when it's asked for: `-i --sound` plays it in the browser (the same page as `--view`'s, its Sound button) and `--wav FILE` keeps it. Tests set machine: `{ vera: true }`; `m.vera.text()` is the screen's text, `m.vera.psg` the PSG's registers and `m.vera.psgOns` its voices' starts; with `vera: { pcmLog: true }`, `m.vera.pcmLog` is the bytes the FIFO took, and `m.vera.pcmUnderruns` counts its runs dry while it played; with `sound: true` too, `start(m)` can listen (`m.audio.on(fn)`: the sound test's).

The assembly SDK

Programs for the Hydra-16 in 65C02 assembly, with the cc65 tools (ca65, ld65). A program is a HYX2 file: a 48-byte header, then its code and data, which SPAWN reads into a task's RAM at \$0800 and starts. Put it on a card or a RAM disk, and rc runs it by its path, or by its name from . or /bin.

The SDK is this folder; node build.js also copies it, with the generated hydra.inc and the samples, to bin/sdk/asm, to take elsewhere.

File	What it is
hydra.inc	The system calls (their addresses in the jump table), the error codes and the constants. Made from spec/api.def by the build (obj/sdk/hydra.inc); never edit it
hyx2.inc	The header: HYX2_PROGRAM "name", main
hyx2.cfg	The link for a program in a file: header, code and data from \$0800, the BSS after them
macros.inc	CALL name (a system call), CHECK label (on to label if the call failed), LDR reg, value (a call register = a 16-bit value), MOVR to, from (one register = another), PRINT label or PRINT "text" (a string to fd 1); CR, LF, TAB
toollib.inc, toollib.s	What the system's tools share: flags, errors and exit statuses as Plan 9's, buffered output, input a file at a time, directories, paths, numbers (the comment at its top lists them)
srvlib.inc, srvlib.s	A file server's library (the system's drivers use it)
nslib.s	A task's default namespace, from the namespace file (Plan 9's newns)
samples/	hi (arguments, task, directory, environment), upper (a filter on toollib), tick (a note handler), counter (a server: a driver, a module that runs in place, on srvlib)

The calls are described in /rom/doc/api.md on the Hydra (the build's obj/gen/api.md), and the rules the system keeps in docs/conventions.md.

The Hydra assembles the same sources itself: as file.s (docs/using/tools.md, "The assembler"), with this folder's .inc and .s files in /lib/as, makes the same program ca65 and ld65 make with hyx2.cfg.

A program

```
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

HYX2_PROGRAM "hello", main

.code
main:                                ; r0: the arguments

    PRINT    "Hello"
    lda      #LF
    CALL     PUTC
    lda      #0                        ; (Returning: EXITS with code 0)
    rts
```

- main gets r0 pointing at its arguments: zero-terminated strings one after another, an empty one after the last (hello a b: "a", 0, "b", 0, 0). Its name is the header's (11 characters at most).
- Returning ends the task with code 0. EXITS ends it with a code (.A) and a message (r0, or 0): rc's \$status is the message if there is one, else the code (true is 0).

- A call is a `jsr` to its name (`CALL` name says the same). Arguments and results are `.A`, `.X`, `.Y` and the call registers `r0-r15` (`$02-$21`); a call may change all of them. `C = 0` is success; `C = 1` is failure, with the error in `.A` (`ERRSTR` gives its text: "not found"); `CHECK label` goes on to label on a failure.
- The program's own zero page is `$22-$7F` (`.zeropage`: no call touches it); its RAM is `$0800` up to its break (`BREAK` moves it), and pages above (`PAGES_ALLOC`), to `$7FFF`. It has its own RAM banks at `$8000-$9FFF` too (16 a RAM module: `BANKS`), and its stack.
- Its `fds` 0, 1 and 2 are its window's console, or what `rc` gave it (`<`, `>`, `|`); `putc`, `puts` and `getc` write and read them a byte or a string at a time; `open`, `read`, `write` and `close` do the rest. Names go through its namespace, as `rc`'s do.
- A note (`Ctrl-C` at its window: `NOTE_INTERRUPT`) ends it, unless it has a handler (`NOTIFY: tick`).
- Its environment (`ENV_GET`, `ENV_PUT`) is a copy of `rc`'s: `rc`'s variables, `$window`, `$path` ...

Building it

In this repository, a program in a folder of its own (its `.s` files):

```
node build.js prog path/to/hello          # path/to/hello/hello.hyx
```

With the `cc65` tools and `bin/sdk/asm` alone:

```
ca65 --cpu 65C02 -D HYX2_RAM -I sdk/asm -o hello.o hello.s
ld65 -C sdk/asm/hyx2.cfg -o hello.hyx hello.o
```

(`-D HYX2_RAM`: a program in a file. Without it, `hyx2.inc` makes a module of the paged ROM, run in place, linked with `module.cfg` (`module2.cfg` to `module8.cfg` for one of two to eight banks); those are the system's, built into the ROM with `modules/rom.txt`. A server is one: a driver, `HYX2_DRIVER`, which registers a device letter and answers its clients' requests through `srvlib`. The sample counter is one; the build makes it a module, and the tools test puts it in its ROM. A module may have library modules of its own, `HYX2_LIBRARY`, linked by its own config: `docs/conventions.md`.)

Running it

Copy it to a card, or to the RAM disk, and type its path (`/sd/0/hello`), or its name if it's in `.` or a `bin` (`$path` is (`.` `/bin`); a card's `/bin` is part of `/bin`). In the emulator, from `reborn/`:

```
node sim/tools/hydrafs.js mkfs card.img 8 MINE          # an empty card image, 8 MB
node sim/tools/hydrafs.js put card.img path/to/hello/hello.hyx hello
node sim/run.js -i --sd card.img                      # then, at the % prompt: /sd/0/hello
```

The samples are on the ROM disk: `/rom/sample/hi` Ann Bob, echo hi | `/rom/sample/upper`, `/rom/sample/tick` (`Ctrl-C` to stop it).

The C SDK

Programs for the Hydra-16 in C, with cc65 (cc65, ca65, ld65). A program is a HYP2 file, as an assembly one is (sdk/asm/README.md): a 48-byte header, then its code and data, which SPAWN reads into a task's RAM at \$0800 and starts. cc65's own target none does the compiling; the Hydra's part is the start-up, the link and the library under the standard one.

The SDK is this folder; node build.js builds the library and also copies the SDK, with the generated hydracalls.h, the library and the samples, to bin/sdk/c, to take elsewhere.

File	What it is
include/snd.h	The YM2151, and the Vera X's PSG (channels 8-23), through the sound driver: channels claimed, patches, notes, volumes, bends, drums, waveforms, raw registers
include/hydra.h	The Hydra's own calls (tasks and exit statuses, the namespace, the tick, RAM banks, any call by hy_call), and what cc65's headers leave to a target: setenv, fstat, isatty, conio's colours and keys
hydracalls.h	Every system call's address, error code and constant, each with HY_ before its name. Made from spec/ by the build (obj/sdk/c/hdracalls.h); never edit it. hydra.h includes it
hydra.cfg	The link: the header, code and data from \$0800, the BSS after them, the heap, and the C stack (2K) down from \$7F00
lib/hydra.lib	The library: cc65's none.lib, with the modules of lib/ in place of cc65's that a target gives (the build: obj/sdk/c/hydra.lib)
lib/	Its sources: crt0.s (the header, the start, exit), the files and stdio's buffers, the environment, system, signal, time and clock, conio, errors
samples/	hello (arguments), upper (a filter), code (exit statuses), keys (conio: the screen and raw keys), tones (sound: snd.h), jukebox (a song in the background: snd_play), ctest (the library's test); the multitasking demos (below): race, chorus, philo, prodcons, round

A program

```
#include <stdio.h>

int main (int argc, char* argv[])
{
    printf ("hello, %s\n", argc > 1 ? argv[1] : "world");
    return 0;
}
```

- argv[0] is its name: its file's (/rom/sample/c/hello's is hello); its arguments are rc's words.
- main's value, or exit's, is its exit code (0 is success); hy_exits ("why") ends it with a message and code 1, as Plan 9's exits does. rc's \$status is the message if there is one, else the code.
- Its fds 0, 1 and 2 are stdin, stdout and stderr: its window's console, or what rc gave it (<, >, |). stdio is buffered: a file's bytes are read and written 255 at a time, a console's output a line at a time (and before any read), stderr's at once; it all goes out at fflush, fclose and the program's end.
- Files: fopen, open, read, write, lseek, stat, fstat, remove, rename, mkdir, rmdir, chdir, getcwd, and opendir/readdir (hy_dirstat: an entry's whole stat record). Names go through its namespace, as rc's do; hy_bind, hy_mount and hy_unmount change it.
- A failed call returns -1 (or NULL) and sets errno (cc65's: ENOENT ...) and _oserror (the kernel's own code, HY_E_NOENT ...; _stroserror and hy_errstr give its text, the kernel's).
- Its environment is a copy of rc's variables: getenv, setenv, putenv, unsetenv.
- system ("cmd") runs a command line with rc (rc -c): its value is the command's exit code. hy_spawn starts a program in a task of its own, hy_wait waits for it and takes its code and message; hy_parent is the task that started this one.

- Semaphores, for tasks that share something: `hy_sem_new` (a count, or `HY_SEM_MUTEX`), `hy_sem_acquire` (it waits), `hy_sem_try`, `hy_sem_release`, `hy_sem_free`. Every task's, by number; the program's end frees those it made and gives back the mutexes it holds.
- Shared segments, memory several tasks see: `hy_seg_create` (banks of 8K: its number, this task attached), `hy_seg_attach` (another task, given the number), `hy_seg_map` (a bank of it at `HY_BANK_WINDOW`, \$8000), `hy_seg_detach`. The last task attached to go frees it.
- `signal` (`SIGINT`, `f`): Ctrl-C at its window (the interrupt note) runs `f`; `SIG_IGN` ignores it; the default ends the program.
- `conio` (`conio.h`) works the console as an ANSI terminal: `clrscr`, `gotoxy`, `textcolor`, `revers`, `cursor`; `cgetc` reads keys raw (no echo, each as it's typed, the cursor and function keys as one code each: `CH_*` in `hydra.h`) until the program ends.
- Sound (`snd.h`): claim the channels it uses (`snd_claim`), then patches, notes, volumes, bends and drums on them; they're given back as it ends. `snd_play` plays a song in the background (`play`, in a task of its own).
- `time` is the system's clock (`/dev/time`; `hy_time`, in seconds since 2000), `clock` the ticks since the program's start; `sleep` and `hy_sleep_ticks` let the other tasks run, and `hy_sleep_until` sleeps till a tick count (for steady timing: a time already passed returns at once).
- Its RAM: \$0800 to \$7F00, for the program, its BSS, the heap (`malloc`) and the C stack (2K); its own RAM banks at \$8000-\$9FFF (`hy_banks_alloc`, `hy_bank`). The 6502's stack is 256 bytes: deep recursion runs out of it. `cc65`'s runtime has the zero page from \$22 (26 bytes); the program's own goes after it, to \$7F.

Building it

In this repository, a program in a folder of its own (its `.c` files, and `.s` files if it has any):

```
node build.js prog path/to/hello          # path/to/hello/hello.hyx
```

With `cc65` and `bin/sdk/c` alone:

```
cc65 -t none --cpu 65C02 -O -I sdk/c/include hello.c
ca65 --cpu 65C02 hello.s
ld65 -C sdk/c/hydra.cfg -o hello.hyx hello.o sdk/c/lib/hydra.lib
```

Running it

As an assembly program is run (`sdk/asm/README.md`): copy it to a card or the RAM disk, and type its path, or its name if it's in `.` or a `bin`. The samples are on the ROM disk: `/rom/sample/c/hello you`, `echo hi` | `/rom/sample/c/upper`, `/rom/sample/c/keys`, `/rom/sample/c/tones`; `cd /ram`; `/rom/sample/c/ctest a 'b c'` runs the library's test.

The multitasking demos

Five samples show tasks working together. Each is one program that starts copies of itself as its workers (each in a task of its own, given what it needs as arguments: a shared segment's number; semaphores' numbers), draws what they do as they do it, and says how it went. They find themselves as `/bin/NAME`, `./NAME` or `/rom/sample/c/NAME`, so after `bind -a /rom/sample/c` `/bin` (at `rc`, or `HyForth`'s `bind`) they run by name. Each needs four or five tasks free besides its own; Ctrl-C ends them all, and the program says how far it got.

Sample	What it shows
<code>race</code> [<code>TASKS</code> [<code>ADDS</code>]]	Why shared memory needs a lock: four tasks add 1 to a counter in a shared segment, 500 times each (read it, work a little, write it back). With nothing to keep them apart, a task stopped between its read and its write undoes the others' adds, and many are lost; with a mutex around each read and write, none are
<code>chorus</code>	The console as a shared resource: four tasks print a line each a letter at a time. With nothing between them the letters tangle; with a mutex held for a whole line each line is whole; with a baton (a semaphore each, passed round: wait for yours, print, give the next one's) the lines come in turn

Sample	What it shows
<code>philo [-d] [-n MEALS] [N]</code>	The dining philosophers: five tasks, a fork (a mutex) between each two, a live table of who's thinking, hungry or eating and who holds which fork. Taking the lower-numbered fork first, they never deadlock; with <code>-d</code> each takes their left fork first, and they soon do: the program sees it and ends them
<code>prodcons [-n ITEMS]</code>	Producers and consumers: two tasks make items and two use them, through a ring of 8 slots in a shared segment, kept in step by two counting semaphores (free slots, filled ones) and a mutex. The ring fills while the producers are quicker and empties while the consumers are, each side waiting (using no CPU) when it must
<code>round [TIMES [TICKS]]</code>	"Row, Row, Row Your Boat" as a round: four tasks a voice each, on YM2151 channels 0-3, two bars apart. They meet at a barrier (semaphores), then each keeps its own time by the tick (<code>hy_sleep_until</code>), and the words light up as they're sung. In the emulator, <code>sim/run.js -i --sound</code> plays it in a browser

Each runs as a test too (`tests/tests.js`: `race`, `chorus`, `philo`, `prodcons`, `round`): no adds lost with the mutex, the lines whole and in turn, no fork ever in two hands and the deadlock seen, every item used once, every note in time.

The Hydra-16: hardware reference

This is the Hydra-16 main board (V1) as its schematic describes it (board/hydra-16.kicad_sch and its sheets), with the two companion cards in board/, and the Vera X video card that goes in slot 0. Reference designators (U25, J18, ...) are the schematic's. How HydraOS uses the hardware is [the guide's](#) and [the programmer's guide's](#); the registers' names are include/hw.inc's (made from this document).

To test a board, run the hardware test: `hwtest` at the shell, or a `T` typed during POST. It's the old system's, kept unchanged in paged ROM bank 1 ([its guide](#)), with the checksums of HydraOS's images.

Contents

1. [Overview](#)
2. [The CPU view: memory map](#)
3. [The pseudo-registers T, U, V, W](#)
4. [Task RAM and the bank registers \(\\$00, \\$01\)](#)
5. [The paged RAM window \(\\$8000-\\$9FFF\)](#)
6. [The paged ROM \(\\$A000-\\$DFFF\)](#)
7. [The BIOS ROM \(\\$E000-\\$FFFF\)](#)
8. [I/O space \(\\$FF00-\\$FFFF\)](#)
9. [Interrupts](#)
10. [Clocks](#)
11. [Reset, power and bus control](#)
12. [On-board devices](#): VIA, ACIA (serial), YM2151 (sound), SPI
13. [Expansion slots](#)
14. [Companion cards](#): the memory daughter card, the bus breakout card, the Vera X (slot 0)
15. [Connectors and jumpers](#)
16. [V1 errata](#)
17. [Parts by function](#)
18. [In the emulator](#)

Overview

The Hydra-16 is a W65C02S computer built for multitasking. Its address decoding gives each of **16 tasks** its own 32K of RAM, including zero page and the stack, and its own RAM and ROM bank selections. So switching tasks is a single write to one register, T.

CPU	WDC W65C02S (U1), 3.58 MHz as built (jumper: 0.89, 1.79, 3.58 or 7.16 MHz)
Task RAM	512K (U7, HM628512): 32K at \$0000-\$7FFF for each of 16 tasks
Paged RAM	Seen through an 8K window at \$8000-\$9FFF. Task banks : up to 15 modules of 2 MB on memory daughter cards; each task sees 16 banks of 8K per module. Shared banks : 2 MB on the board (U25, U27-U29), the same for all tasks: 256 banks of 8K, 16 at a time
Paged ROM	4 MB (U30-U37, 8 x SST39SF040), seen in 16K banks at \$A000-\$DFFF
BIOS ROM	U6, SST39SF010/020/040 (128-512K): 8K pages at \$E000-\$FFFF, selected by W
I/O	15 ports of 16 bytes at \$FF00-\$FFFE, plus the system port at \$FFF0-\$FFFF
Interrupts	16 prioritised IRQ lines, each with its own vector (a 16-entry vector RAM at \$FFFE)
Devices	65C22 VIA (timers, SPI, I2C, GPIO), 65C51 ACIA (RS-232 on a DE-9), YM2151 + YM3012 (stereo FM sound)

Expansion	6 slots (62-pin edge connectors, 8-bit bus), 8 SPI device headers, a GPIO/I2C header, memory card connectors
Video	The Vera X in slot 0 (the X16's VERA: VGA, two layers, sprites, a 16-voice PSG and PCM): below
Power	ATX-24 supply (+5 V, +3.3 V, +12 V, -12 V)
As built	3.58 MHz; three task RAM modules (48 banks of 8K for each task); the Vera X card is there, its carrier card still to be made

The CPU view: memory map

What the CPU sees at each address, and what selects it:

Addresses	What	Selected by
\$0000	RAM bank register (writes); task RAM (reads)	per task (see below)
\$0001	ROM bank register (writes); task RAM (reads)	per task
\$0002-\$7FFF	Task RAM (zero page, stack, program RAM)	T
\$8000-\$9FFF	Paged RAM: a task bank (\$00-\$EF) or a shared bank (\$F0-\$FF)	\$00; T for task banks, U for shared banks
\$A000-\$DFFF	Paged ROM, 16K banks	\$01
\$E000-\$FEFF	BIOS ROM	W
\$FF00-\$FFEF	I/O ports 0-14, 16 bytes each	
\$FFF0-\$FFF3	Pseudo-registers T, U, V, W	
\$FFF4-\$FFF9	Not decoded (reads float)	
\$FFFA-\$FFFD	BIOS ROM: the NMI and RESET vectors	W
\$FFFE-\$FFFF	IRQ/BRK vector RAM (16 entries)	the highest-priority active IRQ, or V

Every task has its own \$0000-\$7FFF and its own \$00/\$01 values. All tasks share the I/O space, the shared RAM banks, the ROMs, and the pseudo-registers themselves.

The pseudo-registers T, U, V, W

(Sheet FFF_Registers)

Four 8-bit registers in the system port. Each is a 74F573 latch, written with the CPU's write strobe, and read back through a 74F541 buffer. So a read gives the whole byte last written, all 8 bits.

Address	Register	Latch / read-back	Bits used by the hardware
\$FFF0	T, task	U48 / U49	T0-T3: which task's RAM (task RAM A15-A18; the modules' task lines)
\$FFF1	U, shared macro-page	U50 / U51	U0-U3: shared RAM A13-A16
\$FFF2	V, vector select	U52 / U53	V0-V3: the vector RAM index when no IRQ is active. V4-V7 aren't wired: the software keeps a software interrupt number there
\$FFF3	W, BIOS ROM page	U54 / U55	W0-W5: BIOS ROM A13-A18 (64 pages with an SST39SF040; 16 with the '010)

The latches have no reset, so they power up random; the reset code sets them. Nothing on the board resets them when the reset button is pressed. Bits 4-7 of T and U, and 6-7 of W, are stored but go nowhere.

Task RAM and the bank registers

Task RAM is one HM628512 (U7, 512K) on the main board. CPU A0-A14 go straight to it, CPU A15 is its chip enable (low for \$0000-\$7FFF), and T0-T3 drive its A15-A18. So each value of T selects a different 32K, and a task switch is a single write to \$FFF0. Zero page and the stack page are part of it, so every task has its own zero page and stack.

A **DS1747 in U7** gives the Hydra a clock that keeps the time while it's off. The DS1747 (the 5 V part; the DS1747W is 3.3 V) is a 512K battery-backed RAM with a clock, pin compatible with the HM628512, in a 600-mil module. Its clock registers are the chip's top 8 bytes, \$7FFF8-\$7FFFF, so they're **task F's** \$7FF8-\$7FFF (T0-T3 reach U7's A15-A18 through U48 and U21 in order). Task F is the console driver's, whose RAM stops below \$7FF8, so nothing else writes them; the kernel finds the chip at boot and sets the system's clock from it, and /dev/time (date) sets both ([the tools](#)). The rest of the chip is task RAM as before, kept while the power's off. A new DS1747 comes with its battery disconnected until it first gets power, and its oscillator may be stopped: setting the time starts it.

The bank registers \$00 and \$01 (*sheet* ZPMirrorRAM) are four 74LS219 (16 x 4-bit RAM) chips, addressed by T0-T3:

Register	Chips	Drives
\$00: RAM bank	IC1 (bits 0-3), IC2 (bits 4-7)	RAMB0-7: which bank is at \$8000-\$9FFF
\$01: ROM bank	IC3 (bits 0-3), IC4 (bits 4-7)	ROMB0-7: which paged ROM bank is at \$A000-\$DFFF

- A write to \$0000 or \$0001 stores the byte in the current task's entry. The address decoding (U8-U10 on sheet AddressDecode) spots a write to \$000x, and U16 picks the register with A0.
- The write also goes to task RAM as usual, and a **read** of \$00/\$01 comes from task RAM: the registers themselves are write-only, and task RAM mirrors them.
- The outputs are always enabled, so the bank lines follow T at once. A task switch selects the new task's RAM bank and ROM bank without further writes.
- On V1, bits 2 and 3 of both registers are crossed on their way to the bank lines (see [V1 errata](#)).

The paged RAM window

(\$8000-\$9FFF; *sheets* ZPMirrorRAM, SharedMem, AddressDecode)

\$8000-\$9FFF shows one 8K bank. The **bank ID** is the current task's \$00 value:

Bank ID	What
\$m0-\$mF (m = \$0-\$E)	Task RAM module m: 16 banks for each task (the module's chips also get T0-T3)
\$F0-\$FF	Shared RAM: 16 banks, the same for every task, from shared macro-page U

- **Module select.** The bank ID's upper nibble (RAMB4-7) goes to a 74LS154 (U23), qualified by the window's select (nBRA_S, \$8000-\$9FFF in PHI2). Its outputs nBRC0-nBRC14 go to the memory card connector (J1), one per module; nBRC15 selects the shared RAM on the board.
- **Memory card connector.** The lower nibble and T are buffered to it by U21 (74F541), as RAMB_M0-3 and T_M0-3, along with the read/write strobes (RWB_M, !RWB_M). J2 carries D0-D7, A0-A12 and +5 V.
- **Task RAM modules** are [memory daughter cards](#): 4 x HM628512 per card, set to a module number with a jumper. In a module:
 - T_M0-3 go to the chips' A13-A16, so each task has its own 16 banks.
 - RAMB_M0-1 go to A17-A18.
 - RAMB_M2-3 pick the chip.
 - A module is 2 MB: 16 tasks x 16 banks x 8K.
- **Shared RAM** is four HM628512 on the main board: U25, U27, U28 and U29.
 - RAMB_M2-3 pick the chip, through a 74F139 (U26) enabled by nBRC15.
 - RAMB_M0-1 go to the chips' A17-A18, and U0-U3 to A13-A16.

- So the 16 bank IDs \$F0-\$FF are seen in 16 macro-pages (U = 0-F): 256 shared banks of 8K, 2 MB.
- The software numbers them as **shared bank ID** = U << 4 | (bank & \$0F).

Bank IDs (\$00)	Shared RAM chip (V1)	Shared bank IDs
\$F0-\$F3	U25	\$x0-\$x3 for each U = x
\$F4-\$F7	U28 (V1: bits 2/3 crossed)	\$x4-\$x7
\$F8-\$FB	U27 (V1)	\$x8-\$xB
\$FC-\$FF	U29	\$xC-\$xF

A bank with no chip behind it (a missing module) reads whatever floats on the bus; POST finds the modules that are there (RAM modules: 03 at boot), and the kernel gives out banks only on those.

HM628512 pins, for tracing a bad line (the POST prints bad lines by number):

Signal	Pin	Signal	Pin
A0-A7	12, 11, 10, 9, 8, 7, 6, 5	D0-D2	13, 14, 15
A8, A9, A10, A11, A12	27, 26, 23, 25, 4	D3-D7	17, 18, 19, 20, 21
A13-A16 (U0-U3 on shared RAM)	28, 3, 31, 2	~CE, ~OE, ~WE	22, 24, 29
A17, A18 (RAMB_M0, RAMB_M1)	30, 1	VCC, GND	32, 16

The paged ROM

(\$A000-\$DFFF; *sheets* BankedROM, ZPMirrorRAM)

Eight SST39SF040 (512K each, U30-U37) give 4 MB, seen as 256 banks of 16K. The bank is the current task's \$01 value:

- ROMB0-4 go to the chips' A14-A18 (32 banks per chip).
- ROMB5-7 pick the chip through a 74F138 (U24), qualified by the window's select (nBRO_S: \$A000-\$DFFF in PHI2):

\$01 bits 5-7	Chip	Banks
0	U31	\$00-\$1F
1	U32	\$20-\$3F
2	U34	\$40-\$5F
3	U36	\$60-\$7F
4	U30	\$80-\$9F
5	U33	\$A0-\$BF
6	U35	\$C0-\$DF
7	U37	\$E0-\$FF

- **The halves are swapped.** CPU A13 goes to the chips' A13 unchanged, but in the window \$A000-\$BFFF has A13 = 1 and \$C000-\$DFFF has A13 = 0. So CPU \$A000 reads chip offset \$2000 of the bank, and \$C000 reads offset \$0000. HydraOS's build writes the chips' view (tools/romimg.js), a 512K image for each chip it fills: bin/prom0.bin for U31 (chip select 0), prom1.bin for U32, prom2.bin for U34, prom3.bin for U36; burn each whole, at offset 0.
- **V1: bits 2 and 3, and 6 and 7, trade places.** On the V1 board ROMB2/ROMB3 and ROMB6/ROMB7 are swapped on their way to the chips (as are the RAM bank bits; the schematic shows the board as built), so the bank the CPU selects as b is the chips' bank swap(b). Banks whose two bits match (\$00-\$03, \$0C-\$0F, ...) aren't affected. The build writes the image in the chips' order (tools/romimg.js), and the emulator reads it that way (sim/lib/machine.js).
- nBROMD (a slot pin, pulled up) disables the whole paged ROM when a card pulls it low, so the card can answer \$A000-\$DFFF itself.
- The chips' ~OE is the inverted R/W; there's no write path in circuit (program the chips in a programmer).

The BIOS ROM

(U6, root sheet)

An SST39SF0x0 in a 32-pin socket: the '010 (128K, 16 pages), '020 (256K, 32 pages) or '040 (512K, 64 pages).

- CPU A0-A12 go to the chip's A0-A12, and w0-w5 to its A13-A18: each w value selects an 8K page.
- It's selected for \$E000-\$FFFF and for \$FFFA-\$FFFD (the NMI and RESET vectors). I/O space and the vector RAM take the rest of \$FF00-\$FFFF.
- HydraOS's build makes bin/bios.bin, 128K: 16 pages, for the '010.

Changing w changes the code being run. The next instruction is fetched from the new page, at the same address. The software handles this by keeping identical code at the same address on every page: the COMMON block at \$FD00 (the IRQ entry and exit, the kernel's far call), and the reset stub at \$E000 (kernel/bios.cfg; [the kernel's pages](#)). Because w isn't reset, **every page must start with the reset code**: the RESET vector on every page points to \$E000, which sets w to 0.

I/O space

(Sheet AddressDecode)

\$FF00-\$FFFF is decoded when A8-A15 are all 1 (U14, a 74F30), qualified by PHI2 (nIO_S). A 74LS154 (U19) splits it by A4-A7 into 16 ports of 16 bytes, nIOP0_S-nIOP15_S. Ports 0-14 are for devices; port 15 is the system port.

Port	Addresses	Use
0	\$FF00-\$FF0F	VIA (65C22, U2)
1	\$FF10-\$FF1F	ACIA (65C51, U3): 4 registers, repeated every 4 bytes
2	\$FF20-\$FF2F	Slot 0, select A (intended for video)
3	\$FF30-\$FF3F	Slot 0, select B (intended for video)
4	\$FF40-\$FF4F	YM2151 (U38): \$FF40 address/status, \$FF41 data (repeated)
5-9	\$FF50-\$FF9F	Slots 1-5, select A
10-14	\$FFA0-\$FFEF	Slots 1-5, select B
15	\$FFF0-\$FFFF	System port (below)

The system port is decoded by a 74F138 (U20) on A0-A2, enabled by nIOP15_S and by A3 differing from A2 (U12):

Address	Select	Use
\$FFF0-\$FFF3	nIOB0-3	T, U, V, W
\$FFF4-\$FFF9	none	Not decoded (reads float; reserved)
\$FFFA-\$FFFD	(BIOS ROM)	NMI and RESET vectors, from the current BIOS ROM page
\$FFFE, \$FFFF	nIOB14, nIOB15	The vector RAM, low and high bytes

Slot port selects (nIOA_S, nIOB_S on each slot) and each slot's two IRQ lines follow the same numbering: see [Expansion slots](#). A card can also decode its own addresses from the whole I/O space with nIO_S and A0-A7.

Interrupts

(Sheet IRQ_Priority_Encoder)

Sixteen active-low IRQ lines, nIRQ0-nIRQ15, each pulled up (RN2, RN3). Line 0 has the highest priority.

Line	Source	Line	Source
0	VIA	8	Slot 4, A

Line	Source	Line	Source
1	ACIA	9	Slot 5, A
2	Slot 0, A	10	Slot 1, B
3	Slot 0, B	11	Slot 2, B
4	YM2151	12	Slot 3, B
5	Slot 1, A	13	Slot 4, B
6	Slot 2, A	14	Slot 5, B
7	Slot 3, A	15	Not connected: used for software interrupts

How it works:

- Two cascaded 74LS148 priority encoders (U17 for lines 0-7, U18 for 8-15) find the highest-priority active line.
- Any active line asserts the CPU's $\sim$ IRQ (U44, then U22). The IRQ input is level-triggered: a device holds its line until the handler clears it.
- A 74F157 (U45) forms a 4-bit index $q0-q3$:
 - while any IRQ line is active, the index is the encoder's output;
 - otherwise it's $v0-v3$.
- The index addresses the **vector RAM**, four 74LS219 (IC5-IC8), a 16-entry table of 16-bit vectors. The CPU reads its IRQ/BRK vector from $\$FFFE/\$FFFF$, so it gets the entry for the active line, and each line has its own handler.

The index is the line number XOR 7. The '148s encode active-low inputs with the highest-priority input as 7, so line n gives index $n \wedge 7$: line 0 is entry 7, line 7 entry 0, line 8 entry 15, line 15 entry 8. `include/hw.inc`'s `IRQ_INDEX(line)` is `line ^ 7` for this reason.

Writing vectors. A write to $\$FFFE / \$FFFF$ stores the low / high byte of entry q . That's $v0-v3$ when no IRQ line is active. So set v to the entry wanted, then write the vector, with no IRQ pending. HydraOS's boot does this once, with interrupts off (`kernel/reset.s`): every line's entry is the kernel's one IRQ path.

BRK. BRK also reads $\$FFFE/\$FFFF$. With no IRQ line active, that's entry $v0-v3$. HydraOS sets v to line 15's entry (`IRQ_INDEX(15)`: line 15 has no hardware) as it starts and leaves it, so a BRK comes to the kernel by that vector: a program's stray BRK is a note to it, and the debugger's breakpoints are BRKs.

NMI (`NMIB`, pulled up, on every slot) uses the ROM's $\$FFFA$ vector. Nothing on the board drives it.

Clocks

(Sheet Clocks)

A 14.31818 MHz crystal (Y1) with a 74ACT14 (U39) oscillator, divided by a 74F191 counter (U40):

Clock	Frequency	Goes to
HS_CLK	14.318 MHz	Slots (e.g. for video)
Q0	7.159 MHz	CPU clock jumper J8
SND_CLK (Q1)	3.580 MHz	YM2151 master clock; slots; CPU clock jumper J7
SER_CLK (Q2)	1.790 MHz	ACIA clock (XTAL1); slots; CPU clock jumper J6
Q3	0.895 MHz	CPU clock jumper J5

The CPU clock (`PHI2`) is whichever of J5-J8 is fitted: fit exactly one. As built, it's **J7, 3.58 MHz**. The CPU's `PHI0` input is `PHI2` gated by `DMAB` (U11), so a card asserting `DMAB` stops the CPU clock (the W65C02S is fully static). `PHI1` (inverted `PHI2`) qualifies the address decoding.

HydraOS's timing (the tick, the serial port's pacing, sound) is built for one clock: `node build.js --clock 2` builds for 7.16 MHz (J8), the default for 3.58 MHz. At 7.16 MHz the YM2151 is too slow for the bus: there are no wait states on V1, so don't use the sound chip at that speed. The 0.89 and 1.79 MHz settings aren't supported.

Reset, power and bus control

Power comes from an ATX-24 connector (J11):

- +5 V for the logic, +3.3 V, and ± 12 V for the audio op-amps and the slots.
- +5 V standby is brought out as +5VA.
- The slots also have a -5 V pin, which the board doesn't supply: ATX no longer has -5 V.

Front panel (J10, 2x6):

Pin	Signal
7	RESB: a push button from here to ground (pins 5, 6, 9 or 10) resets the machine
8	+5 V
11	ATX PS_ON#: connect to ground (a latching switch) to turn the supply on
12	Power LED: from the supply's PWR_OK, through R2 (470 Ω)
5, 6, 9, 10	Ground
1-4	Not connected

Reset (RESB):

- A DM1813 supervisor (U43) holds reset low at power-up and during brown-outs.
- The front panel button and the slots (open-collector) can pull it low too.
- It resets the CPU, VIA, ACIA, YM2151 and YM3012. It doesn't reset the pseudo-registers, the bank registers or RAM.

Bus control:

- RDY: pulled up through jumper **J4**, which must be fitted. The W65C02S drives RDY low itself during WAI, so RDY must never be driven high directly.
- BE (bus enable): pulled up, and on the slots.
- **DMA**: a card pulls DMAB low to take the bus.
 - It stops the CPU clock, and turns off the address buffers (U58, U60, 74F541), the data transceiver (U59, 74F245) and the R/W buffer (U61, 74F125).
 - The card then drives the address and data buses itself.
- SYNC from the CPU is on the slots, for single-stepping or debugging hardware.
- MLB, VPB are unused. SOB is tied high.

On-board devices

VIA (65C22, U2): port 0, IRQ line 0

(Registers at \$FF00-\$FF0F.)

Port A is general purpose I/O, on header **J27** (2x6), with ESD protection (J28, SP720):

J27 pin	Signal	J27 pin	Signal
1	GND	2	+5 V
3	PA0: I2C_SCL	4	PA1: I2C_SDA
5	PA2	6	PA3

J27 pin	Signal	J27 pin	Signal
7	PA4	8	PA5
9	PA6	10	PA7
11	CA1	12	CA2

- PA0/PA1 are the I2C bus (bit-banged; the SDA and SCL pull-ups are in RN1), which also goes to every slot. HydraOS's GPIO driver serves it: `/dev/i2c` (its `ctl`, and a file each device).
- **The pins, CA1 and CA2 are files:** `/dev/gpio` (0-7, port, `ctl`, `ca1`: [the devices](#)); CA1 can interrupt (IRQ line 0, `/dev/gpio/ca1`).
- **Port B is the SPI bus** (below).
- **Timer 1** is the scheduler's tick: free-running, 200 interrupts a second.
- **Timer 2** paces the serial port's sending (the console driver's, at every rate, on either ACIA).
- CB1/CB2 aren't brought out.

ACIA (65C51, U3): port 1, IRQ line 1

(Registers at \$FF10-\$FF13.)

Chip and clock:

- The board has a Rockwell R65C51 socket. A WDC W65C51N works if HydraOS is built for it (`node build.js --acia wdc`): its transmitter status and interrupt don't work, and the console driver paces sending with VIA timer 2 on either chip.
- **The ACIA's clock is `SER_CLK`, 1.790 MHz**, not the 1.8432 MHz its baud rate table is made for. So every rate is 2.9% slow (9600 gives about 9,320 baud), which terminals and USB serial adapters accept.
- DCD and DSR are tied active.

Serial settings: 9600 baud, 8 data bits, no parity, 1 stop bit at boot, with RTS/CTS. `/dev/serctl` changes the rate: `b300`, `b600`, `b1200`, `b2400`, `b4800`, `b9600`, `b19200`, `b115200` (the ACIA clock / 16): `echo b115200 >/dev/serctl`.

Sending is paced. Sent back to back at 115200, long output loses and garbles characters on the board (the old system found it: a WOZMON dump): the rate is 2.9% slow and the MAX232 is near its limit, so the receiver has little margin. A second stop bit helps but isn't enough. So HydraOS's console driver sends each byte from VIA timer 2, not the ACIA's TDRE interrupt: a character's time and two idle bits at 115200, one at the other rates.

The DE-9 connector (J3, male) is driven by a MAX232 (U5), and wired like a modem (DCE):

DE-9 pin	Signal	Direction
2	Hydra transmits (ACIA TxD)	out
3	Hydra receives (ACIA RxD)	in
5	Ground	
7	ACIA ~CTS (from the terminal's RTS)	in
8	ACIA ~RTS (to the terminal's CTS)	out

Cable: a PC or USB serial adapter connects with a **straight-through** cable with two female ends, not a null-modem cable. The ACIA only transmits while its ~CTS is asserted, so the cable must carry pin 7. Terminal programs assert RTS by default.

YM2151 sound (U38): port 4, IRQ line 4

(Sheet Sound.)

The chip:

- A Yamaha YM2151 (OPM, 8 FM channels), clocked by `SND_CLK` (3.58 MHz).
- `$FF40` selects a register (and reads the status: bit 7 = busy), and `$FF41` writes it.
- After a data write the chip is busy for 64 of its clocks (about 18 μ s); writes while it's busy are lost.

- **CT1/CT2**, its two general-purpose outputs, are on header J9.
- **Timer B on the board**: re-armed every song tick (its flag reset through \$14, which on the board seems to restart it), it didn't keep its period: songs timed by it ran up to twice as fast, by a different amount each run. So the song player times songs by the system's tick (the VIA's timer 1) instead; the hardware test's timer checks (one period from a clean start) pass.

The audio path:

- A YM3012 DAC (U41) converts its serial output, buffered and filtered by a TL074 (U42).
- A mixer (IC9, LF353, ±12 V) adds, for each channel:
 - the YM3012's output;
 - each slot's audio lines (SND_CL0-5, SND_CR0-5), so a sound card in a slot is mixed in;
 - a line input on header J29.
- **Output**: stereo, on the 3.5 mm jack J26. The schematic has the tip on the right channel and the ring on the left, the reverse of the usual convention (see [V1 errata](#)).

SPI bus (VIA port B)

SPI is bit-banged on the VIA's port B (the VIA's shift register uses CB1/CB2, which aren't brought out, so it can't drive these lines); a 74F138 (U4) decodes the device select:

Port B bit	Signal
PB0	SPI_CLK (SCLK)
PB1	nSPI_CS: low selects the device given by PB3-PB6
PB2	SPI_MOSI
PB3-PB5	SPI_A0-A2: device 0-7
PB6	SPI_A3: 0 = the board's devices 0-7 (U4); 1 = devices 8-15, for cards to decode (the slots carry nSPI_CS and SPI_A0-A3)
PB7	SPI_MISO

The eight SPI device headers are J18-J25, for devices 0-7. Each is a 1x6 header:

Pin	Signal
1	~CS
2	SCLK
3	MOSI
4	MISO
5	+5 V
6	GND

HydraOS's storage driver runs SPI in mode 0 for SD cards, and serves an SD card on any device (`/dev/sd/N`, N the device in hex; its file system at `/sd/N`); device 0 (J18) is the usual place for an SD card adapter. Any device is a file too, `/dev/spi/N` (a write and a read are a transaction; modes 0 and 3: [the devices](#)). The headers supply +5 V, so the adapter must regulate and level-shift to 3.3 V for the card (common SD card modules do).

Expansion slots

(Sheet Connectors: J12-J17, "Hydra bus 8-bit", 62-pin card edge.)

Six slots, all carrying the same bus except for each slot's two port selects, two IRQ lines and its audio pair:

Slot	Connector	nIOA_S (pin 51)	nIOB_S (pin 49)	IRQA (pin 55)	IRQB (pin 53)	Audio (SND_CL/CR)
0	J12	port 2 \$FF20	port 3 \$FF30	line 2	line 3	pair 0
1	J13	port 5 \$FF50	port 10 \$FFA0	line 5	line 10	pair 1

Slot	Connector	nIOA_S (pin 51)	nIOB_S (pin 49)	IRQA (pin 55)	IRQB (pin 53)	Audio (SND_CL/CR)
2	J14	port 6 \$FF60	port 11 \$FFB0	line 6	line 11	pair 2
3	J15	port 7 \$FF70	port 12 \$FFC0	line 7	line 12	pair 3
4	J16	port 8 \$FF80	port 13 \$FFD0	line 8	line 13	pair 4
5	J17	port 9 \$FF90	port 14 \$FFE0	line 9	line 14	pair 5

Slot 0's A lines (port 2, IRQ 2) have the highest priority of the slots; the B lines of slots 1-5 the lowest.

Pinout (the same on every slot; ~ = active low):

Pin	Signal	Pin	Signal
1	GND	2	~nBROMD (disable the paged ROM)
3	~RESB	4	D7
5	+5 V	6	D6
7	~NMIB	8	D5
9	-5 V (not supplied)	10	D4
11	BE	12	D3
13	-12 V	14	D2
15	SER_CLK (1.79 MHz)	16	D1
17	+12 V	18	D0
19	GND	20	RDY
21	R/~W	22	SND_CLK (3.58 MHz)
23	~DMAB	24	SND_CR (audio in, right)
25	~nIO_S (any I/O port)	26	SND_CL (audio in, left)
27	SYNC	28	I2C_SDA
29	SPI_SCLK	30	I2C_SCL
31	SPI_MOSI	32	A15
33	SPI_MISO	34	A14
35	~nSPI_CS	36	A13
37	PHI1	38	A12
39	PHI2	40	A11
41	SPI_A3	42	A10
43	SPI_A2	44	A9
45	SPI_A1	46	A8
47	SPI_A0	48	A7
49	~nIOB_S	50	A6
51	~nIOA_S	52	A5
53	~IRQB	54	A4
55	~IRQA	56	A3
57	+5 V	58	A2
59	HS_CLK (14.318 MHz)	60	A1
61	GND	62	A0

A card's IRQ outputs should be open-collector: the board pulls each line up. The address and data buses are buffered on the board; a DMA card drives them after asserting ~DMAB.

Companion cards

Memory daughter card

(board/MemoryDaughterCard: *through-hole and surface-mount versions*.)

One task RAM module: 4 x HM628512, 2 MB.

- **Connectors:** it plugs into the main board's memory connectors J1 (1x32) and J2 (1x24).
- **Module number:** a 2x15 jumper block (J3) connects the card's select to one of `nBRC0-nBRC14`. That's its module number `m`, so it answers bank IDs `$m0-$mF`.
- **Addressing:**
 - a 74LS139 (U1) picks one of the four chips with `RAMB_M2-3`;
 - `RAMB_M0-1` go to the chips' A17-A18, and `T_M0-3` to A13-A16;
 - A0-A12 come from J2.

Main board J1 (memory card connector):

Pins	Signals
1-3	A13, A14, A15 (the card doesn't use them)
5-8	<code>T_M0-3</code>
9-12	<code>RAMB_M0-3</code>
13, 14	<code>RWB_M</code> , <code>!RWB_M</code>
15-17	GND
18-32	<code>nBRC0_S-nBRC14_S</code> (module selects)

J2: D0-D7 (pins 1-8), +5 V (9-11), A0-A12 (12-24).

POST finds the installed modules and tests each one (`RAM modules: 03`), and the kernel gives out banks only on modules that are present.

Bus breakout card

(board/HydraBusBreakoutCard.) A slot card that brings every bus signal out to headers, for a logic analyzer, a scope or prototyping:

Header	Signals
J2	Data bus
J3	Address bus
J4	SPI (A0-A3, <code>~CS</code> , MISO, MOSI, SCLK)
J5	NMI, <code>nIO_S</code> , the slot's port selects and IRQs
J6	Clocks
J7	Bus control (<code>nBROMD</code> , BE, RDY, R/W, SYNC)
J8	Reset
J9	Audio
J10	I2C
J11	Power

Two of its labels are wrong: the card's connector symbol and the main board's slot disagree, and the slot carries what the main board's says. **J11 pin 3, "+5VA", is -5 V** (slot pin 9), and **J7 pin 5, "!RWB", is /DMAB** (slot pin 23), not an inverted R/W. Wiring the Vera X through it: [vera-wiring.md](#).

The Vera X (slot 0)

(Not in board/: the plan is [design/plans/VIDEO.md](#).) The Hydra's video card: Joe Burks's VERA X, the Commander X16's VERA (an iCE40UP5K FPGA with 128K of video RAM inside: VGA at 640x480, two layers of text, tiles or bitmaps, 128 sprites, a 256-colour palette, a 16-voice PSG and a PCM FIFO, an SD card's SPI controller), running the X16 community's gateway (v47 on, which has the version register, DCSEL 63). The X16's documentation is its own: *The Commander X16 Programmer's Reference*, chapters 9 and 10.

Registers	The VERA's 32, at slot 0's ports 2 and 3, \$FF20-\$FF3F (the card answers either select): VERA_* in include/hw.inc
Interrupt	Its IRQ# on slot 0's IRQ A, line 2 : the highest priority after the VIA and the ACIA
Audio	Its DAC's left and right into slot 0's audio pair (SND_CL0, SND_CR0), mixed with the YM2151's on the board
Reset	RESB to its RES#: a reset reloads the FPGA from its flash, and it doesn't answer till that's done (vid looks for it for 0.3 s; the emulator takes 0.1 s)
IRQ B, line 3	Kept for the card's input controller (a PS/2 keyboard and mouse, pads), to talk over the slot's I2C

As built (October 2026): the user's VERA X 6.1, the module alone; its **carrier card** is still to be made. That's VIDEO.md's option A: a small slot card with a socket for the module and a little glue logic, CS# from either port select, RD# and WR# from PHI2 and R/W (the VERA latches a write as its strobe ends, so the strobe must end with PHI2, while the CPU still drives the data), IRQ# through an open-collector buffer; the audio through a divider to the mixer's level. The input controller comes after it. Till the carrier exists, the card can be wired through a bus breakout card in slot 0, with two glue chips ([vera-wiring.md](#)); the emulator's Vera X (sim/run.js --vera) is where the software's been run.

The software: vid, its driver, finds the card as the system starts (by the version register; v0.9's by ADDR0 read back), shows the console on its screen, and serves it as /dev/vid; the sound driver's channels 8-23 are its PSG's voices, and /dev/vid/pcm its PCM ([programming/video.md](#)).

Connectors and jumpers

Ref	What	Notes
J1, J2	Memory daughter card	Task RAM modules
J3	DE-9 male: serial console	DCE wiring; straight-through F-F cable
J4	RDY pull-up jumper	Fit
J5-J8	CPU clock jumpers: 0.89 / 1.79 / 3.58 / 7.16 MHz	Fit one: J7 (3.58 MHz) for the standard ROM
J9	YM2151 CT1/CT2 outputs	
J10	Front panel	Reset, power switch, power LED
J11	ATX-24 power	
J12-J17	Expansion slots 0-5	
J18-J25	SPI devices 0-7	SD card adapter on J18 (device 0)
J26	Audio out, 3.5 mm stereo	
J27	VIA port A: GPIO, I2C, CA1/CA2	
J28	SP720 ESD protection for J27 and the SPI lines	(a part, not a connector)
J29	Line input to the audio mixer	

V1 errata

- **Bank register bits 2 and 3 are crossed.**
 - In the bank registers (sheet ZPMirrorRAM), data bit 2 drives RAMB3, and bit 3 drives RAMB2: IC1 for the RAM bank, IC3 for the ROM bank.

- So a bank ID's bits 2 and 3 trade places before they reach the hardware. For example, shared bank IDs \$F4-\$F7 are on U28 and \$F8-\$FB on U27, module 4 and module 8 trade places, and so do paged ROM banks \$04-\$07 and \$08-\$0B.
- IDs whose bits 2 and 3 are equal aren't affected, and the software never needs to care: an ID always reaches the same memory. It matters when you map a bank ID to a chip, for example to act on a POST report.
- **Bits 6 and 7** are crossed too, on the V1 board as built (RAMB6/RAMB7, ROMB6/ROMB7). HydraOS's build (`tools/romimg.js`) and the emulator (`sim/lib/machine.js`) assume both swaps for the paged ROM.
- **No wait states.** RDY only has a pull-up, so slow devices can't stretch a bus cycle. This is why the YM2151 can't be used above 3.58 MHz. Board V2 is planned to have programmable RDY wait states (see [design/plans/IDEAS.md](#)).
- **Audio jack channels.** J26 has the right channel on the tip and the left on the ring, per the schematic; the usual convention is the reverse, so left and right may come out swapped.
- **ACIA clock.** The ACIA runs from 1.790 MHz instead of 1.8432 MHz, so its baud rates are 2.9% slow (see [ACIA](#)).

Parts by function

Function	Parts
CPU	U1 W65C02S
Bus buffers	U58, U60 74F541 (address); U59 74F245 (data); U61 74F125 (R/W)
Address decoding	U8, U9 74LS260; U10 74F20; U11 74F08; U12 74F86; U13, U16 74F32; U14 74F30; U15 74F00; U19 74LS154 (I/O ports); U20 74F138 (system port); U21 74F541 (memory card buffer); U22 74F240
Pseudo-registers	U48, U50, U52, U54 74F573; U49, U51, U53, U55 74F541; U56 74F32; U57 74F02
Bank registers	IC1-IC4 74LS219; U23 74LS154 (module select); U24 74F138 (paged ROM chip select)
Interrupts	U17, U18 74LS148; U44 74F00; U45 74F157; IC5-IC8 74LS219 (vector RAM); RN2, RN3 pull-ups
Task RAM	U7 HM628512
Shared RAM	U25, U27, U28, U29 HM628512; U26 74F139
Paged ROM	U30-U37 SST39SF040
BIOS ROM	U6 SST39SF010/020/040
Clocks	Y1 14.31818 MHz; U39 74ACT14; U40 74F191
Reset	U43 DM1813
VIA, SPI	U2 R65C22; U4 74F138 (SPI device select)
Serial	U3 R65C51; U5 MAX232
Sound	U38 YM2151; U41 YM3012; U42 TL074; IC9 LF353

In the emulator

`sim/lib/machine.js` is this board, cycle by cycle, for `sim/run.js` and the tests: the W65C02S; T, U, V and W, random at power-up as the latches are; each task's \$00/\$01 and RAM; the RAM window's task banks (only the modules installed: `--modules N`, 2 by default, 3 as built; the rest float) and the shared banks; the paged ROM with its halves swapped and the V1 board's bank bits; the BIOS ROM's pages; the I/O ports and the system port; the interrupt lines, their priority and the vector RAM ($n \wedge 7$); the VIA (its timers, port A's pins and CA1, I2C devices on PA0/PA1, SD cards and other devices on its SPI port); the ACIA (Rockwell's, or `--acia wdc`); the YM2151 (its timers, its busy time, and with `--sound` its sound); a DS1747 in U7 (rtc); the reset button (Ctrl-A r); `--clock 7.15909` for J8 (with a `build.js --clock 2 build`); and the Vera X in slot 0 (`--vera`). Not modelled: the analog side (the audio path, the serial line's levels), wait states (V1 has none), and DMA.

Wiring the Vera X to the Hydra-16 through a bus breakout card

How to connect the VERA X 6.1 (Joe Burks's, the 2x13 header) to the Hydra-16 with a **HydraBusBreakoutCard** in **slot 0**, two glue chips on a small protoboard, and wires. Until the carrier card (the plan's option A: [design/plans/VIDEO.md](#); the card in [hardware.md](#)) exists, this is how the driver (vid) meets the real chip.

Everything here comes from the board's own schematics (board/, read only): netlists exported with KiCad 9's `kicad-cli` from board/HydraBusBreakoutCard/HydraBusBreakoutCard.kicad_sch and board/hydra-16.kicad_sch, the VERA's gateway (`c:\source\vera-module, fpga/source/top.v`), the OtterX's VERA connector and bus decoder (which hosts the same card), and the W65C02S data sheet.

What you need

- The VERA X 6.1, a VGA monitor, and its cable.
- A HydraBusBreakoutCard.
- A small protoboard with a **2x13 female header** (0.1") for the VERA X, and:
 - **U1: 74AHCT138** (a 3-to-8 decoder): the VERA's chip select.
 - **U2: 74ACT00** (four 2-input NANDs): its read and write strobes.
 - (Optional) **U3: 74AHCT245** (an 8-bit bus transceiver): full 5 V data levels (below).
 - A 0.1 µF capacitor at each chip, and 10 µF across the VERA's power pins.
- Dupont or wire-wrap wire, kept short (15 cm or less for the bus).

The plan

- **Slot 0** (the main board's J12). Its two I/O port selects are ports 2 and 3, \$FF20-\$FF3F, the VERA's 32 registers; its IRQ A is IRQ line 2, which vid owns; its audio pair goes into the mixer's left and right. In any other slot the address, the interrupt line and the mixer channel would all be different.
- **No buffers on the bus side.** The breakout card is only headers: each slot pin to a header pin. The slot's data lines are on the system data bus itself.
- **Glue logic.** The VERA wants CS#, RD# and WR# (active low). The bus has R/W (RWB) and PHI2, and an I/O page select; the strobes are made from them as the X16 and the OtterX make them.

First: two labels on the breakout card are wrong

The breakout card's connector symbol and the main board's slot symbol disagree on two pins. The main board drives what its own schematic says:

Breakout header	Its label	What slot 0 really carries	So
J11 pin 3	+5VA	-5 V (slot pin 9)	Never use it as a 5 V supply
J7 pin 5	!RWB	/DMAB (slot pin 23)	It's not an inverted R/W: don't use it for the strobes

The other differences between the two symbols are only their point of view (an output of the board is an input of the card).

The breakout card's headers, as slot 0 drives them

Only the pins this wiring uses:

Header	Pin	Signal	Header	Pin	Signal
J2 (Data Bus)	1	D7	J3 (Address Bus)	9	A7
	2	D6		10	A6
	3	D5		11	A5
	4	D4		12	A4
	5	D3		13	A3
	6	D2		14	A2
	7	D1		15	A1
	8	D0		16	A0
J5 (Interrupts & I/O)	2	nIO_S: the \$FFxx page, low only while PHI2 is high	J6 (Clocks)	4	PHI2
	3	nIOB_S: port 3, \$FF30-\$FF3F	J7 (Bus Control)	4	RWB (high: a read)
	4	nIOA_S: port 2, \$FF20-\$FF2F	J8 (Reset)	1	RESB (low: reset)
	6	IRQA: IRQ line 2 (3.3K pull-up on the board)	J9 (Audio)	2	SND_CR: mixer right
J11 (Power)	4	+5 V		3	SND_CL: mixer left
	5	GND			

(J3 pin 1 is A15, down to A0 at pin 16. nIO_S is the main board's \$FFxx decode ORed with PHI1, so it's low only in the second half of a cycle, while PHI2 is high; the board's 74LS154, U19, splits it into 16-byte ports by A4-A7.)

The glue logic

U1, 74AHCT138: CS# for \$FF20-\$FF3F. Enabled by nIO_S, it decodes A5-A7: output Y1 is low for A7 A6 A5 = 0 0 1.

U1 pin	Function	Connect to
1	A0	J3 pin 11 (A5)
2	A1	J3 pin 10 (A6)
3	A2	J3 pin 9 (A7)
4	/E1	J5 pin 2 (nIO_S)
5	/E2	GND
6	E3	+5 V
8	GND	GND
14	/Y1	VERA pin 13 (CS#)
16	VCC	+5 V, 0.1 μF to GND

Leave the other outputs (7, 9-13, 15) unconnected.

U2, 74ACT00: RD# and WR#. RD# = NOT (PHI2 AND RWB): low while PHI2 is high in a read. WR# = NOT (PHI2 AND NOT RWB): low while PHI2 is high in a write. Gate 2 is the inverter.

U2 pin	Function	Connect to
1	1A	J6 pin 4 (PHI2)
2	1B	J7 pin 4 (RWB)
3	1Y	VERA pin 18 (RD#)
4	2A	J7 pin 4 (RWB)
5	2B	J7 pin 4 (RWB)
6	2Y	U2 pin 10 (NOT RWB)
7	GND	GND
8	3Y	VERA pin 15 (WR#)

U2 pin	Function	Connect to
9	3A	J6 pin 4 (PHI2)
10	3B	U2 pin 6
11	4Y	unconnected
12, 13	4A, 4B	GND (an unused CMOS input never floats)
14	VCC	+5 V, 0.1 μ F to GND

Use the ACT/AHCT families as listed: their inputs take the TTL-level signals of the board's LS and F parts, and they're fast, which the write strobe needs (Timing, below).

The VERA X's 2x13 header, pin by pin

Pin 1 is SCL; odd pins on one row, even on the other (the OtterX's VERA connector; VIDEO.md's table).

VERA pin	Signal	Connect to
1	SCL	unconnected (the VERA doesn't need it)
2	SDA	unconnected
3	+5 V	J11 pin 4 (+5 V)
4	GND	J11 pin 5 (GND)
5	D7	J2 pin 1
6	D6	J2 pin 2
7	D5	J2 pin 3
8	D4	J2 pin 4
9	D3	J2 pin 5
10	D2	J2 pin 6
11	D1	J2 pin 7
12	D0	J2 pin 8
13	CS#	U1 pin 14
14	RES#	J8 pin 1 (RESB)
15	WR#	U2 pin 8
16	IRQ#	J5 pin 6 (IRQA; or through an open-collector buffer, below)
17	A4	J3 pin 12
18	RD#	U2 pin 3
19	A2	J3 pin 14
20	A3	J3 pin 13
21	A0	J3 pin 16
22	A1	J3 pin 15
23	GND	GND
24	GND	GND
25	Audio left	J9 pin 3 (SND_CL)
26	Audio right	J9 pin 2 (SND_CR)

So the VERA's registers 0-31 are \$FF20-\$FF3F, VERA_BASE in include/hw.inc; the reset makes its FPGA configure itself again (vid looks for the card for 0.3 s after one).

Optional: 5 V data levels (U3, 74AHCT245)

The VERA X takes 5 V inputs (the X16's VERA module does it with an SN74CBTD3861 bus switch) but drives 3.3 V highs. The W65C02S's data inputs are specified high at 0.7 VDD, **3.5 V**, so a 3.3 V high is 0.2 V short of the data sheet. The X16 and the OtterX run this way, as a CMOS input's real threshold is near 2.5 V, so try without it first; if the card is found unreliably or reads come back wrong, put a 74AHCT245 between the VERA's data pins and the bus (its TTL inputs take the 3.3 V, its outputs give 5 V):

U3 pin	Function	Connect to
1	DIR	J7 pin 4 (RWB: high, A to B, the VERA to the bus)
2-9	A1-A8	VERA pins 12, 11, 10, 9, 8, 7, 6, 5 (D0-D7)
10	GND	GND
11-18	B8-B1	J2 pins 1-8 (D7 at pin 11, down to D0 at pin 18)
19	/OE	U1 pin 14 (CS#)
20	VCC	+5 V, 0.1 μ F to GND

With it, the VERA's data pins go to U3's A side only, not to J2.

Power, grounds, wires

- **Power:** J11 pin 4 (+5 V) to VERA pin 3 and the chips; J11 pin 5 (GND) to VERA pins 4, 23 and 24 (all three) and the chips. Use a heavier wire for the ground than for the signals: the whole data bus returns through it. 10 μ F and 0.1 μ F across the VERA's pins 3 and 4, at the header.
- **Never J11 pin 3** (-5 V, whatever the card says).
- Keep the bus wires short and together; run a ground wire next to PHI2. Plug and unplug with the power off.
- The VERA X's SD-card header and its I2C pins stay unconnected; its VGA output goes to the monitor.

Audio

The VERA's line outputs go into the mixer's slot 0 inputs: 10K resistors into an LF353 on ± 12 V, wired as an inverting summer (its inputs at a virtual ground), beside the YM2151. A ground-centred line output connects straight in. Before wiring them, measure pins 25 and 26 against GND with the card running: if they sit at a DC level (a DAC biased at half its supply) rather than about 0 V, put a 10 μ F capacitor in series with each (its + toward the VERA if the level is positive), or the mixer will amplify the offset too.

The mixer is a unity-gain summer (10K in, 10K feedback, for each of the slots' pairs and the YM2151's), so the VERA comes out as loud as its line output is, which may be louder than the YM2151. If so, a divider in each line evens them (10K in series and 4.7K to GND at the mixer's side: about a third), as the carrier card's plan has it.

The interrupt

Slot 0's IRQ A is its alone, with a 3.3K pull-up on the board into the 74LS148, so the VERA's `IRQ#` can drive it directly: low for an interrupt, high (3.3 V) or let go otherwise. The carrier card's plan puts an open-collector buffer in the way, so the card only ever pulls the line low; on the bench a 74LS07 gate (VERA pin 16 to its input, its output to J5 pin 6) does the same, if the line misbehaves.

Timing (why the strobes are made so)

- **The VERA latches a write's data when its write strobe ends:** `bus_write = !CS# && !WR#`, data captured on its falling edge (`top.v`). The 65C02S holds write data only **10 ns** after PHI2 falls (`tDHW`). So `WR#` must end the write, promptly: it's

PHI2 through one 74ACT00 gate, rising 3-9 ns after PHI2 falls. (CS# comes through the board's decode and ends later: as the write's end it would miss the data.)

- **Reads:** the 65C02S needs data 10 ns before PHI2 falls (tDSR). CS# falls some 15-20 ns after PHI2 rises (nIO_S from the board's 74F32, then U1), and the VERA's data follows within tens of ns. PHI2 is high about 140 ns at 3.58 MHz and 70 ns at 7.16 MHz: room at both.
- **No glitches:** nIO_S is low only while PHI2 is high, when the address is steady, so CS# can't pulse while it changes.
- **The slot's own port selects instead** (CS# = J5 pin 4 AND J5 pin 3, one gate of a 74AHCT08, in place of U1) also work, at 3.58 MHz: they come through the board's 74LS154, 25-35 ns more, which leaves too little of a 7.16 MHz cycle. U1 is as many chips and works at both speeds.

Bringing it up

1. **The glue board alone** (no VERA on it), the breakout card in slot 0, the Hydra on at 3.58 MHz: check +5 V and GND at the VERA socket's pins 3 and 4. With a logic probe or a scope, watch CS# (U1 pin 14), RD# (U2 pin 3) and WR# (U2 pin 8) while something touches \$FF20-\$FF3F: in BASIC, 10 POKE 65313,85: X=PEEK(65313): GOTO 10, RUN (Ctrl-C stops it). CS# should pulse on every access, WR# on the POKE, RD# on the PEEK, each only while PHI2 is high.
2. **The VERA X**, power off, then on: the console comes up on the monitor (vid found the card; cons shows the windows on both the screen and the serial terminal). cat /dev/vid/ctl starts vera 47.0.2 (the gateway's version).
3. **Try it:** echo border 4 >/dev/vid/ctl (the border's colour), cat /lib/font/cp437 >/dev/vid/font (the PC's characters), echo note 8 69 >/dev/sndctl (an A from the PSG, through the mixer; echo off 8 >/dev/sndctl), play a WAV file (the PCM).
4. **No /dev/vid** (no card found): check +5 V, the data lines' order (VERA pin 5 is D7), A0-A4, CS#, RD# and WR# (each active low), and RES# (it must be high once the reset is over). Then U3, for the 3.3 V highs.
5. **Then 7.16 MHz**, once everything works at 3.58 (the board's faster clock, and a build with --clock 2).

[programming/video.md](#) is the programmer's side: the driver's files, claims, the PSG and the PCM.